

Experiment 1

Title

Responsive Web Page using Bootstrap Grid System

Problem Statement

Create a basic web page containing a header, navigation bar, content section, and footer. Use the Bootstrap grid system (rows and columns) and breakpoints to make the page responsive.

Objectives

After completing this experiment, students will be able to:

- Understand responsive web design.
- Use Bootstrap grid system.
- Apply rows and columns in webpage layout.
- Use responsive breakpoints such as col-sm, col-md, and col-lg.
- Create structured layouts that adapt to different screen sizes.

Theory

Bootstrap is a popular front-end framework used for designing responsive and mobile-first websites. It provides a **grid system based on 12 columns** that allows developers to structure page layouts easily.

Key elements include:

- **Container** – used to wrap the webpage content.
- **Row** – horizontal group of columns.
- **Column** – vertical section used to divide content.
- **Breakpoints** – predefined screen sizes that help adjust layout for mobile, tablet, and desktop devices.

Using Bootstrap's grid system ensures that webpages automatically adjust according to screen size.

Procedure:

1. Start the HTML document.
2. Include the Bootstrap CDN link.
3. Create a container for the webpage layout.
4. Add a header section with a website title.
5. Create a navigation bar with menu items (Home, About, Services, Contact).
6. Create a content section using Bootstrap rows and columns.
7. Apply responsive breakpoints (col-sm, col-md, col-lg).
8. Add a footer section with copyright text.
9. Save and run the webpage in a browser.
- 10.

Program / Code:

index.html

```
<!DOCTYPE html>

<html>
<head>
  <title>My Website</title>
  <link rel="stylesheet" href="index.css">
</head>
<body>
<!-- Header -->
<div class="header">
```

```

    <h1>My Website</h1>
</div>
<!-- Navigation Bar -->
<div class="navbar">
    <div class="logo">Website</div>
    <div class="menu">
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Services</a>
        <a href="#">Contact</a>
    </div>
</div>
<!-- Content -->
<div class="container">
    <div class="column">
        <h2>Column 1</h2>
        <p>This is the first column. On small devices the column occupies full width, while on medium and large devices it occupies half width using Bootstrap grid classes.</p>
    </div>
    <div style="border-left: 3px solid black; height: 200px;"></div>
    <div class="column">
        <h2>Column 2</h2>
        <p>This is the second column. Bootstrap grid system helps to create responsive layout.</p>
    </div>
</div>
<!-- Footer -->
<div class="footer">
    Copyright © 2026 My Website
</div>
</body>
</html>

```

index.css

```

body{

    margin:0;
    font-family: Arial, sans-serif;
}

/* Header */
.header{
    background:#2d7be5;
    color:white;
    text-align:center;
    padding:25px;
    font-size: 25px;
}

```

```
/* Navbar */
.navbar{
  background:#222;
  color:white;
  display:flex;
  justify-content:space-between;
  padding:10px 20px;
  font-size: 25px;
}

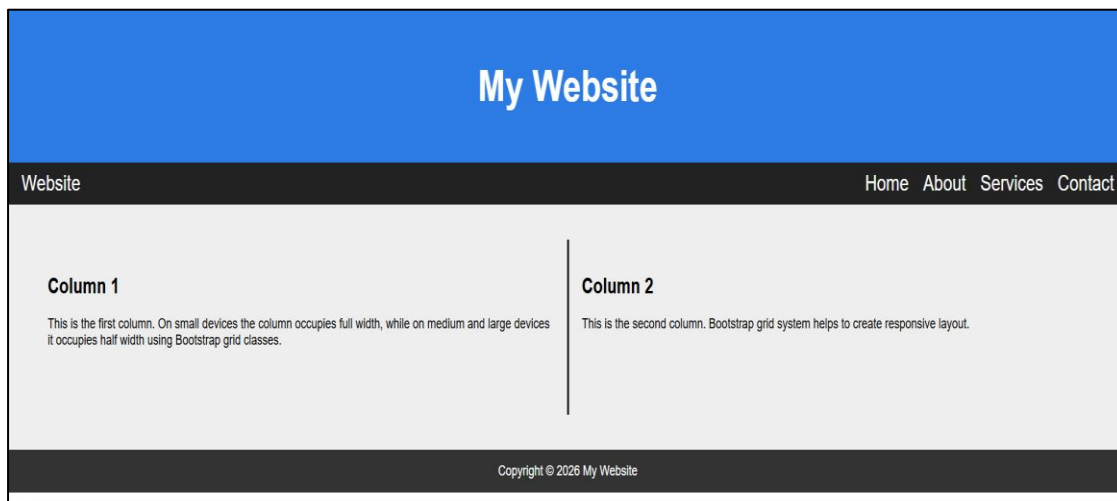
.navbar a{
  color:white;
  text-decoration:none;
  margin-left:15px;
}

/* Container */
.container{
  display:flex;
  padding:40px;
  background:#eee;
}

/* Columns */
.column{
  flex:1;
  padding:20px;
}

/* Footer */
.footer{
  background:#333;
  color:white;
  text-align:center;
  padding:15px;
}
```

Expected Output:



Outcome

A responsive webpage is created successfully using the Bootstrap grid system that adjusts properly on mobile, tablet, and desktop screens.

Experiment 2

Title

Using Bootstrap UI Components for Web Page Design

Problem Statement

Design a webpage using Bootstrap components such as navbar, cards, badges, buttons, images, and modal.

Objectives

After completing this experiment, students will be able to:

- Use Bootstrap navbar for navigation.
- Design responsive card layouts.
- Use Bootstrap badges and buttons.
- Implement modal pop-up components.
- Build structured and visually appealing web interfaces.

Theory

Bootstrap provides various **UI components** that help developers build professional web interfaces quickly. These components are pre-styled and responsive.

Important components include:

- **Navbar** – navigation menu for websites.
- **Cards** – content containers used to display products or information.
- **Buttons** – interactive elements used for actions.
- **Badges** – highlight labels such as “New” or “Sale”.
- **Modals** – popup windows used to display detailed information.

Using Bootstrap components improves development speed and ensures responsive design.

Procedure:

1. Create an HTML webpage.
2. Add the Bootstrap CDN link.
3. Create a responsive navbar with the brand name **TechStore**.
4. Add navigation links (Home, Products, About, Contact).
5. Create a product section using Bootstrap cards.

6. Display at least three product cards with image, description, price, and badge.
7. Add a “View Details” button to open a modal popup.
8. Implement a modal to display product details.
9. Add a footer section.
10. Run the webpage and verify responsiveness.

Program Code:

Index.html

```
<!DOCTYPE html>

<html>
<head>
<title>TechStore</title>
<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<!-- Internal CSS -->
<style>
body{
background:#f8f9fa;
}

.card:hover{
transform: scale(1.05);
transition: 0.3s;
}

footer{
margin-top:50px;
}
</style>

</head>
<body>

<!-- NAVBAR -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
<div class="container">
<a class="navbar-brand" href="#">TechStore</a>
<button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#menu">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="menu">
<ul class="navbar-nav ms-auto">
<li class="nav-item"><a class="nav-link" href="#">Home</a></li>
<li class="nav-item"><a class="nav-link" href="#">Products</a></li>
<li class="nav-item"><a class="nav-link" href="#">About</a></li>
```

```

<li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
</ul>
</div>
</div>
</nav>

<!-- PRODUCTS -->
<div class="container mt-5">
<div class="row">

<!-- Card 1 -->
<div class="col-lg-4 col-md-6 col-12 mb-4">
<div class="card">

<div class="card-body">
<h5>Laptop</h5>
<p>High performance laptop</p>
<h6>₹50,000 <span class="badge bg-success">New</span></h6>
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#m1">View Details</button>
</div>
</div>
</div>

<!-- Card 2 -->
<div class="col-lg-4 col-md-6 col-12 mb-4">
<div class="card">

<div class="card-body">
<h5>Mobile</h5>
<p>Latest smartphone</p>
<h6>₹30,000 <span class="badge bg-danger">Sale</span></h6>
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#m2">View Details</button>
</div>
</div>
</div>

<!-- Card 3 -->
<div class="col-lg-4 col-md-6 col-12 mb-4">
<div class="card">

```

```


<div class="card-body">
<h5>Headphones</h5>
<p>Noise cancelling</p>
<h6>₹5,000 <span class="badge bg-success">New</span></h6>
<button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#m3">View
Details</button>
</div>
</div>
</div>

<!-- MODALS -->

<div class="modal fade" id="m1">
<div class="modal-dialog">
<div class="modal-content">
<div class="modal-header">
<h5>Laptop Details</h5>
<button class="btn-close" data-bs-dismiss="modal"></button>
</div>
<div class="modal-body">
16GB RAM, SSD, Fast performance
</div>
<div class="modal-footer">
<button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
</div>
</div>
</div>
</div>

<div class="modal fade" id="m2">
<div class="modal-dialog">
<div class="modal-content">
<div class="modal-header">
<h5>Mobile Details</h5>
<button class="btn-close" data-bs-dismiss="modal"></button>
</div>
<div class="modal-body">
High camera quality smartphone
</div>
<div class="modal-footer">
<button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
</div>

```

```

</div>
</div>
</div>

<div class="modal fade" id="m3">
<div class="modal-dialog">
<div class="modal-content">
<div class="modal-header">
<h5>Headphones Details</h5>
<button class="btn-close" data-bs-dismiss="modal"></button>
</div>
<div class="modal-body">
Best sound quality with bass
</div>
<div class="modal-footer">
<button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
</div>
</div>
</div>
</div>

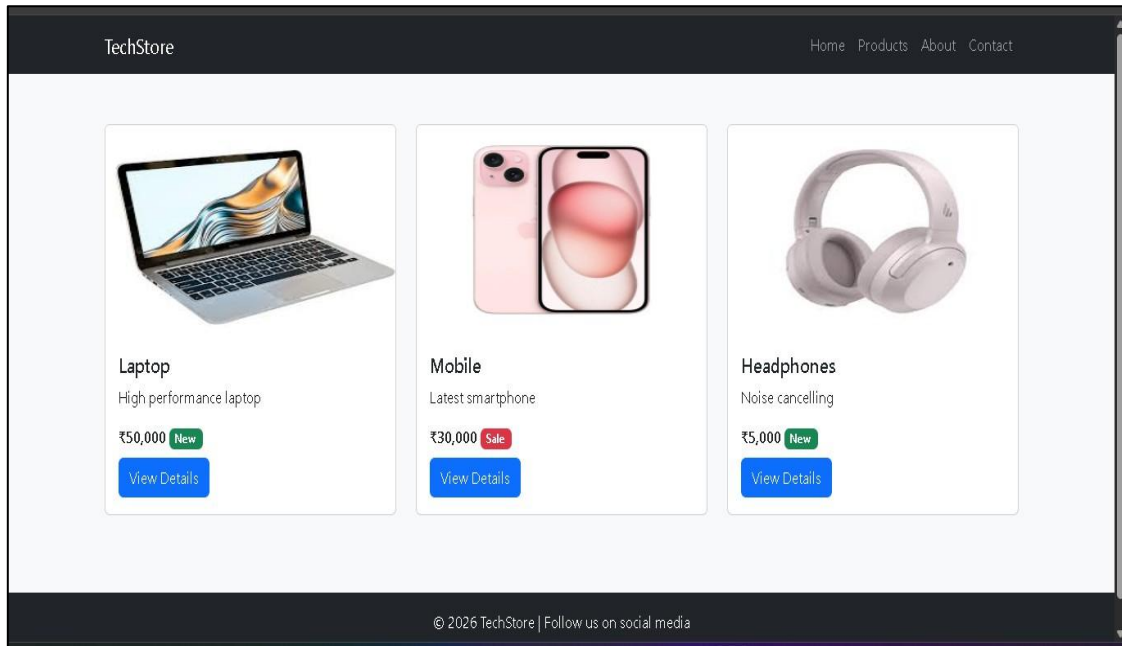
<!-- FOOTER -->
<footer class="bg-dark text-white text-center p-3">
<p>© 2026 TechStore | Follow us on social media</p>
</footer>

<!-- Bootstrap JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></scri
pt>

</body>
</html>

```

Expected Output:



Outcome

A visually appealing and responsive webpage using Bootstrap UI components is created successfully.

Experiment 3

Title

Mobile-First Responsive Design using Tailwind CSS

Problem Statement

Create a mobile-first webpage using Tailwind CSS utility classes for layout, spacing, typography, and colors. Apply responsive breakpoints to make the design adaptable to different screen sizes.

Objectives

After completing this experiment, students will be able to:

- Understand the concept of mobile-first design.
- Use Tailwind CSS utility classes.
- Apply spacing, typography, and color utilities.
- Use responsive prefixes (sm, md, lg).
- Create modern responsive landing pages.

Theory

Tailwind CSS is a **utility-first CSS framework** that allows developers to design interfaces using small reusable classes.

Examples of Tailwind utilities:

- **Spacing:** p-4, m-2, gap-4
- **Typography:** text-xl, font-bold
- **Layout:** flex, grid
- **Responsive design:** sm:, md:, lg:

Tailwind follows a **mobile-first approach**, meaning styles are first designed for small screens and then enhanced for larger screens.

Procedure:

1. Create an HTML webpage.
2. Include Tailwind CSS using CDN.

3. Create a header with the brand name **FitLife**.
4. Add navigation links for Home, Programs, Trainers, and Contact.
5. Create a hero section with heading, description, and CTA button.
6. Add a services section with three cards.
7. Apply responsive grid layout using Tailwind classes.
8. Ensure mobile layout shows one column.
9. Apply responsive classes for medium and large screens.
10. Add a footer section.

Program Code:

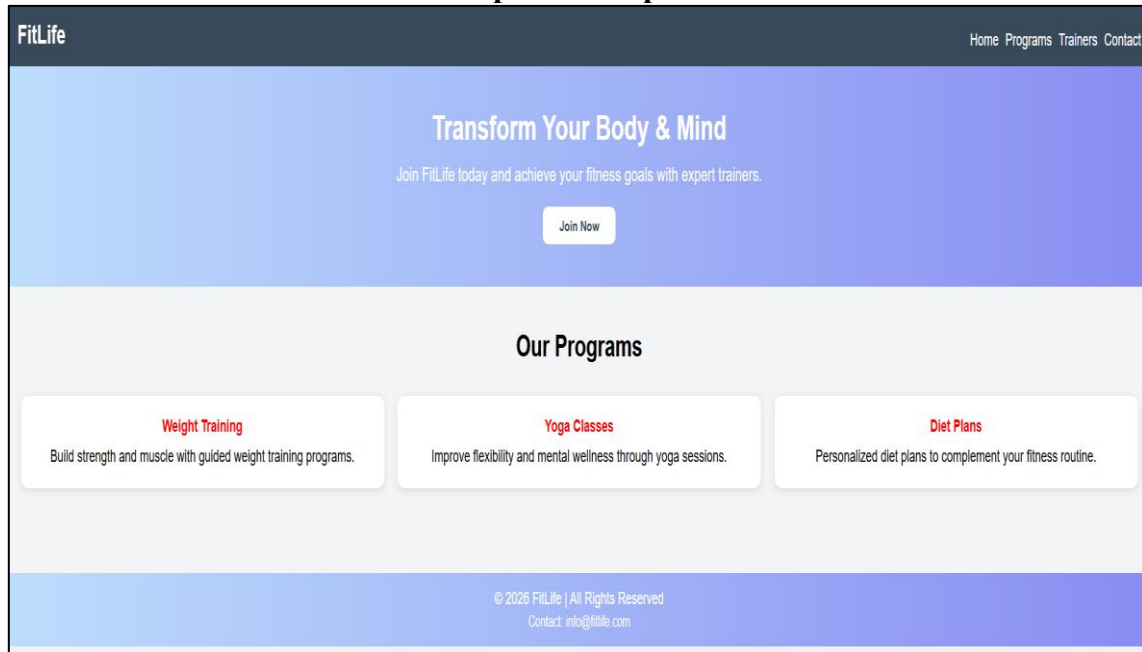
index.html

```
<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="UTF-8">
<title>FitLife</title>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<!-- Tailwind CSS CDN -->
<script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-200 font-sans">
<!-- HEADER -->
<header class="bg-gradient-to-r from-slate-600 to-indigo-400 text-white p-4">
<div class="flex justify-between items-center">
<h1 class="text-xl font-bold">FitLife</h1>
<nav class="space-x-4 text-sm">
<a href="#" class="hover:text-gray-200">Home</a>
<a href="#" class="hover:text-gray-200">Programs</a>
<a href="#" class="hover:text-gray-200">Trainers</a>
<a href="#" class="hover:text-gray-200">Contact</a>
</nav>
</div>
</header>
<!-- HERO SECTION -->
<section class="bg-gradient-to-r from-sky-200 to-indigo-400 text-white text-center py-16">
<h2 class="text-4xl font-bold mb-4">
Transform Your Body & Mind
</h2>
<p class="mb-6 text-lg">
Join FitLife today and achieve your fitness goals with expert trainers.
</p>
<button class="bg-gray-200 text-gray-700 px-6 py-2 rounded-md font-semibold hover:bg-white">
Join Now
</button>
```

```
</section>
<!-- PROGRAM SECTION -->
<section class="py-12 px-6">
  <h3 class="text-3xl font-bold text-center mb-10">
    Our Programs
  </h3>
  <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
    <!-- Card 1 -->
    <div class="bg-gray-100 p-6 rounded-lg shadow text-center">
      <h4 class="text-red-600 font-bold mb-2">
        Weight Training
      </h4>
      <p>
        Build strength and muscle with guided weight training programs.
      </p>
    </div>
    <!-- Card 2 -->
    <div class="bg-gray-100 p-6 rounded-lg shadow text-center">
      <h4 class="text-red-600 font-bold mb-2">
        Yoga Classes
      </h4>
      <p>
        Improve flexibility and mental wellness through yoga sessions.
      </p>
    </div>
    <!-- Card 3 -->
    <div class="bg-gray-100 p-6 rounded-lg shadow text-center">
      <h4 class="text-red-600 font-bold mb-2">
        Diet Plans
      </h4>
      <p>
        Personalized diet plans to complement your fitness routine.
      </p>
    </div>
  </div>
</section>
<!-- FOOTER -->
<footer class="bg-gradient-to-r from-sky-300 to-indigo-400 text-white text-center py-6">
  <p>© 2026 FitLife | All Rights Reserved</p>
  <p class="text-sm mt-1">
    Contact: info@fitlife.com
  </p>
</footer>
</body>
</html>
```

Expected Output:



Outcome

A clean and responsive landing page is created successfully using Tailwind CSS.

Experiment 4

Title

JavaScript Program using Variables, Conditions, and Loops

Problem Statement

Create a JavaScript program that takes user input and displays output using conditions and loops.

Objectives

After completing this experiment, students will be able to:

- Understand variables and data types.
- Use loops for repetitive operations.
- Apply conditional statements.
- Accept user input and display results dynamically.

Theory

JavaScript is a scripting language used to add interactivity to webpages.

Key programming concepts include:

- **Variables** – used to store data values.
- **Loops** – used to repeat tasks multiple times.
- **Conditional statements** – used to make decisions in a program.
- **User input** – data entered by the user through input fields or prompts.

These concepts allow developers to create interactive web applications.

Procedure

1. Start the program.
2. Take the number of subjects as input from the user.
3. Use a loop to enter marks for each subject.
4. Calculate the total marks.
5. Calculate the average marks.

6. Use conditional statements to determine the grade.
7. Display the total, average, grade, and result on the webpage.
8. End the program.

Program Code:

Index.html

```
<!DOCTYPE html>

<html>
<head>
<title>Student Result Calculator</title>
<link rel="stylesheet" href="/src/index.css">
</head>
<body>
<div class="container">
<h2>Student Result Calculator</h2>
<label>Enter Number of Subjects</label><br>
<input type="number" id="subjects"><br><br>
<button onclick="calculateResult()">Calculate Result</button>
<div id="result"></div>
</div>
<script src="/src/main.jsx"></script>
</body>
</html>
```

Index.css

```
body{

font-family: Arial;
background-color:#f2f2f2;
text-align:center;
padding:40px;
}

.container{
background:white;
padding:25px;
width:400px;
margin:auto;
border-radius:10px;
box-shadow:0px 0px 10px gray;
}

input{
padding:8px;
width:80%;
margin-top:5px;
}

button{
padding:10px 20px;
```

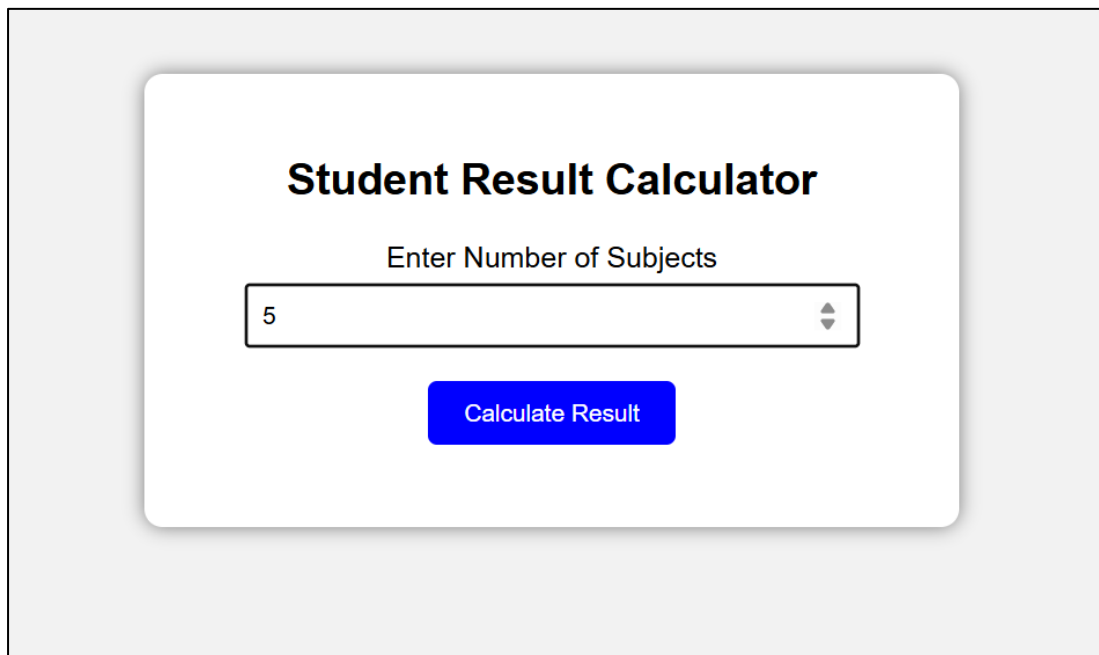
```
background-color:blue;
color:white;
border:none;
border-radius:5px;
cursor:pointer;
}
button:hover{
background-color:darkblue;
}
#result{
margin-top:20px;
font-weight:bold;
font-size:18px;
}
```

```
main.jsx
function calculateResult(){

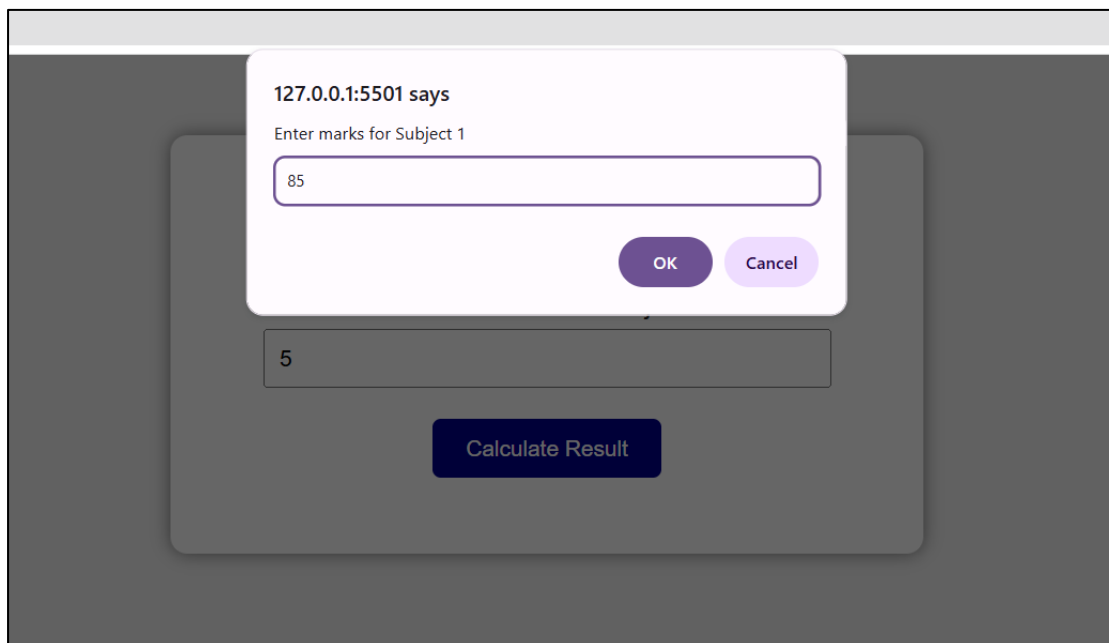
let n = document.getElementById("subjects").value;
let total = 0;
for(let i=1;i<=n;i++){
let marks = parseFloat(prompt("Enter marks for Subject " + i));
total = total + marks;
}
let average = total / n;
let grade;
let result;
if(average >= 90){
grade = "A+";
}
else if(average >= 75){
grade = "A";
}
else if(average >= 60){
grade = "B";
}
else if(average >= 50){
grade = "C";
}
else{
grade = "F";
}
if(average >= 40){
result = "PASS";
}
else{
result = "FAIL";
}
document.getElementById("result").innerHTML =
```

```
"Total Marks: " + total + "<br>" +  
"Average Marks: " + average.toFixed(2) + "<br>" +  
"Grade: " + grade + "<br>" +  
"Result: " + result;  
}
```

Expected Output:



The image shows a web application titled "Student Result Calculator". It features a text input field labeled "Enter Number of Subjects" with the value "5". Below the input field is a blue button labeled "Calculate Result".



The image shows a modal dialog box titled "127.0.0.1:5501 says" with the text "Enter marks for Subject 1". The dialog contains a text input field with the value "85" and two buttons: "OK" and "Cancel". Below the dialog, the "Student Result Calculator" form is visible, showing the "Enter Number of Subjects" field with the value "5" and the "Calculate Result" button.

The screenshot shows a web application titled "Student Result Calculator". It features a text input field labeled "Enter Number of Subjects" with the value "5" entered. Below the input field is a blue button labeled "Calculate Result". Underneath the button, the results are displayed in bold black text: "Total Marks: 385", "Average Marks: 77.00", "Grade: A", and "Result: PASS". The entire interface is centered within a light gray rectangular frame.

Outcome

The program successfully processes user input, calculates total and average marks, and displays the grade and result dynamically.

Experiment 5

Title

Array and Object Operations using JavaScript

Problem Statement

Store employee details using arrays and objects and perform operations such as displaying records, filtering employees based on salary, and calculating salary statistics.

Objectives

After completing this experiment, students will be able to:

- Use arrays to store multiple records.
- Use objects to store structured data.
- Perform filtering operations.
- Calculate total and average salary.
- Display dynamic data on webpages.

Theory

In JavaScript:

- **Arrays** store multiple values in a single variable.
- **Objects** represent real-world entities using key-value pairs.

Example object:

Employee

Name

Salary

Department

Array methods such as map(), filter(), and forEach() help process data efficiently.

Procedure

1. Create an empty array to store employees.
2. Take employee details from the user.
3. Store each employee as an object inside the array.
4. Provide a button to display all employees.
5. Filter employees with salary greater than ₹50,000.
6. Calculate total salary payout.
7. Calculate average salary.
8. Count employees in a specific department.
9. Display the results dynamically.

Program Code:

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Employee Salary Management</title>
  <link rel="stylesheet" href="/src/index.css">
</head>
<body>
<div class="container">
  <h2>Employee Salary Management System</h2>
  <input type="text" id="name" placeholder="Employee Name"><br>
  <input type="text" id="empId" placeholder="Employee ID"><br>
  <input type="number" id="salary" placeholder="Salary"><br>
  <input type="text" id="dept" placeholder="Department"><br>
  <button onclick="addEmployee()">Add Employee</button><br>
  <button onclick="displayEmployees()">Display All</button>
  <button onclick="filterSalary()">Salary > 50000</button>
  <button onclick="totalSalary()">Total Salary</button>
  <button onclick="averageSalary()">Average Salary</button><br>
  <button onclick="countDepartment()">Count by Department</button>
  <div id="output"></div>
</div>
<script src="/src/main.jsx"></script>
</body>
</html>
```

Index.css

```
body {
  font-family: Arial;
  background-color: #eef2f3;
  text-align: center;
```

```

}
.container {
  width: 650px;
  margin: 40px auto;
  padding: 20px;
  background: white;
  border-radius: 10px;
  box-shadow: 0 0 10px gray;
}
input {
  padding: 8px;
  margin: 5px;
}
button {
  padding: 8px 12px;
  margin: 5px;
  background-color: #28a745;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
button:hover {
  background-color: #1e7e34;
}
#output {
  margin-top: 20px;
  text-align: left;
}
main.jsx
let employees = [];

function addEmployee() {
  let name = document.getElementById("name").value;
  let empId = document.getElementById("empId").value;
  let salary = parseFloat(document.getElementById("salary").value);
  let dept = document.getElementById("dept").value;
  if (name === "" || empId === "" || isNaN(salary) || dept === "") {
    alert("Please fill all fields properly");
    return;
  }
  let employee = {
    name: name,
    id: empId,
    salary: salary,
    department: dept
  };
  employees.push(employee);
  alert("Employee Added Successfully!");
}

```

```

document.getElementById("name").value = "";
document.getElementById("empId").value = "";
document.getElementById("salary").value = "";
document.getElementById("dept").value = "";
}
function displayEmployees() {
  let output = "<h3>All Employees</h3>";
  employees.forEach(emp => {
    output += `
      Name: ${emp.name} |
      ID: ${emp.id} |
      Salary: ₹${emp.salary} |
      Dept: ${emp.department} <br>`;
  });
  document.getElementById("output").innerHTML = output;
}
function filterSalary() {
  let filtered = employees.filter(emp => emp.salary > 50000);
  let output = "<h3>Employees with Salary > ₹50,000</h3>";
  filtered.forEach(emp => {
    output += `
      Name: ${emp.name} |
      Salary: ₹${emp.salary} <br>`;
  });
  document.getElementById("output").innerHTML = output;
}
function totalSalary() {
  let total = employees.reduce((sum, emp) => sum + emp.salary, 0);
  document.getElementById("output").innerHTML =
    "<h3>Total Salary Payout: ₹" + total + "</h3>";
}
function averageSalary() {
  if (employees.length === 0) {
    document.getElementById("output").innerHTML =
      "<h3>No employee records available</h3>";
    return;
  }
  let total = employees.reduce((sum, emp) => sum + emp.salary, 0);
  let avg = total / employees.length;
  document.getElementById("output").innerHTML =
    "<h3>Average Salary: ₹" + avg.toFixed(2) + "</h3>";
}
function countDepartment() {
  let deptName = prompt("Enter Department Name:");
  let count = employees.filter(emp => emp.department === deptName).length;
  document.getElementById("output").innerHTML =
    "<h3>Employees in " + deptName + ": " + count + "</h3>";
}
}

```

Expected Output:

Employee Salary Management System

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

127.0.0.1:5502 says
Employee Added Successfully!
OK

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

Employee Salary Management System

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

All Employees

Name: AAA | ID: 100 | Salary: ₹10000 | Dept: CSE
Name: BBB | ID: 101 | Salary: ₹20000 | Dept: CSE
Name: CCC | ID: 103 | Salary: ₹30000 | Dept: AIML
Name: DDD | ID: 104 | Salary: ₹40000 | Dept: DS

Employee Salary Management System

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

Employees with Salary > ₹50,000

Employee Salary Management System

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

Total Salary Payout: ₹1000000

Employee Salary Management System

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

Average Salary: ₹25000.00

127.0.0.1:5502 says

Enter Department Name:

OK

Cancel

Add Employee

Display All

Salary > 50000

Total Salary

Average Salary

Count by Department

Employees in CSE: 2

Employee Salary Management System

Employees in CSE: 2

Outcome

Employee records are successfully stored, processed, filtered, and displayed using arrays and objects.

Experiment 6

Title

DOM Manipulation and Event Handling using JavaScript

Problem Statement

Create buttons and input fields that dynamically change webpage content using JavaScript events.

Objectives

After completing this experiment, students will be able to:

- Understand DOM manipulation.
- Use JavaScript events.
- Modify webpage content dynamically.
- Handle user interactions effectively.

Theory

The **Document Object Model (DOM)** represents the structure of an HTML document.

JavaScript can manipulate the DOM to:

- Change text content
- Modify styles
- Add or remove elements
- Respond to user events

Common events include:

- onclick
- onmouseover
- onchange
- addEventListener()

Procedure

1. Create a webpage with heading, paragraph, and input field.
2. Add buttons for different actions.
3. Capture user input from the text field.
4. Use JavaScript event listeners for button clicks.
5. Change heading text dynamically.
6. Change background color and font size.
7. Show or hide paragraph content.
8. Reset the page content.

Program Code:

Index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Content Changer</title>
  <link rel="stylesheet" href="/src/index.css">
</head>
<body>
<div class="container">
  <h2 id="mainHeading">Welcome to JavaScript Lab</h2>
  <input type="text" id="userInput" placeholder="Enter new heading text">
  <br><br>
  <button id="changeTextBtn">Change Heading</button>
  <button id="bgColorBtn">Change Background</button>
  <button id="fontSizeBtn">Increase Font Size</button>
  <button id="toggleBtn">Show/Hide Paragraph</button>
  <button id="resetBtn">Reset</button>
  <p id="paragraph">
    This is a sample paragraph. JavaScript events can dynamically change content.
  </p>
</div>
<script src="/src/main.jsx"></script>
</body>
</html>
```

Index.css:

```
body {
  font-family: Arial;
  background-color: #f4f4f4;
  text-align: center;
}
.container {
  width: 600px;
  margin: 50px auto;
  padding: 20px;
```

```
    background: white;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
  }
  button {
    padding: 8px 12px;
    margin: 5px;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
  }
  button:hover {
    background-color: #0056b3;
  }
  input {
    padding: 8px;
    width: 250px;
  }
}
```

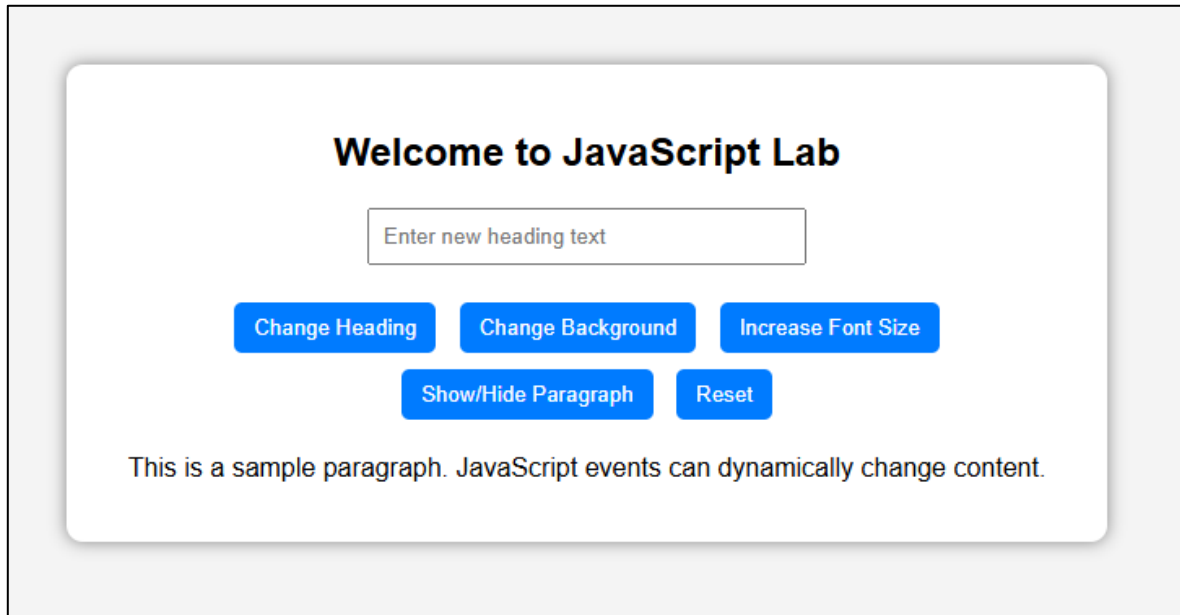
main.jsx

```
let heading = document.getElementById("mainHeading");

let paragraph = document.getElementById("paragraph");
let input = document.getElementById("userInput");
let fontSize = 16;
// Change heading text (onclick using addEventListener)
document.getElementById("changeTextBtn").addEventListener("click", function () {
  if (input.value !== "") {
    heading.innerHTML = input.value;
  }
});
// Change background color
document.getElementById("bgColorBtn").onclick = function () {
  document.body.style.backgroundColor =
    "#" + Math.floor(Math.random() * 16777215).toString(16);
};
// Increase font size
document.getElementById("fontSizeBtn").addEventListener("click", function () {
  fontSize += 2;
  paragraph.style.fontSize = fontSize + "px";
});
// Show/Hide paragraph
document.getElementById("toggleBtn").addEventListener("click", function () {
  if (paragraph.style.display === "none") {
    paragraph.style.display = "block";
  } else {
    paragraph.style.display = "none";
  }
});
```

```
}  
});  
// Reset page  
document.getElementById("resetBtn").addEventListener("click", function () {  
    heading.innerHTML = "Welcome to JavaScript Lab";  
    paragraph.style.display = "block";  
    paragraph.style.fontSize = "16px";  
    document.body.style.backgroundColor = "#f4f4f4";  
    input.value = "";  
    fontSize = 16;  
});
```

Expected Output:



The screenshot displays a web application interface. At the top, there is a heading "Welcome to JavaScript Lab" in a bold, black font. Below the heading is a text input field with the placeholder text "Enter new heading text". Underneath the input field are five blue buttons with white text: "Change Heading", "Change Background", "Increase Font Size", "Show/Hide Paragraph", and "Reset". At the bottom of the interface, there is a paragraph of text: "This is a sample paragraph. JavaScript events can dynamically change content." The entire content is centered within a light gray background.

JavaScript is a scripting language used to make web pages interactive and dynamic.

JavaScript is a scripting language used to

Change Heading

Change Background

Increase Font Size

Show/Hide Paragraph

Reset

This is a sample paragraph. JavaScript events can dynamically change content.

JavaScript is a scripting language used to make web pages interactive and dynamic.

JavaScript is a scripting language used to

Change Heading

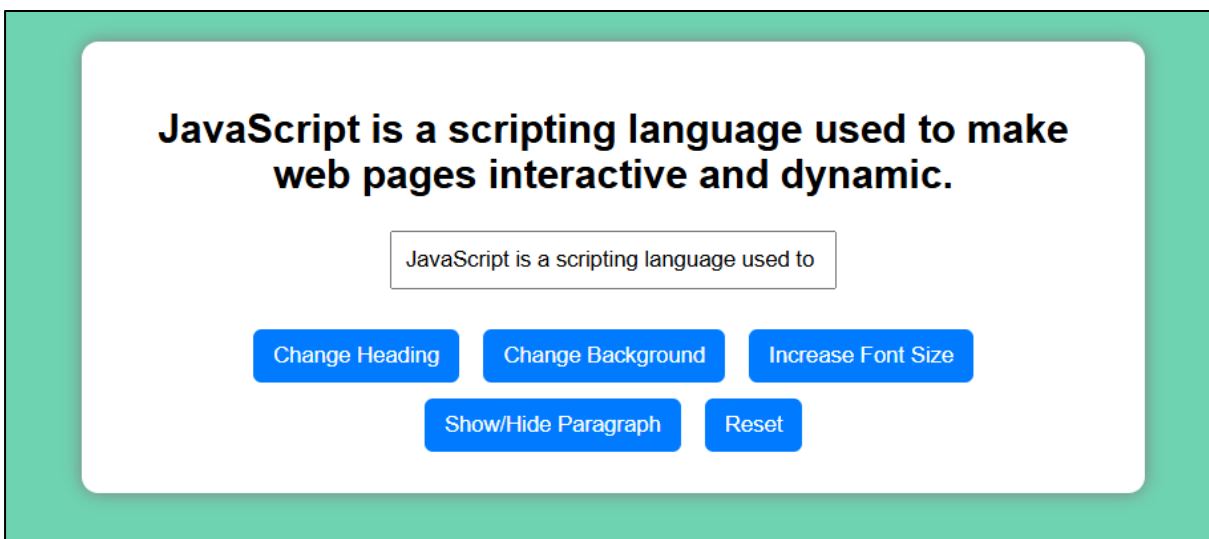
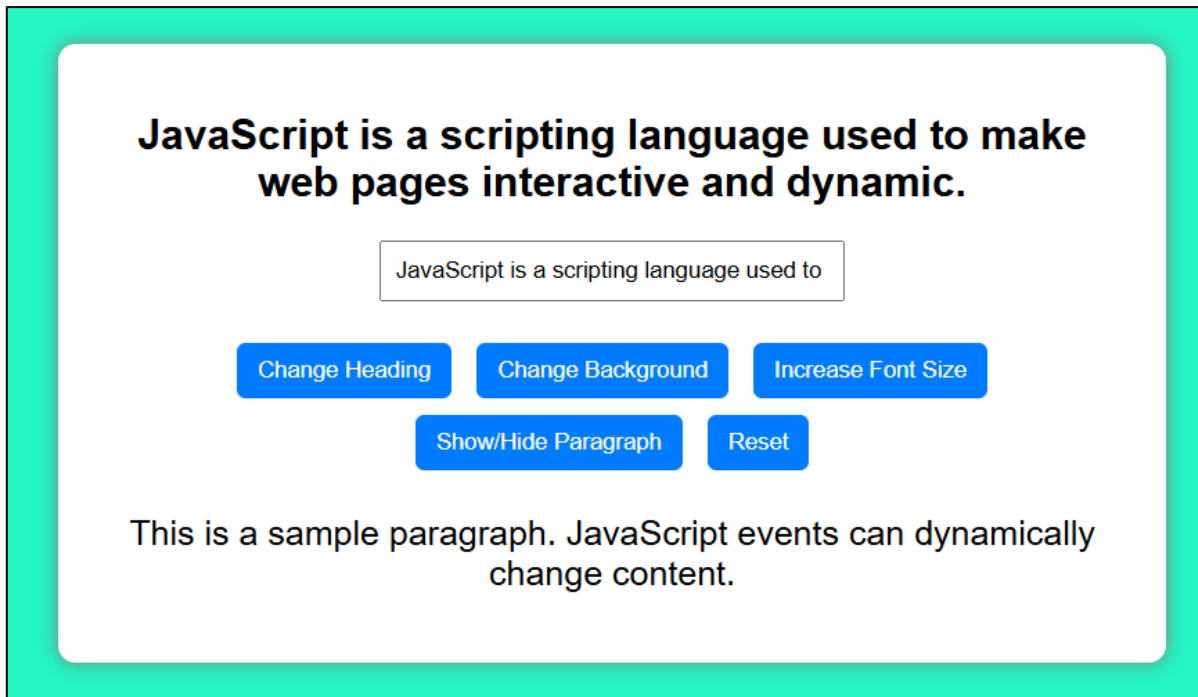
Change Background

Increase Font Size

Show/Hide Paragraph

Reset

This is a sample paragraph. JavaScript events can dynamically change content.



Outcome

The webpage dynamically updates content and styles based on user interactions without reloading.

Experiment 7

Title:

Creating React Components using JSX and Props

Problem Statement:

Create a React application with multiple components and pass data using props.

Objectives:

After completing this experiment, students will be able to:

- Understand React component structure.
- Use JSX syntax.
- Create functional components.

- Pass data using props.
- Display dynamic content.

Theory:

React is a JavaScript library used for building user interfaces.

Key concepts include:

- **Components** – reusable building blocks of React applications.
- **JSX** – syntax that combines JavaScript and HTML.
- **Props** – used to pass data from parent components to child components.

Components make applications modular and reusable.

Procedure:

1. Create a new React application.
2. Create a functional component named **Student**.
3. Accept props such as name, course, and marks.
4. Display student details using JSX.
5. Use the Student component multiple times in the App component.
6. Pass different values using props.
7. Run the application.

Program Code:

Index.html

```
<!DOCTYPE html>

<html>
<head>
  <title>React Student Component</title>
  <!-- React CDN -->
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
  <!-- Babel CDN -->
  <script src="https://unpkg.com/babel-standalone@6/babel.min.js"></script>
  <!-- CSS -->
  <link rel="stylesheet" href="/src/index.css">
</head>
<body>
<div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

Index.css:

```
body{

  font-family: Arial;
  background-color: #f4f4f4;
  text-align: center;
}

.student-card{
  background: white;
```

```
width: 250px;
padding: 15px;
margin: 15px auto;
border-radius: 10px;
box-shadow: 0px 0px 10px gray;
}
.student-card h2{
  color: #2c3e50;
}
.student-card p{
  font-size: 16px;
}
```

App.jsx:

```
import Student from "./Student";
function App() {
  return (
    <div>
      <h1>Student Information</h1>
      <Student
        name="Rahul Sharma"
        course="Computer Science"
        marks="85" />
      <Student
        name="Anita Verma"
        course="Information Technology"
        marks="92" />
      <Student
        name="Rohan Gupta"
        course="Electronics"
        marks="78" />
    </div>
  );
}
export default App;
```

Student.jsx:

```
function Student(props) {

  return (
    <div className="student-card">
      <h2>{props.name}</h2>
      <p>Course: {props.course}</p>
      <p>Marks: {props.marks}</p>
    </div>
  );
}
export default Student;
```

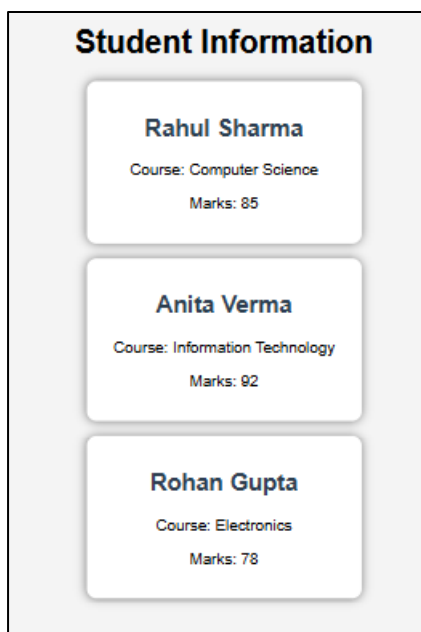
```

main.jsx:
import React from "react";

import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

Expected Output:



Outcome:

Reusable React components successfully display dynamic student information.

Experiment 8

Title:

State Management using useState Hook

Problem Statement:

Develop a React counter application using the useState hook and event handling.

Objectives:

After completing this experiment, students will be able to:

- Understand React hooks.
- Manage state using useState.
- Handle user events.
- Update UI dynamically.

Theory:

In React, **state** represents data that can change over time.

The useState hook allows functional components to manage state.

Example:

- Counter value stored in state
- UI automatically updates when state changes

This enables interactive applications.

Procedure:

1. Create a React application.
2. Import the useState hook.
3. Create a state variable for counter value.
4. Display the counter value.
5. Add buttons for increment, decrement, and reset.
6. Use event handlers to update state.
7. Render updated values in the UI.

Program Code:

Index.html

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>React Counter App</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

main.jsx

```
import React from "react";

import ReactDOM from "react-dom/client";
import App from "./App";
import "./App.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.jsx:

```
import React, { useState } from "react";

function App() {
  const [count, setCount] = useState(0);
  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };
}
```

```

};
const reset = () => {
  setCount(0);
};
return (
  <div className="container">
    <h1>React Counter Application</h1>
    <h2 className="counter">{count}</h2>
    <div className="buttons">
      <button onClick={increment}>Increment (+)</button>
      <button onClick={decrement}>Decrement (-)</button>
      <button onClick={reset}>Reset</button>
    </div>
  </div>
);
}
export default App;

```

App.css:

```

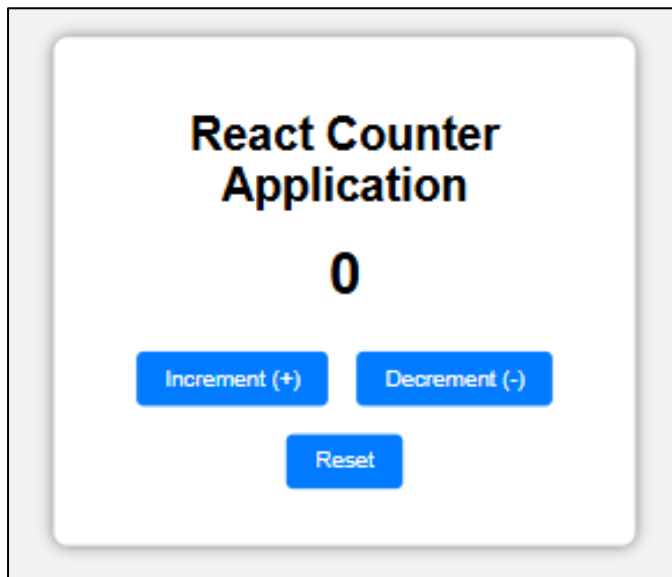
body{

  font-family: Arial, sans-serif;
  background-color:#f2f2f2;
}
.container{
  background:white;
  width:350px;
  margin:100px auto;
  padding:30px;
  text-align:center;
  border-radius:10px;
  box-shadow:0px 0px 10px gray;
}
.counter{
  font-size:40px;
  margin:20px 0;
}
.buttons button{
  margin:10px;
  padding:10px 20px;
  font-size:16px;
  border:none;
  background-color:#007BFF;
  color:white;
  border-radius:5px;
  cursor:pointer;
}
.buttons button:hover{
  background-color:#0056b3;
}

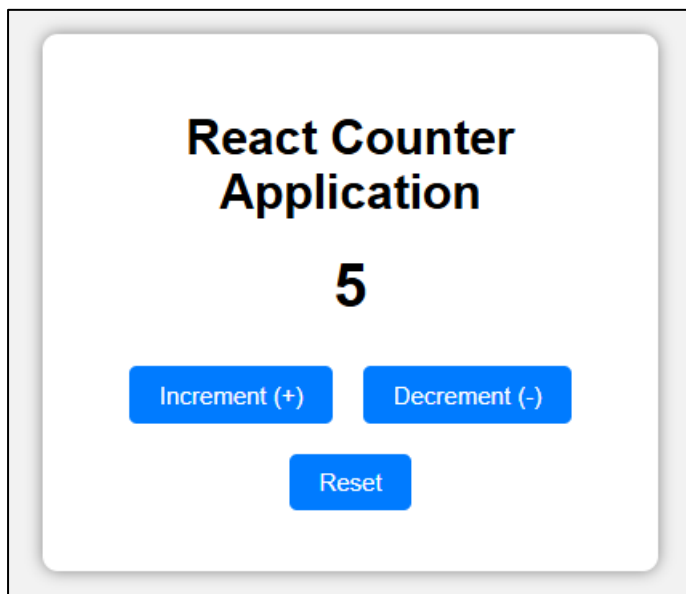
```

```
}
```

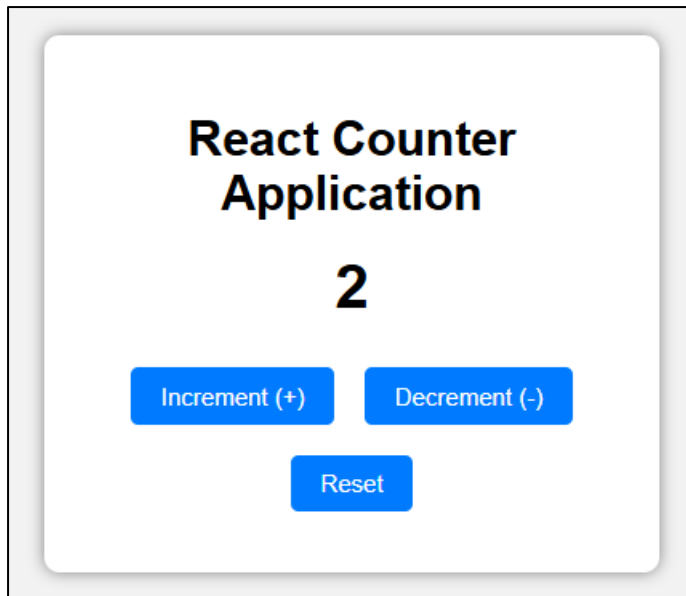
Expected Output:



After Increment:



After Decrement:

**Outcome:**

The React counter application successfully updates the UI dynamically based on state changes.

Experiment 9**Title:**

Fetching API Data using useEffect in React

Problem Statement:

Fetch data from an API using useEffect and display it in a component.

Objectives:

After completing this experiment, students will be able to:

- Use controlled components in React.
- Manage form inputs using useState.
- Perform form validation.
- Handle form submission events.
- Display dynamic error messages.

Theory:

The useEffect hook is used to perform side effects in React components, such as:

- Fetching API data
- Updating the DOM
- Managing subscriptions

It runs automatically when the component loads or when dependencies change.

This hook is commonly used for **API data fetching** in React applications.

Procedure:

1. Create a React application.
2. Import useState and useEffect.
3. Create state variables for form inputs and errors.
4. Design a registration form with name, email, and password fields.
5. Validate user inputs.
6. Display error messages if validation fails.
7. Fetch API data using useEffect.
8. Display the API data in the component.

9. Show success message after valid submission.

Program Code:

Index.html

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>React Registration Form</title>
</head>
<body>
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

main.jsx

```
import React from "react";

import ReactDOM from "react-dom/client";
import App from "./App";
import "./App.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

App.css

```
body{

  font-family: Arial;
  background:#f4f4f4;
  text-align:center;
}

.container{
  width:350px;
  margin:auto;
  padding:20px;
  background:white;
  border-radius:8px;
  box-shadow:0 0 10px gray;
}

input{
  width:90%;
  padding:8px;
  margin:8px 0;
}
```

```

button{
  padding:10px 15px;
  background:green;
  color:white;
  border:none;
  cursor:pointer;
}
button:hover{
  background:darkgreen;
}
.error{
  color:red;
  font-size:14px;
}
.success{
  color:green;
  font-weight:bold;
}
.apiData{
  margin-top:20px;
  background:#e8f5e9;
  padding:10px;
}

```

App.jsx

```

import React, { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [errors, setErrors] = useState({});
  const [success, setSuccess] = useState("");
  const [users, setUsers] = useState([]);
  const handleSubmit = (e) => {
    e.preventDefault();
    let formErrors = {};
    if(name.trim() === ""){
      formErrors.name = "Name is required";
    }
    if(!email.includes("@")){
      formErrors.email = "Email must contain @";
    }
    if(password.length < 6){
      formErrors.password = "Password must be at least 6 characters";
    }
    setErrors(formErrors);
    if(Object.keys(formErrors).length === 0){

```

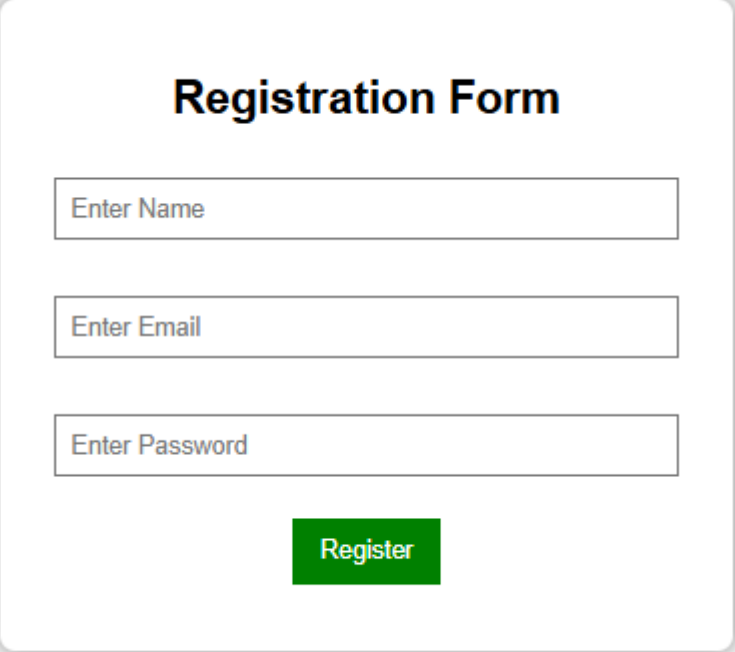
```

const newUser = {
  name: name,
  email: email
};
setUsers([...users, newUser]);
setSuccess("Registration Successful!");
setName("");
setEmail("");
setPassword("");
}
};
return (
  <div className="container">
    <h2>Registration Form</h2>
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        placeholder="Enter Name"
        value={name}
        onChange={(e)=>setName(e.target.value)}
      />
      <p className="error">{errors.name}</p>
      <input
        type="email"
        placeholder="Enter Email"
        value={email}
        onChange={(e)=>setEmail(e.target.value)}
      />
      <p className="error">{errors.email}</p>
      <input
        type="password"
        placeholder="Enter Password"
        value={password}
        onChange={(e)=>setPassword(e.target.value)}
      />
      <p className="error">{errors.password}</p>
      <button type="submit">Register</button>
    </form>
    <p className="success">{success}</p>
    {users.length > 0 && (
      <div className="apiData">
        <h3>Registered Users</h3>
        <ul>
          {users.map((user,index) => (
            <li key={index}>
              {user.name} - {user.email}
            </li>
          ))}
        </ul>
      </div>
    )}
  </div>
);

```

```
    </ul>
  </div>
  )}
</div>
);
}
export default App;
```

Expected Outcome:



The image shows a web form titled "Registration Form". It contains three text input fields stacked vertically, each with a placeholder text: "Enter Name", "Enter Email", and "Enter Password". Below these fields is a green rectangular button with the text "Register" in white. The form is centered on a light gray background.

Registration Form

Enter Name

Enter Email

Enter Password

Register

Registration Successful!

Registered Users

- ABCD - abcd@gmail.com

Outcome

API data is successfully fetched and displayed, and the registration form performs proper validation.

Experiment 10

Title:

Java OOP Concepts Implementation

Problem Statement:

Create Java classes demonstrating **encapsulation, inheritance, and abstraction**.

Create a **Bank Management System** application where:

1. Create an **abstract class BankAccount** with:
 - accountNumber
 - accountHolderName
 - balance
 - Abstract method: calculateInterest()
2. Create two subclasses:
 - SavingsAccount
 - CurrentAccount
3. Implement:
 - **Encapsulation** (private data members with getters and setters)
 - **Inheritance** (SavingsAccount and CurrentAccount extend BankAccount)
 - **Abstraction** (abstract method implemented in subclasses)
4. Create a **main class** to:
 - Create objects
 - Deposit money

- Display account details
- Calculate interest

Objectives:

After completing this experiment, students will be able to:

- Understand **Object-Oriented Programming principles**
- Implement **Encapsulation using private data members**
- Implement **Inheritance using extends keyword**
- Implement **Abstraction using abstract classes**
- Create **real-world object relationships**

Theory:

Object-Oriented Programming (OOP) is a programming paradigm based on objects and classes. It helps in creating modular, reusable, and maintainable software systems.

Encapsulation

Encapsulation means **wrapping data and methods together into a single unit (class)** and restricting direct access to data using **private variables and getter/setter methods**.

Inheritance

Inheritance allows one class to **acquire properties and behaviors of another class** using the **extends** keyword.

Abstraction

Abstraction hides implementation details and shows only essential features using **abstract classes and methods**.

In this experiment, a **BankAccount abstract class** is created and extended by **SavingsAccount and CurrentAccount** classes.

Algorithm:

1. Start the program.
2. Create an abstract class **BankAccount** with variables:
 - accountNumber
 - accountHolderName
 - balance
3. Declare an abstract method **calculateInterest()**.
4. Create a **SavingsAccount class** extending BankAccount.
5. Implement the **calculateInterest()** method.
6. Create a **CurrentAccount class** extending BankAccount.
7. Implement the **calculateInterest()** method.
8. Create a **Main class**.
9. Create objects for SavingsAccount and CurrentAccount.
10. Perform operations:
 - Deposit money
 - Display account details

- Calculate interest

11. Display results on the console.

12. End the program.

Program Code:

Exp10.java

```
// Abstract class
abstract class BankAccount {
    int accountNumber;
    String accountHolderName;
    double balance;
    // Constructor
    BankAccount(int accountNumber, String accountHolderName, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }
    // Deposit Method
    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited: " + amount);
    }
    // Display Account Details
    void displayDetails() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder Name: " + accountHolderName);
        System.out.println("Balance: " + balance);
    }
    // Abstract Method
    abstract void calculateInterest();
}
// SavingsAccount class
class SavingsAccount extends BankAccount {

    SavingsAccount(int accountNumber, String accountHolderName, double balance) {
        super(accountNumber, accountHolderName, balance);
    }
    void calculateInterest() {
        double interest = balance * 0.05; // 5% interest
        System.out.println("Savings Account Interest: " + interest);
    }
}
// CurrentAccount class
class CurrentAccount extends BankAccount {

    CurrentAccount(int accountNumber, String accountHolderName, double balance) {
        super(accountNumber, accountHolderName, balance);
    }
}
```

```

    }
    void calculateInterest() {
        double interest = balance * 0.02; // 2% interest
        System.out.println("Current Account Interest: " + interest);
    }
}
// Main class
public class Exp10 {
    public static void main(String[] args) {
        // Creating objects
        SavingsAccount sa = new SavingsAccount(101, "Rahul", 10000);
        CurrentAccount ca = new CurrentAccount(102, "Anita", 20000);
        System.out.println("----- Savings Account -----");
        sa.deposit(2000);
        sa.displayDetails();
        sa.calculateInterest();
        System.out.println();
        System.out.println("----- Current Account -----");
        ca.deposit(3000);
        ca.displayDetails();
        ca.calculateInterest();
    }
}

```

Expected Outcome:

```

● PS C:\Users\lenovo\Desktop\FSD Project\ABES-TOT-FSD\Java Exp> javac Exp10.java
● PS C:\Users\lenovo\Desktop\FSD Project\ABES-TOT-FSD\Java Exp> java Exp10
----- Savings Account -----
Amount Deposited: 2000.0
Account Number: 101
Account Holder Name: Rahul
Balance: 12000.0
Savings Account Interest: 600.0

----- Current Account -----
● Amount Deposited: 3000.0
Account Number: 102
Account Holder Name: Anita
Balance: 23000.0
Current Account Interest: 460.0
○ PS C:\Users\lenovo\Desktop\FSD Project\ABES-TOT-FSD\Java Exp> █

```

Outcome:

Students successfully implemented **encapsulation, inheritance, and abstraction** concepts in Java by developing a simple **Bank Management System** application.

Experiment 11

Title:

Creating REST API using Spring Boot

Problem Statement:

Create a **Spring Boot application** and expose **REST endpoints** using **@RestController**.
Create a **Student Management REST API** that:

1. Returns a **welcome message**.
2. Returns **student details using GET request**.
3. Accepts **student data using POST request**.

Objectives:

After completing this experiment, students will be able to:

- Create a **Spring Boot project**
- Understand **REST architecture**
- Use **@RestController** annotation
- Create **GET and POST endpoints**
- Test APIs using **browser or Postman**

Theory:

Spring Boot is a Java framework used to build **standalone and production-ready applications** quickly. A **REST API (Representational State Transfer API)** allows communication between client and server using HTTP methods.

Common HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Send new data
PUT	Update existing data
DELETE	Remove data

@RestController

@RestController is used to create RESTful web services. It combines:

- @Controller
- @ResponseBody

This allows methods to return **JSON responses directly**. Spring Boot also provides **auto-configuration**, which reduces manual configuration.

Algorithm:

1. Start the Spring Boot project using **Spring Initializer**.
2. Add required dependencies:

- Spring Web
- 3. Create a main application class.
- 4. Create a **StudentController class**.
- 5. Annotate the class with `@RestController`.
- 6. Create a **GET endpoint** to return a welcome message.
- 7. Create another **GET endpoint** to return student details.
- 8. Create a **POST endpoint** to accept student data.
- 9. Run the Spring Boot application.
- 10. Test endpoints using:
 - Browser
 - Postman
- 11. Verify JSON responses.
- 12. Stop the application.

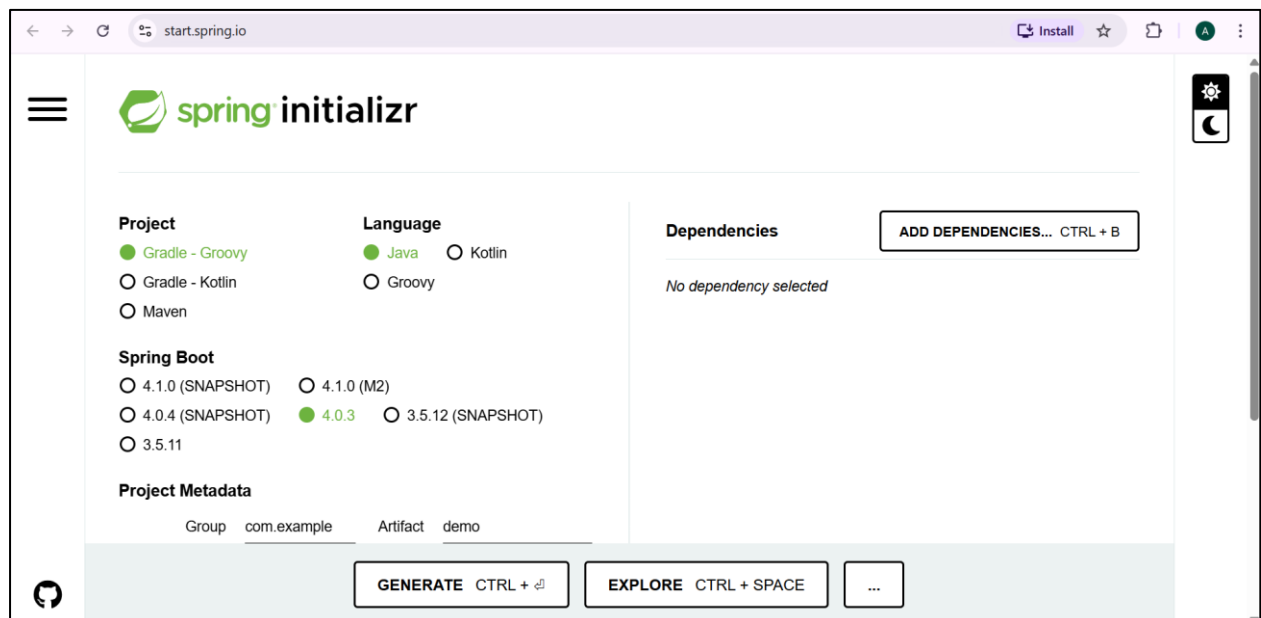
Program Code:

Steps to Create Project using Spring Initializr

Step 1 Open Spring Initializr

Go to:

<https://start.spring.io>



Spring Initializr webpage

Step 2 Configure Project

Project Settings:

Field	Value
Project	Maven

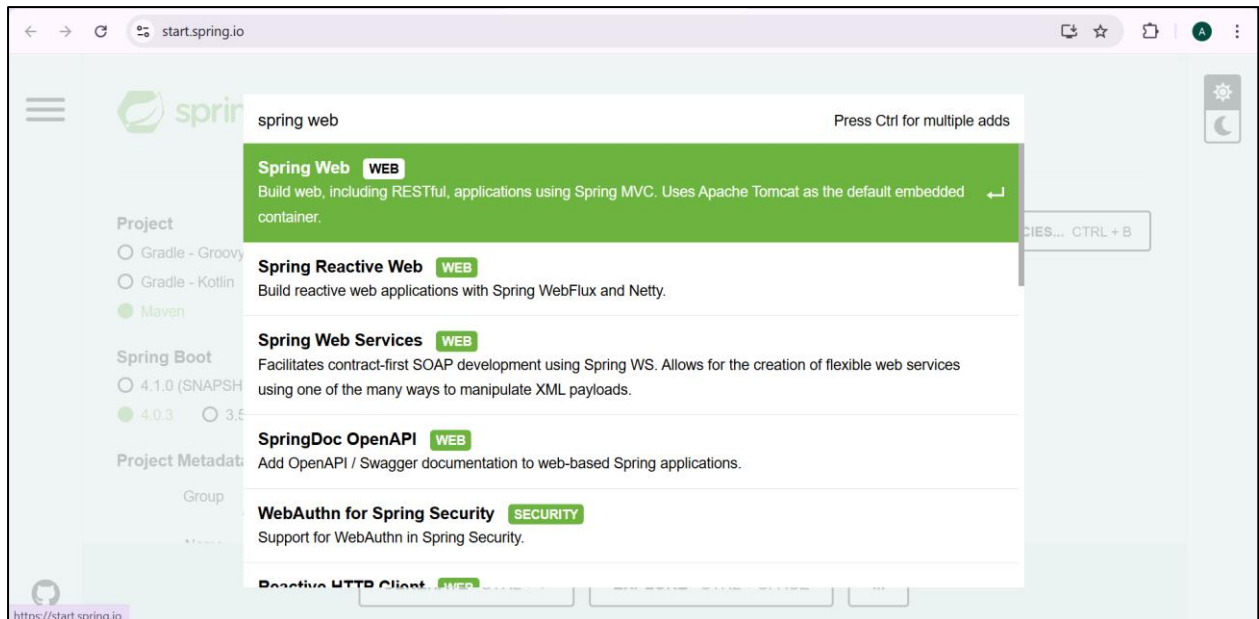
Language Java
Spring Boot Latest stable
Group com.example
Artifact Student-rest-api
Packaging Jar
Java 17

The screenshot shows the Spring Initializr web application at start.spring.io. The interface is divided into several sections:
- **Project:** Radio buttons for Gradle - Groovy, Gradle - Kotlin, and Maven (selected).
- **Language:** Radio buttons for Java (selected), Kotlin, and Groovy.
- **Spring Boot:** Radio buttons for 4.1.0 (SNAPSHOT), 4.1.0 (M2), 4.0.4 (SNAPSHOT), 4.0.3 (selected), 3.5.12 (SNAPSHOT), and 3.5.11.
- **Project Metadata:** Text input fields for Group (com.example), Artifact (demo), and Name (demo).
- **Dependencies:** A section with the text "No dependency selected" and a button "ADD DEPENDENCIES... CTRL + B".
At the bottom, there are three buttons: "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and an ellipsis button.

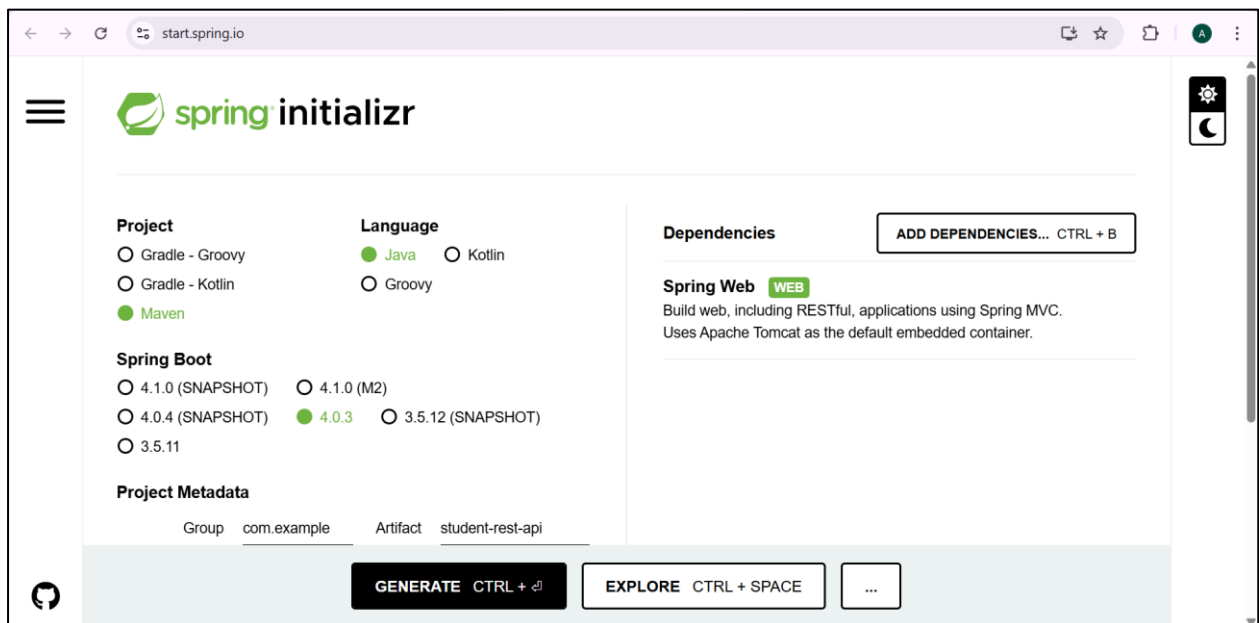
The screenshot shows the second step of the Spring Initializr configuration. The **Spring Boot** section remains the same with 4.0.3 selected. The **Project Metadata** section is expanded to include:
- **Group:** com.example
- **Artifact:** student-rest-api
- **Name:** student-rest-api
- **Description:** Project for Spring Boot
- **Package name:** com.example.studen (partially visible)
- **Packaging:** Radio buttons for Jar (selected) and War.
- **Configuration:** Radio buttons for Properties (selected) and YAML.
The bottom buttons "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and the ellipsis button are still present.

Add Dependency:

Spring Web



Click **Generate** to download project.

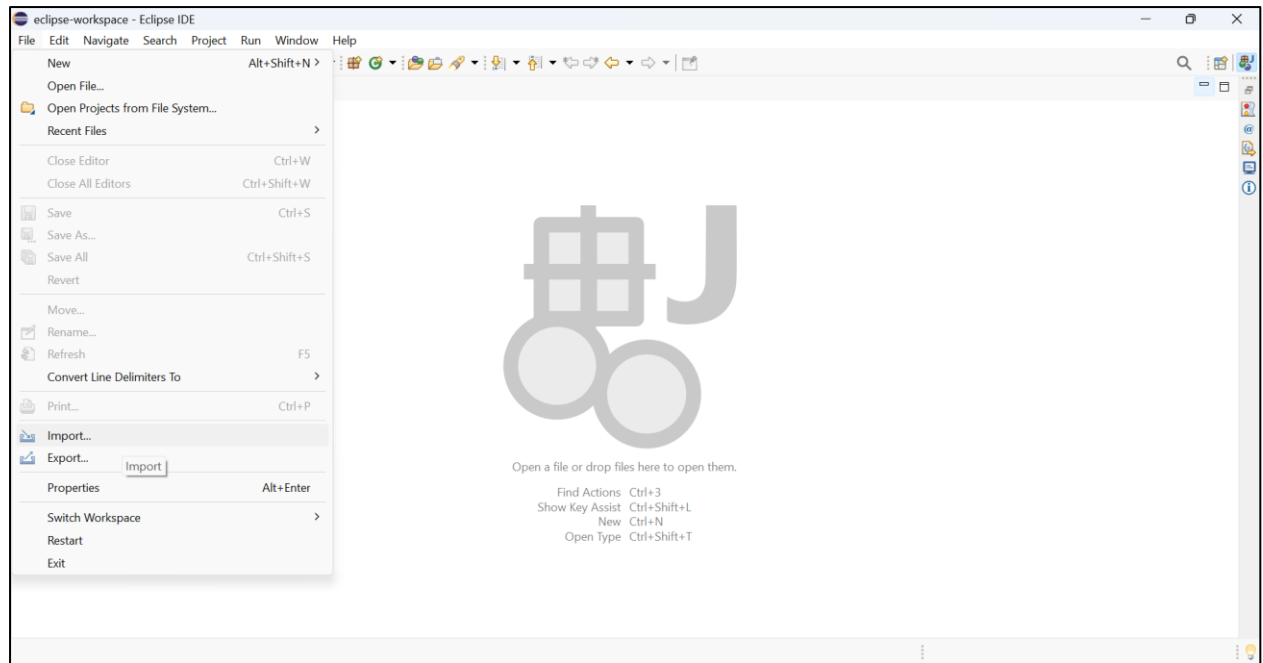


Project configuration page

Step 3 Import Project in Eclipse IDE

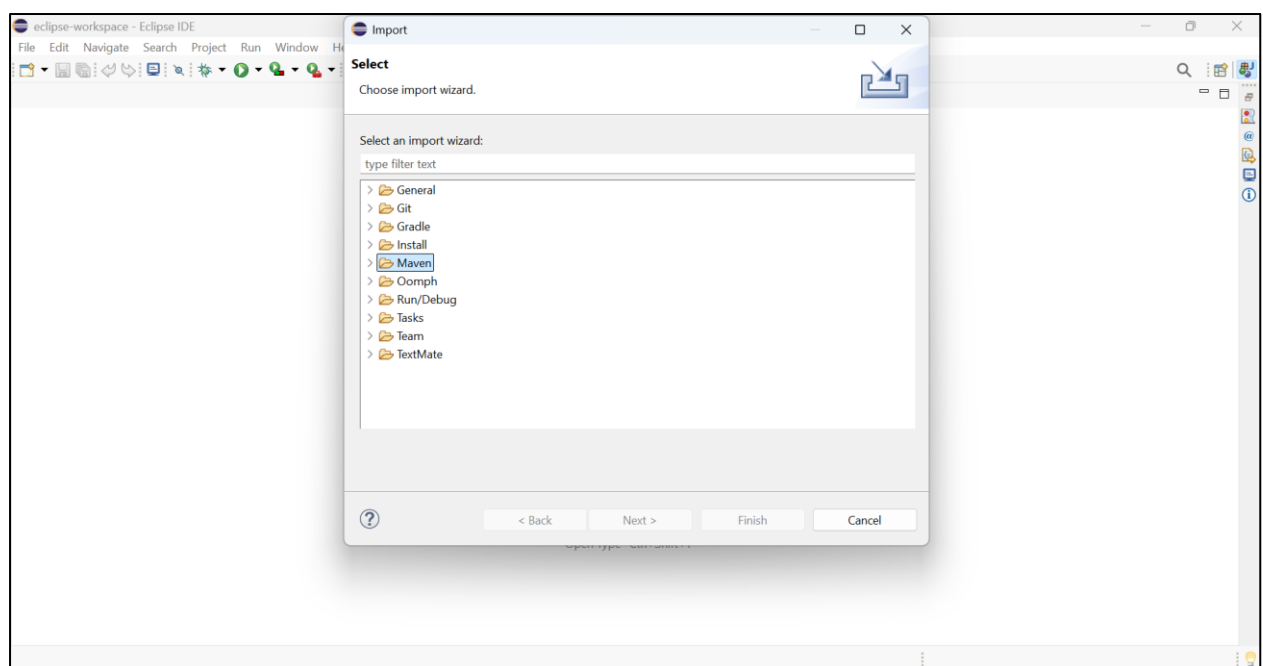
1. Open Eclipse
2. Click

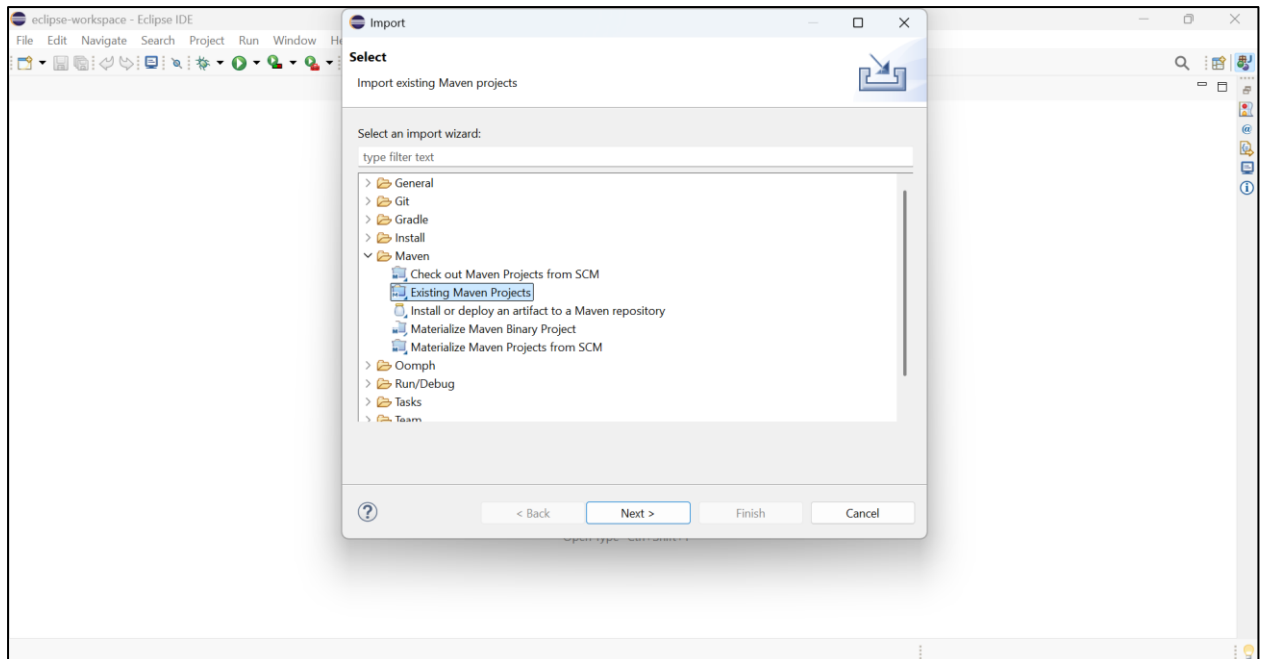
File → Import



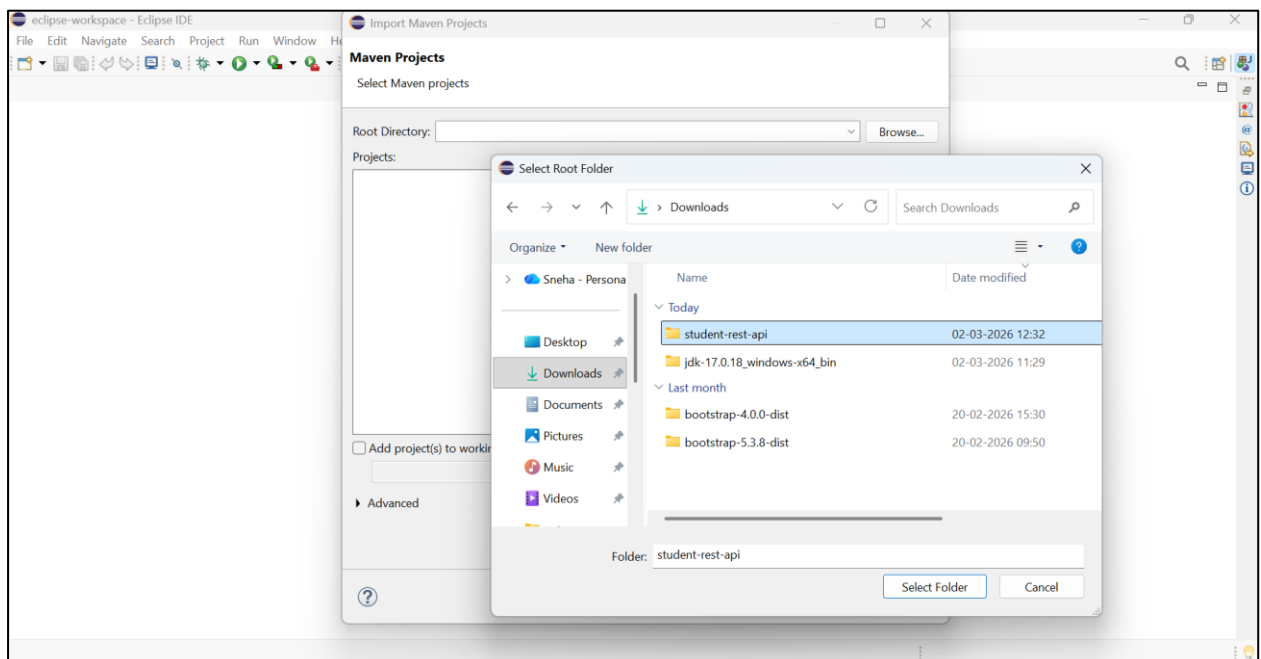
- Select

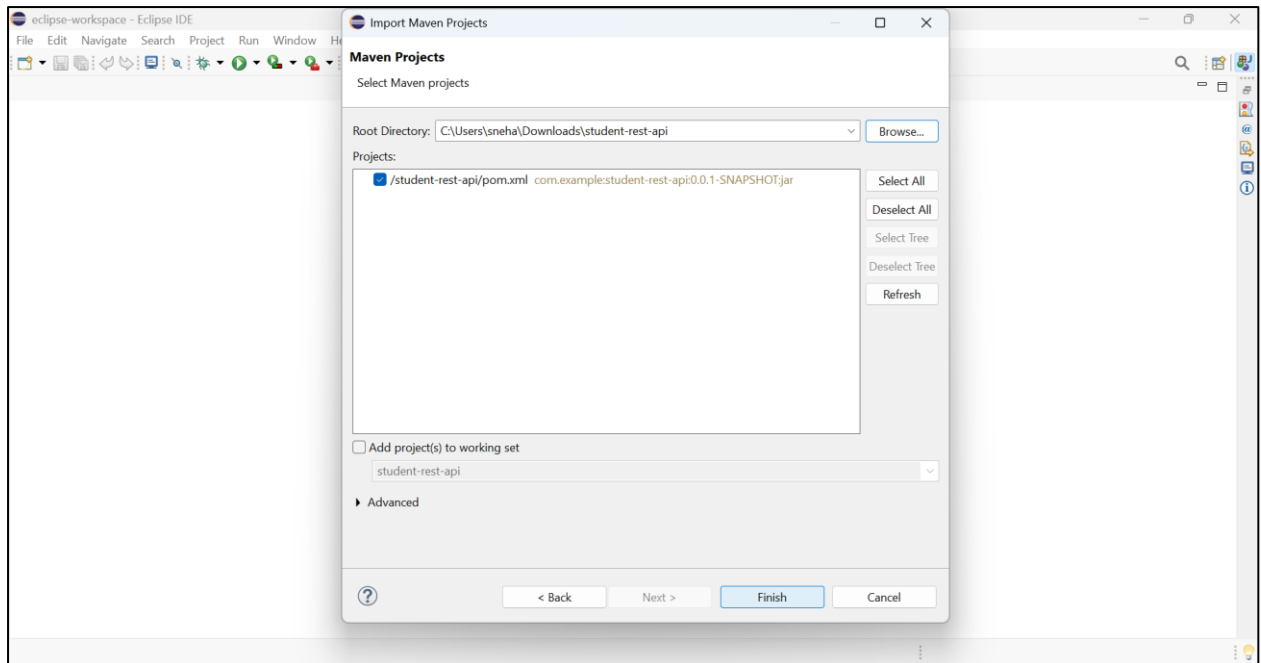
Existing Maven Project





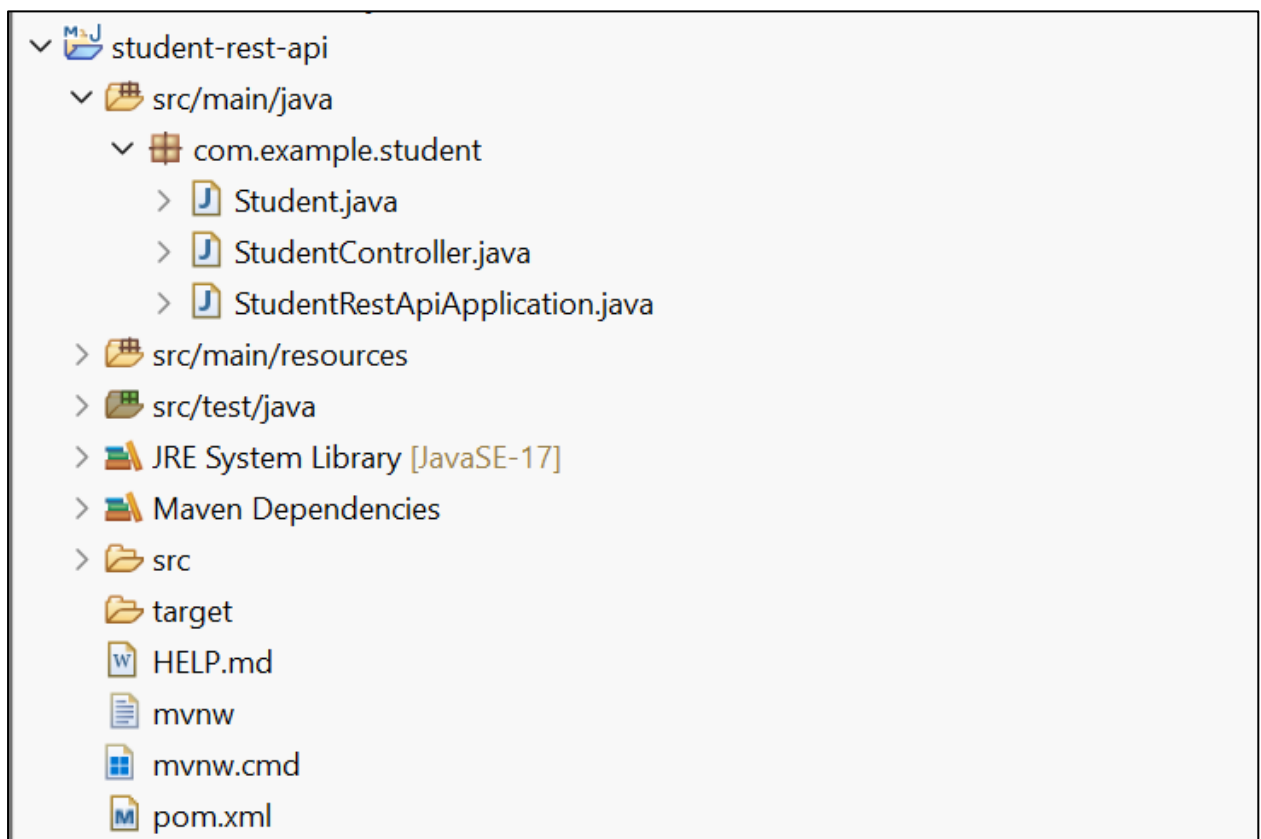
- Select the downloaded folder
- Click **Finish**





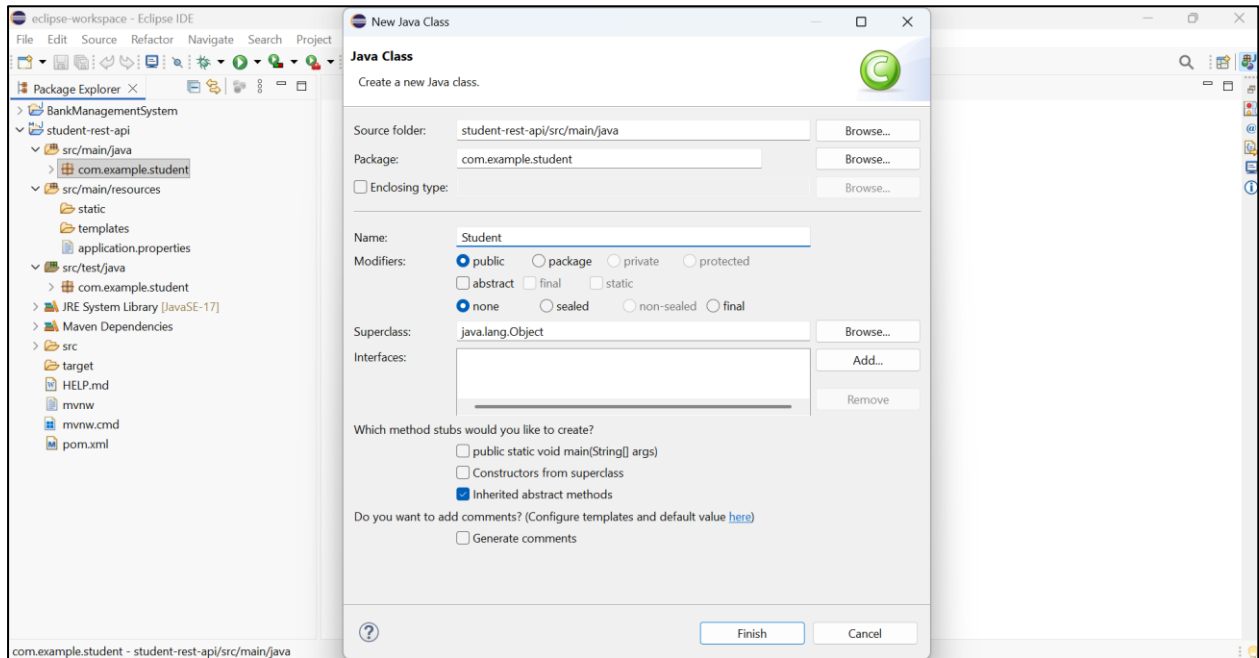
Project import window

Project Structure



Program Code

Create Student class



Write code inside it:

```
package com.example.student;

// Model class representing Student entity
public class Student {

    private int id;
    private String name;
    private String course;

    // Default constructor
    public Student() {
    }

    // Parameterized constructor
    public Student(int id, String name, String course) {
        this.id = id;
        this.name = name;
        this.course = course;
    }

    // Getter and Setter methods
    public int getId() {
        return id;
    }
}
```



```

    }

    public void setId(int id) {
        this.id = id;
    }

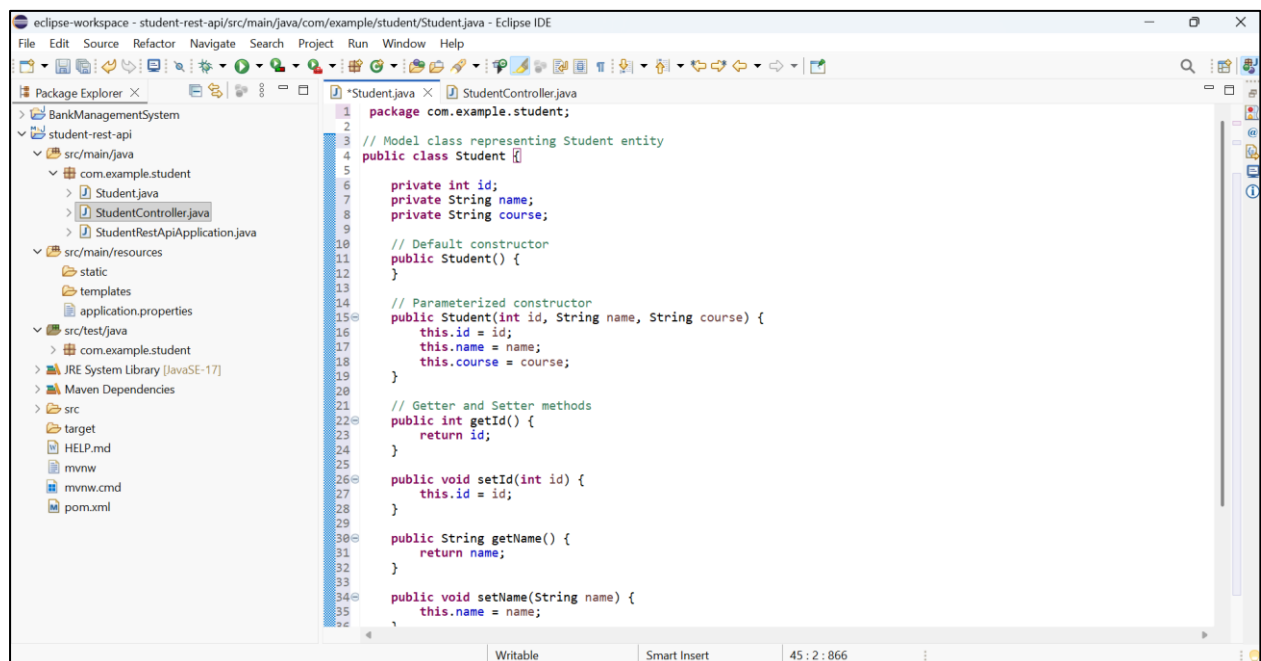
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

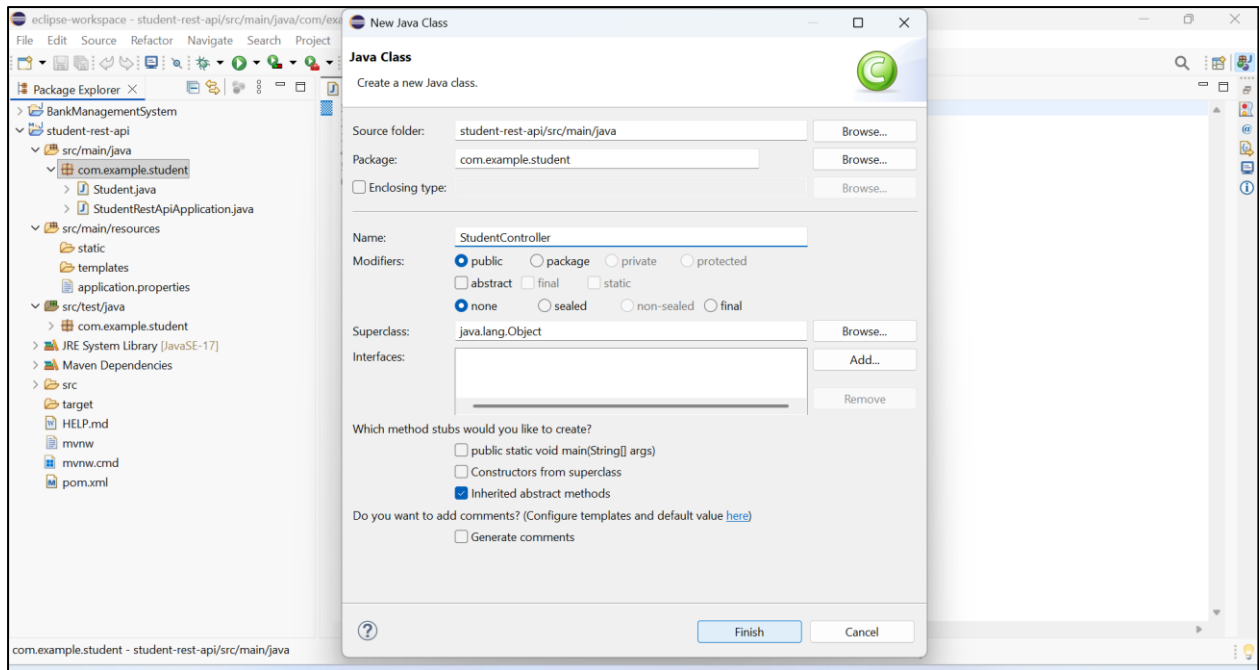
    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}

```



Now Create next class StudentController



Write code inside it:

```
package com.example.student;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

// REST Controller
@RestController
@RequestMapping("/")
public class StudentController {

    private Student student;

    // Welcome endpoint
    @GetMapping
    public String welcomeMessage() {
        return "Welcome to Student Management REST API";
    }

    // GET student details
    @GetMapping("/student")
    public ResponseEntity<Student> getStudent() {

        if (student == null) {
            return new ResponseEntity<>(HttpStatus.NOT_FOUND);
        }
    }
}
```

```

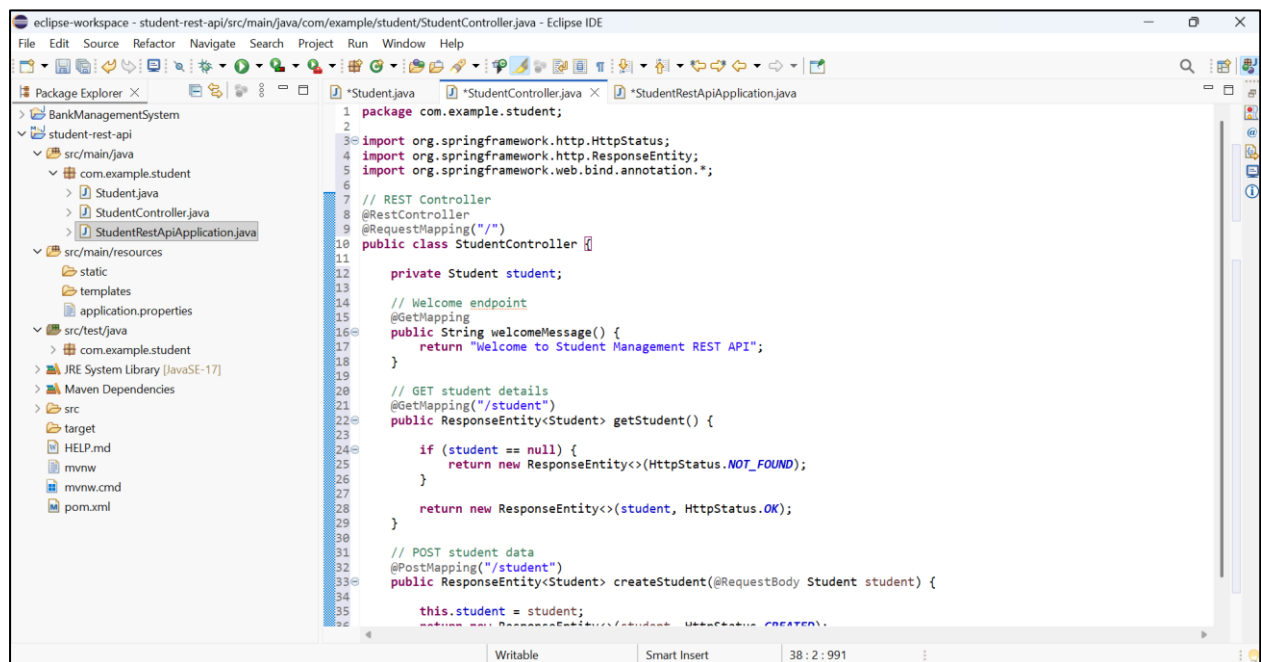
    }

    return new ResponseEntity<>(student, HttpStatus.OK);
}

// POST student data
@PostMapping("/student")
public ResponseEntity<Student> createStudent(@RequestBody Student student) {

    this.student = student;
    return new ResponseEntity<>(student, HttpStatus.CREATED);
}
}

```



Now create Main Application

Create class StudentRestApiApplication

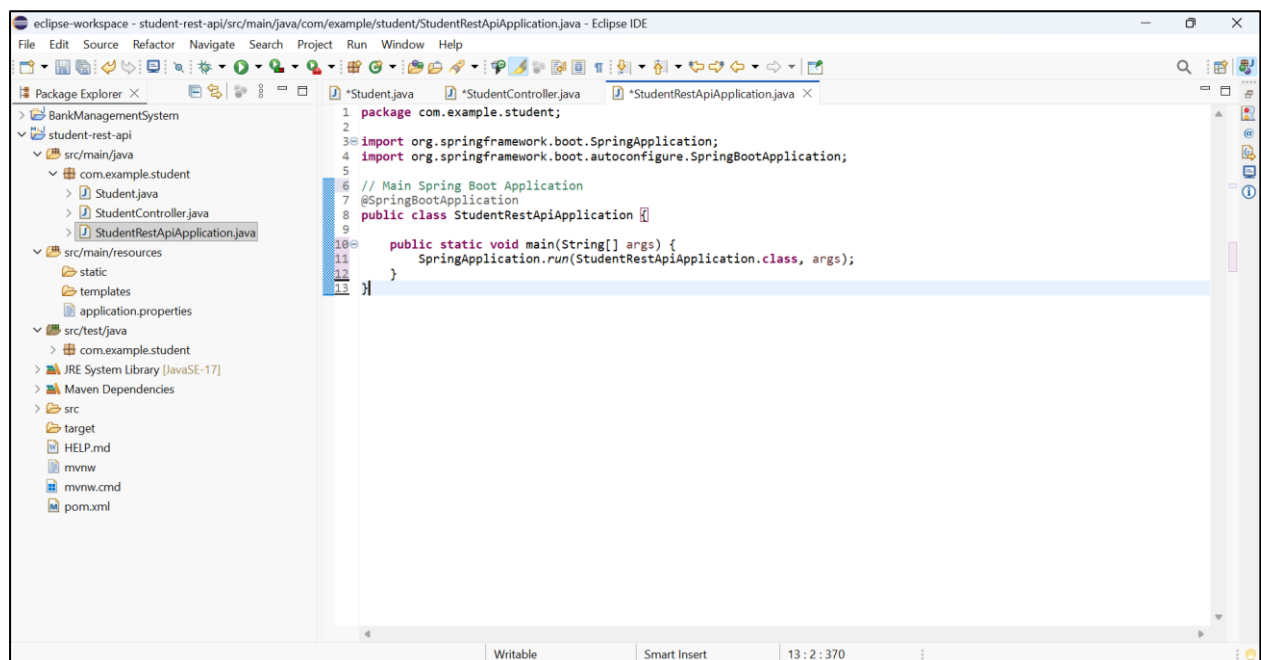
```

package com.example.student;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

// Main Spring Boot Application
@SpringBootApplication

```

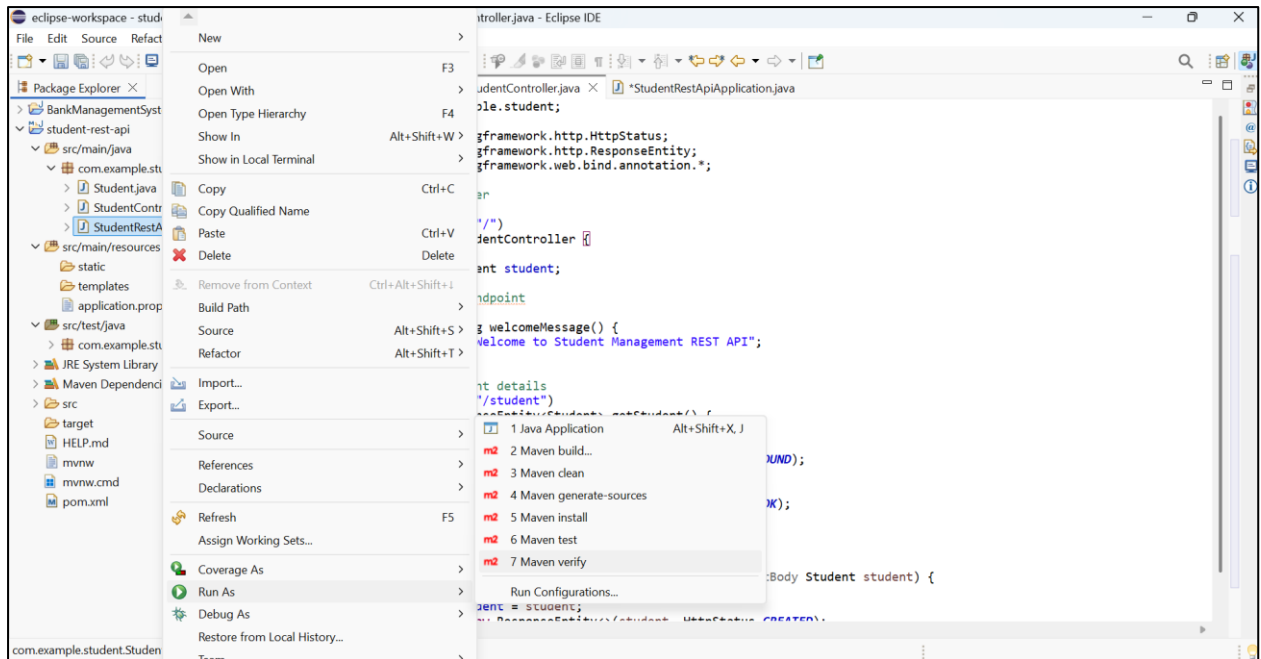
```
public class StudentRestApiApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(StudentRestApiApplication.class, args);  
    }  
}
```



Running the Application in Eclipse IDE

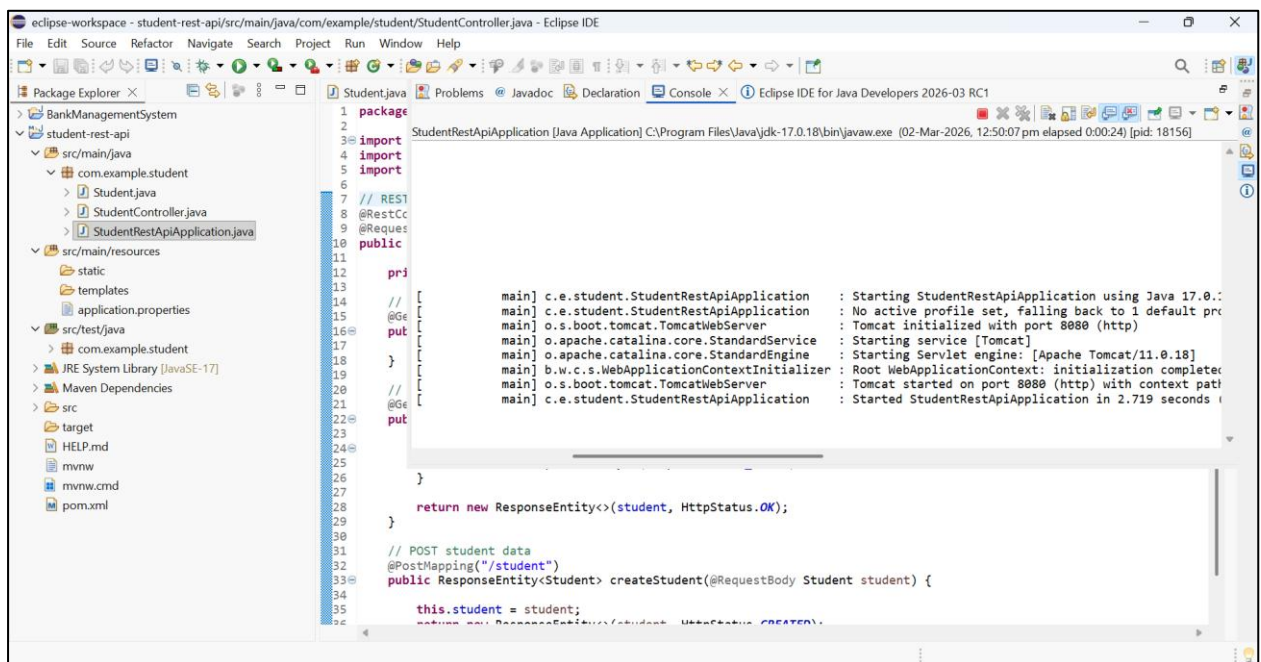
1. Right click the project
2. Click

Run As → Spring Boot App or Java Application



Spring Boot server will start on:

<http://localhost:8080>



Spring Boot console running

Testing the API

Welcome API

Browser:

<http://localhost:8080/>

Output:

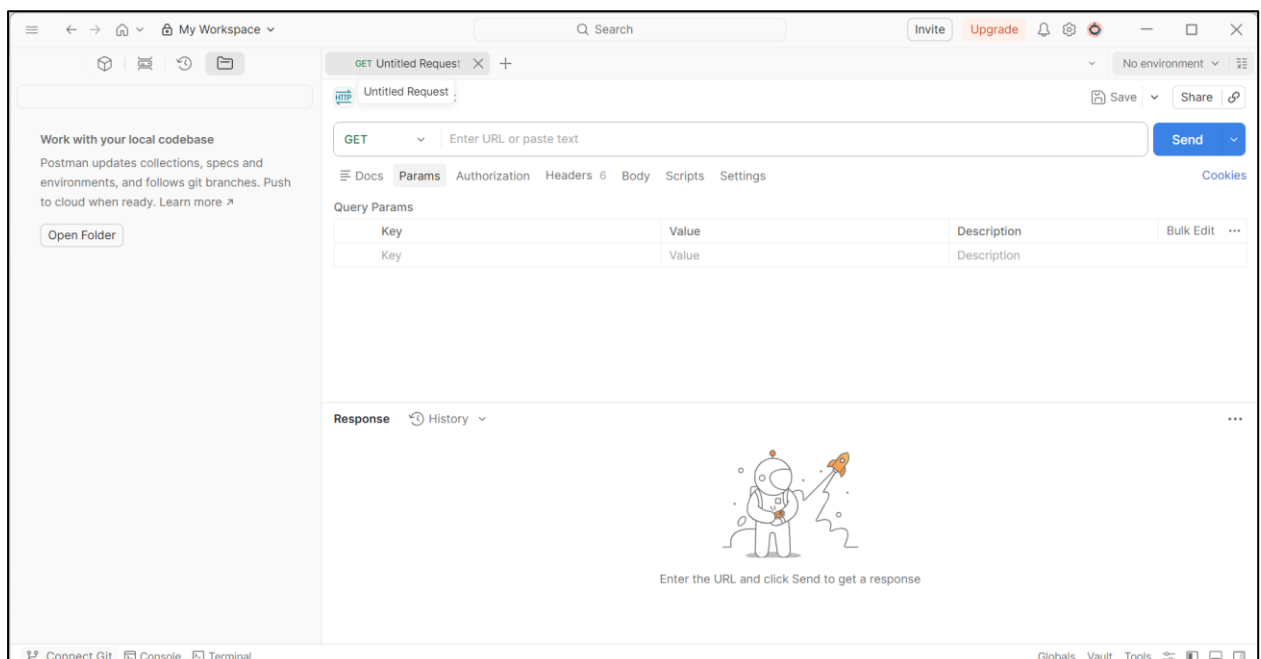


Welcome to Student Management REST API

GET API (If data is post already)

Browser:

<http://localhost:8080/student>



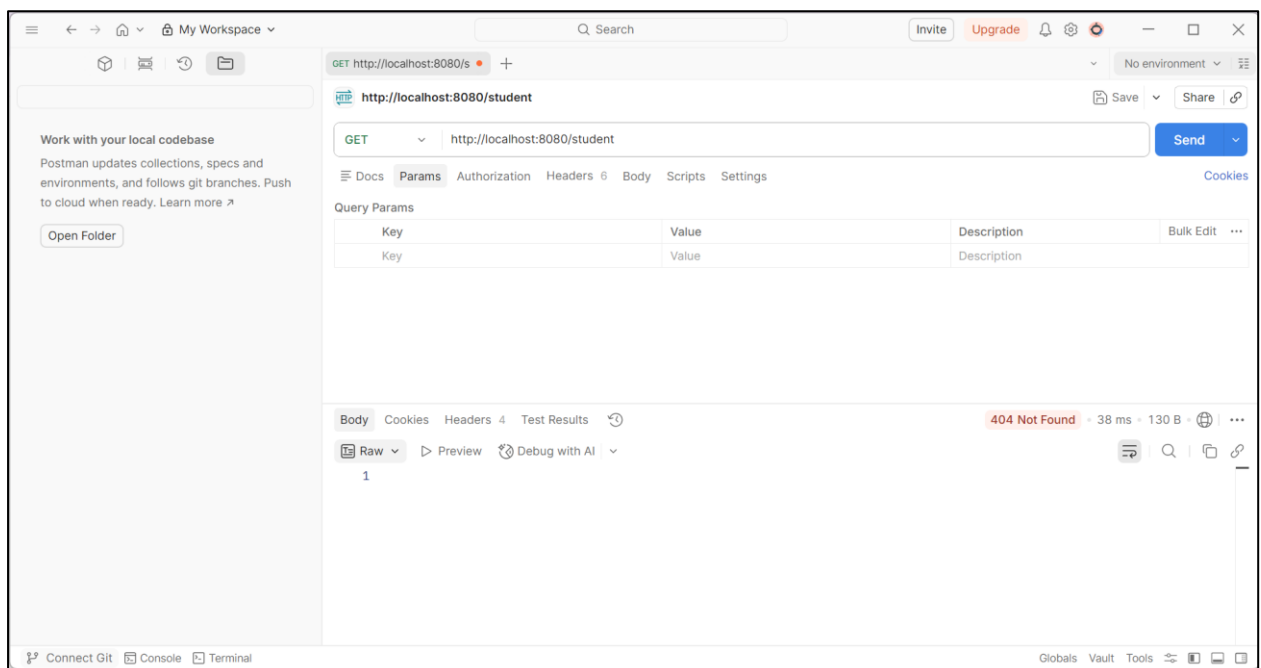
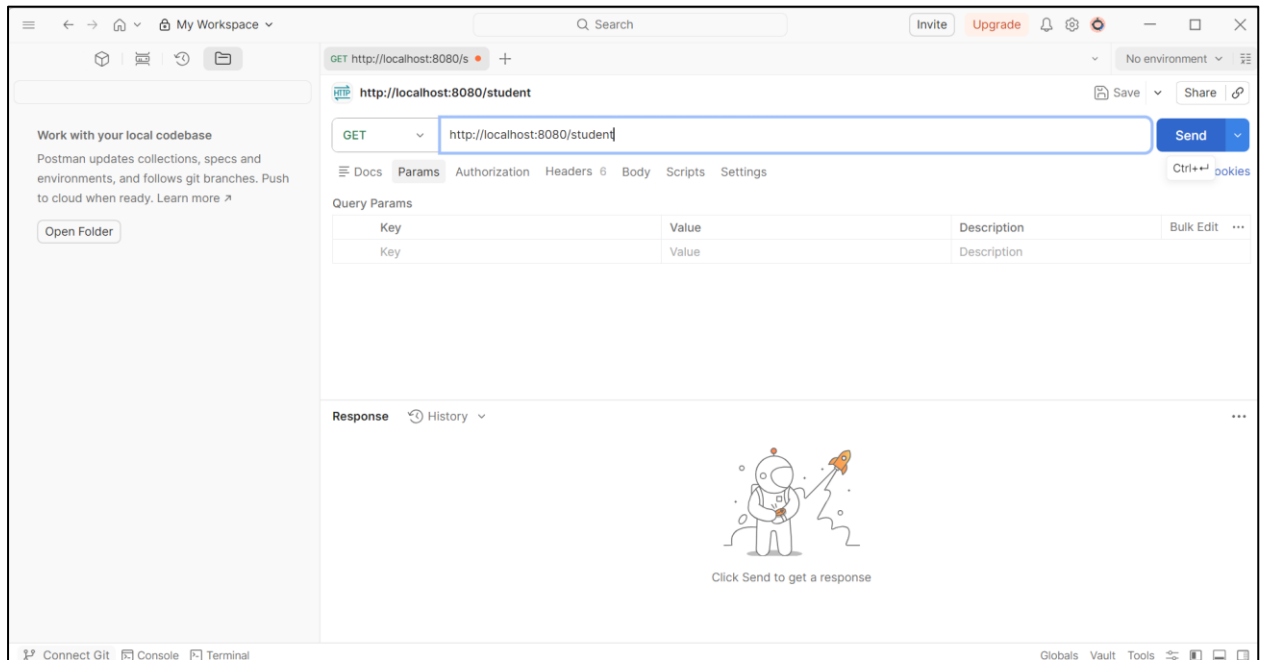
Output (JSON):

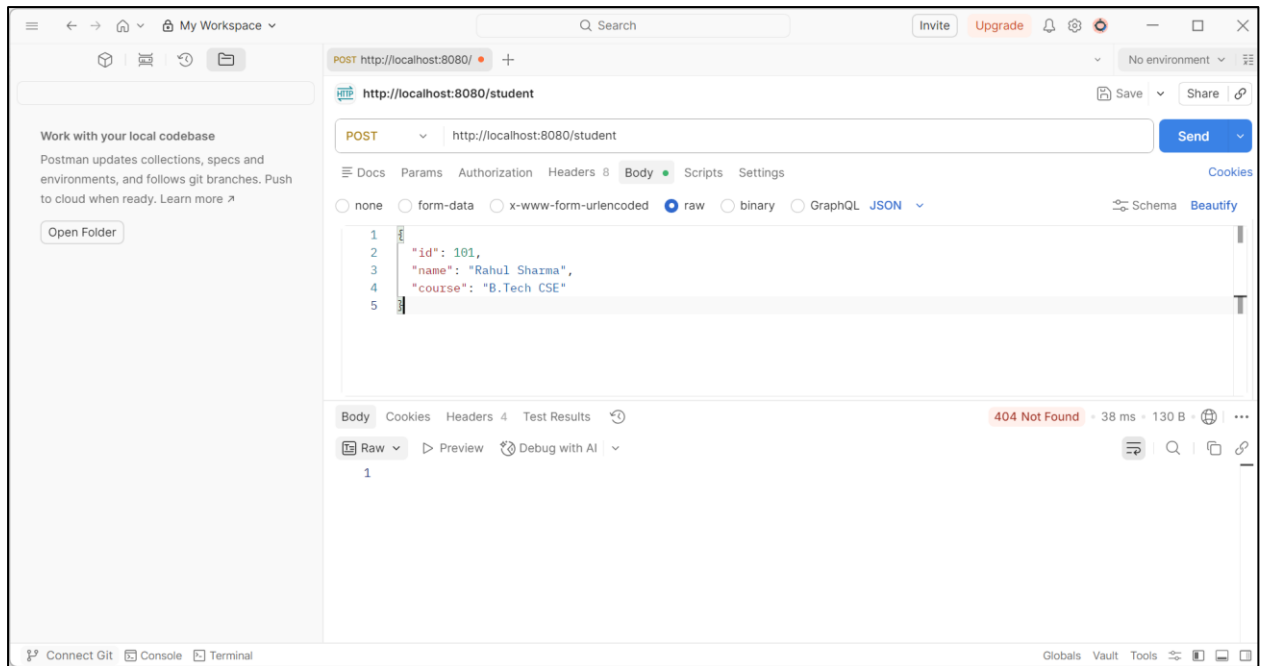
No output available yet.

POST API using Postman

URL

<http://localhost:8080/student>



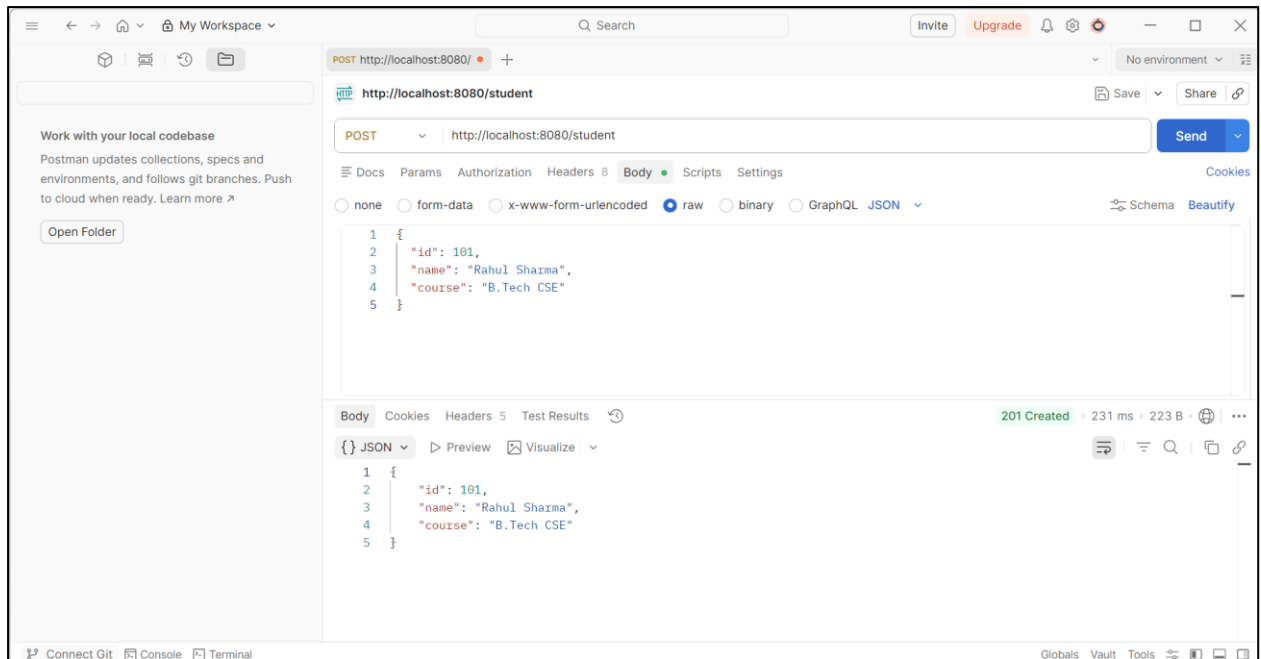


Body → JSON

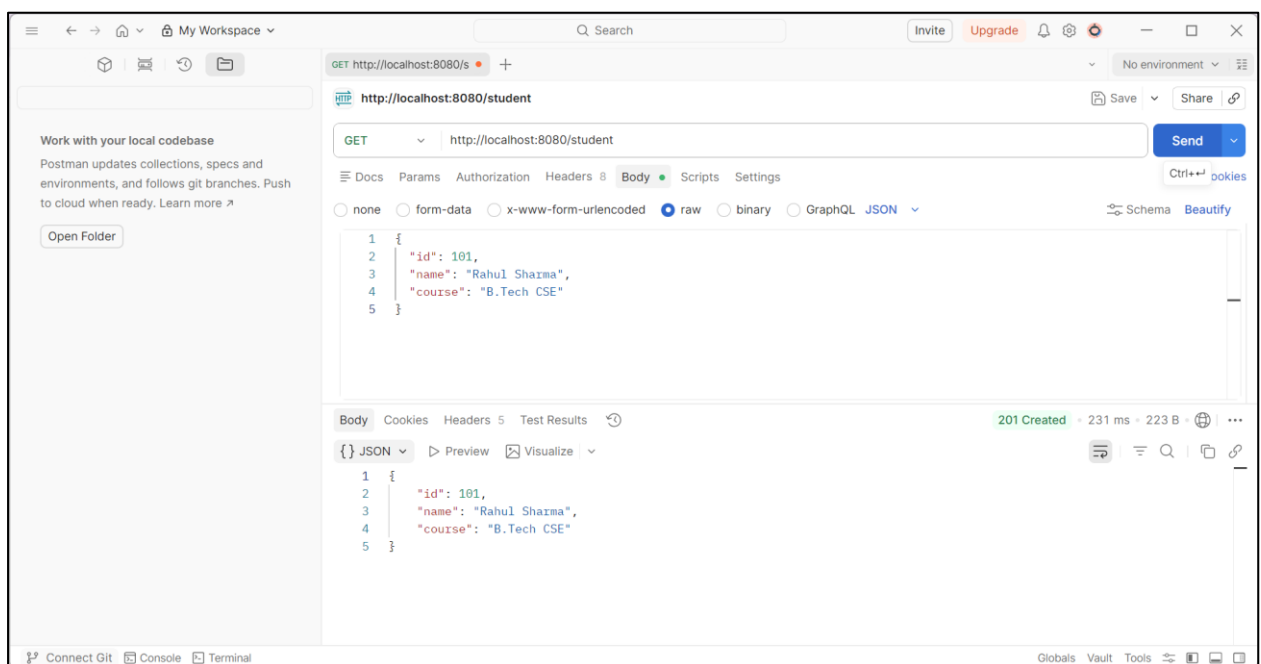
```
{
  "id":101,
  "name":"Rahul Sharma",
  "course":"B.Tech CSE"
}
```

Response:

```
{
  "id":101,
  "name":"Rahul Sharma",
  "course":"B.Tech CSE"
}
```

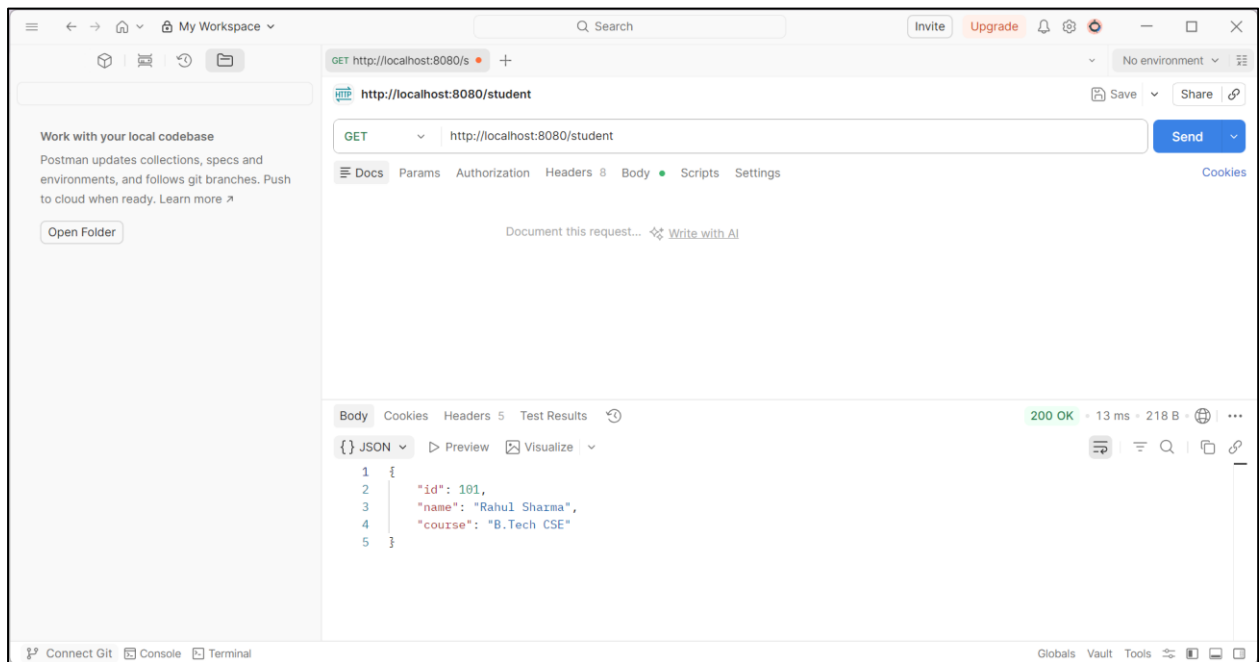
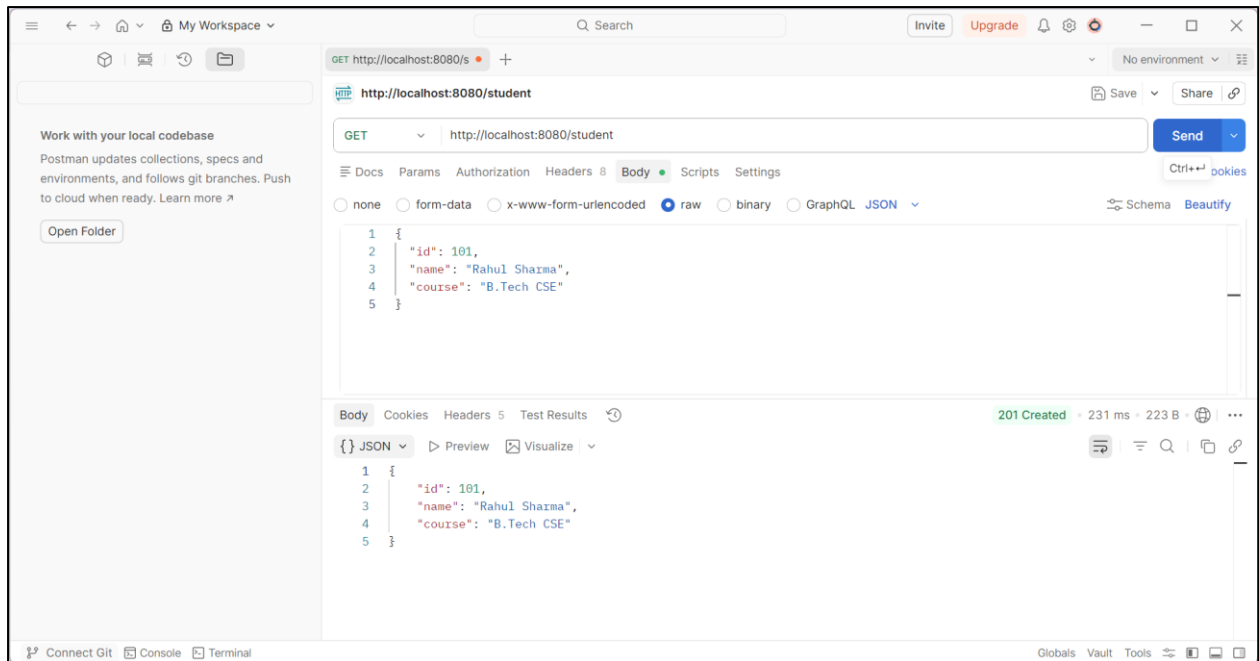
Postman POST request



GET API (Now data is available)

Browser:

<http://localhost:8080/student>



Expected Outcome

Outcome:

Students successfully created **REST APIs** using **Spring Boot** and implemented **GET** and **POST** endpoints for managing student data.

Experiment 12

Title:

CRUD Operations using Spring Boot REST API

Problem Statement:

Implement **Create, Read, Update, and Delete (CRUD)** operations using **Spring Boot REST API**. Create an **Employee Management System REST API** that:

1. Adds a new employee
2. Retrieves all employees
3. Retrieves employee by ID
4. Updates employee details
5. Deletes employee record

Objectives:

After completing this experiment, students will be able to:

- Create **REST APIs using Spring Boot**
- Perform **CRUD operations**
- Connect Spring Boot with **database using JPA**
- Use **@RestController, @Service, and @Repository**
- Test APIs using **Postman**

Theory:

CRUD operations are the basic operations performed on data in applications.

Operation	HTTP Method	Description
Create	POST	Add new data
Read	GET	Retrieve data
Update	PUT	Modify existing data
Delete	DELETE	Remove data

Spring Boot Layers

1. **Controller Layer**
Handles HTTP requests.
2. **Service Layer**
Contains business logic.
3. **Repository Layer**
Communicates with the database using **Spring Data JPA**.

Using these layers ensures **clean architecture and maintainable code**.

Algorithm:

1. Start the Spring Boot project.
2. Add dependencies:

- Spring Web
- Spring Data JPA
- Database driver (H2/MySQL)
- 3. Create an **Employee entity class**.
- 4. Create **EmployeeRepository interface** extending JpaRepository.
- 5. Create **EmployeeService class** for business logic.
- 6. Create **EmployeeController class**.
- 7. Implement endpoints:
 - POST → Add employee
 - GET → Retrieve all employees
 - GET by ID → Retrieve specific employee
 - PUT → Update employee
 - DELETE → Remove employee
- 8. Connect the application to the database.
- 9. Run the application.
- 10. Test API endpoints using **Postman**.
- 11. Verify responses.
- 12. End the program.

Program Code:

Steps to Create Project using Spring Initializr

Go to:

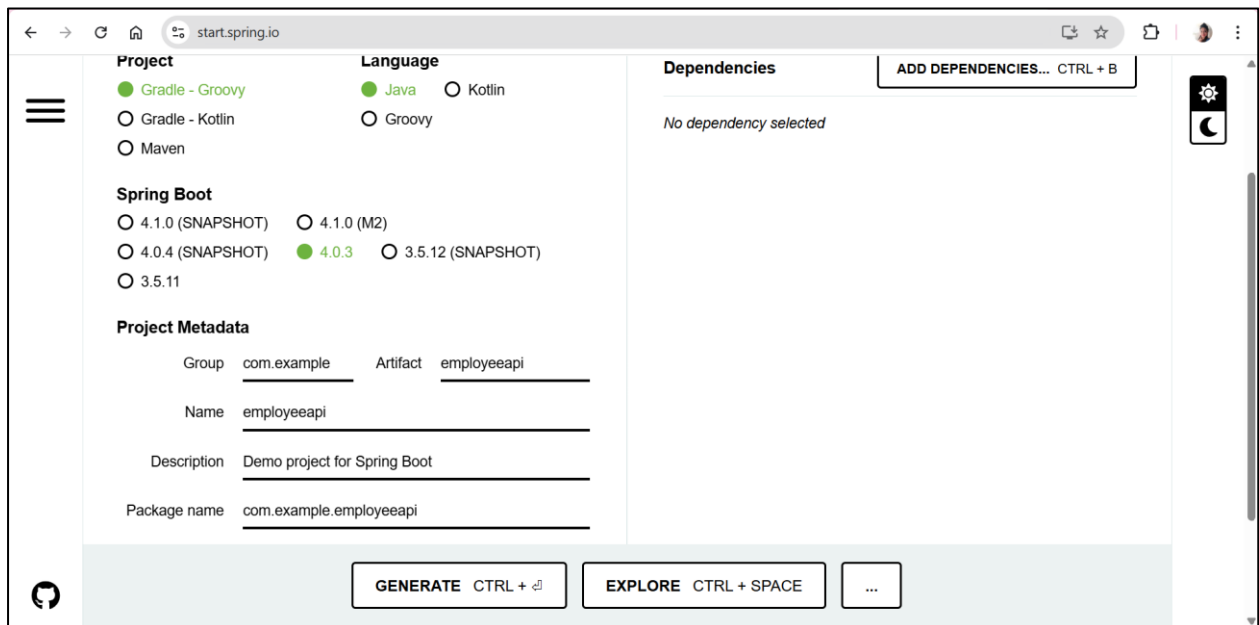
<https://start.spring.io>

The screenshot shows the Spring Initializr web application interface. The browser address bar displays 'start.spring.io'. The page features a sidebar with a hamburger menu and a settings icon. The main content area is divided into sections for Project, Language, Dependencies, Spring Boot, and Project Metadata. The Project section has radio buttons for Gradle - Groovy (selected), Gradle - Kotlin, and Maven. The Language section has radio buttons for Java (selected) and Kotlin. The Dependencies section has a button 'ADD DEPENDENCIES... CTRL + B' and the text 'No dependency selected'. The Spring Boot section has radio buttons for 4.1.0 (SNAPSHOT), 4.1.0 (M2) (selected), 4.0.4 (SNAPSHOT), 4.0.3 (selected), and 3.5.12 (SNAPSHOT). The Project Metadata section has input fields for Group (com.example) and Artifact (demo). At the bottom, there are buttons for 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a three-dot menu.

Project Configuration:

Field	Value
Project	Maven
Language	Java
Spring Boot	Latest

Group com.example
Artifact employeeapi
Java 17



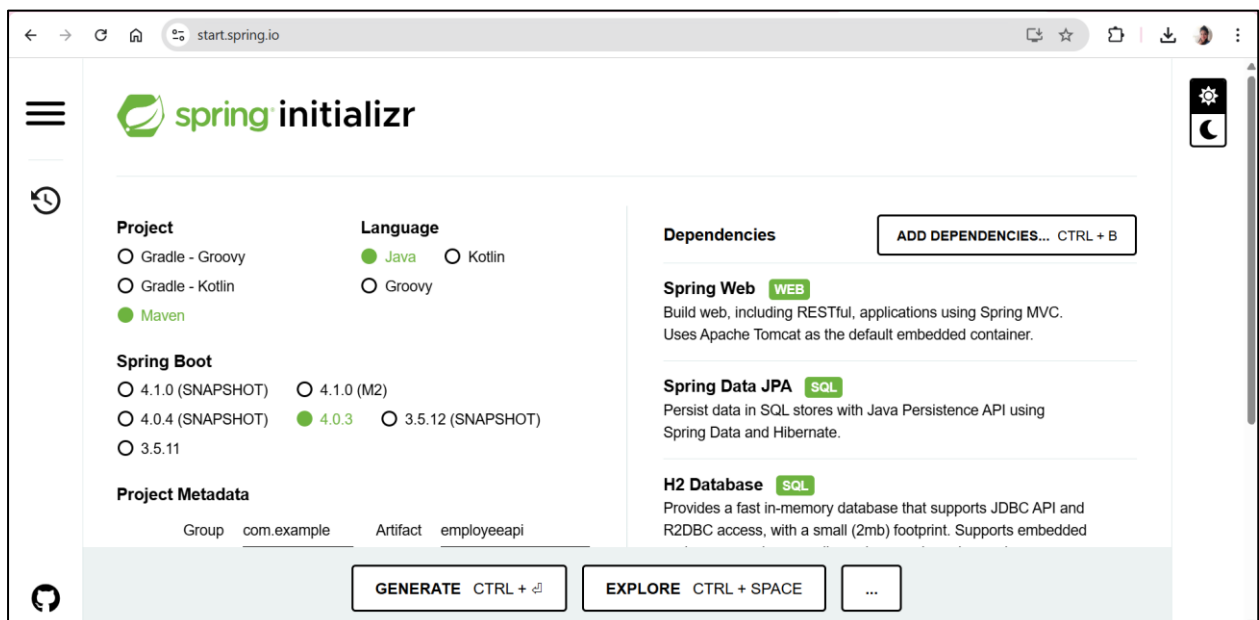
The screenshot shows the start.spring.io web interface. The browser address bar displays 'start.spring.io'. The interface is divided into several sections:

- Project:** Includes radio buttons for 'Gradle - Groovy' (selected), 'Gradle - Kotlin', and 'Maven'.
- Language:** Includes radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'.
- Spring Boot:** Includes radio buttons for versions: '4.1.0 (SNAPSHOT)', '4.1.0 (M2)', '4.0.4 (SNAPSHOT)', '4.0.3' (selected), '3.5.12 (SNAPSHOT)', and '3.5.11'.
- Project Metadata:** Includes input fields for 'Group' (com.example), 'Artifact' (employeeapi), 'Name' (employeeapi), 'Description' (Demo project for Spring Boot), and 'Package name' (com.example.employeeapi).
- Dependencies:** A section with the text 'No dependency selected' and a button 'ADD DEPENDENCIES... CTRL + B'.

At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a button with three dots '...'.

Dependencies:

Spring Web
Spring Data JPA
H2 Database



The screenshot shows the start.spring.io web interface with dependencies added. The browser address bar displays 'start.spring.io'. The interface is divided into several sections:

- Project:** Includes radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', and 'Maven' (selected).
- Language:** Includes radio buttons for 'Java' (selected), 'Kotlin', and 'Groovy'.
- Spring Boot:** Includes radio buttons for versions: '4.1.0 (SNAPSHOT)', '4.1.0 (M2)', '4.0.4 (SNAPSHOT)', '4.0.3' (selected), '3.5.12 (SNAPSHOT)', and '3.5.11'.
- Project Metadata:** Includes input fields for 'Group' (com.example), 'Artifact' (employeeapi), 'Name' (employeeapi), 'Description' (Demo project for Spring Boot), and 'Package name' (com.example.employeeapi).
- Dependencies:** A section with three dependencies listed: 'Spring Web' (WEB), 'Spring Data JPA' (SQL), and 'H2 Database' (SQL). Each dependency has a brief description.

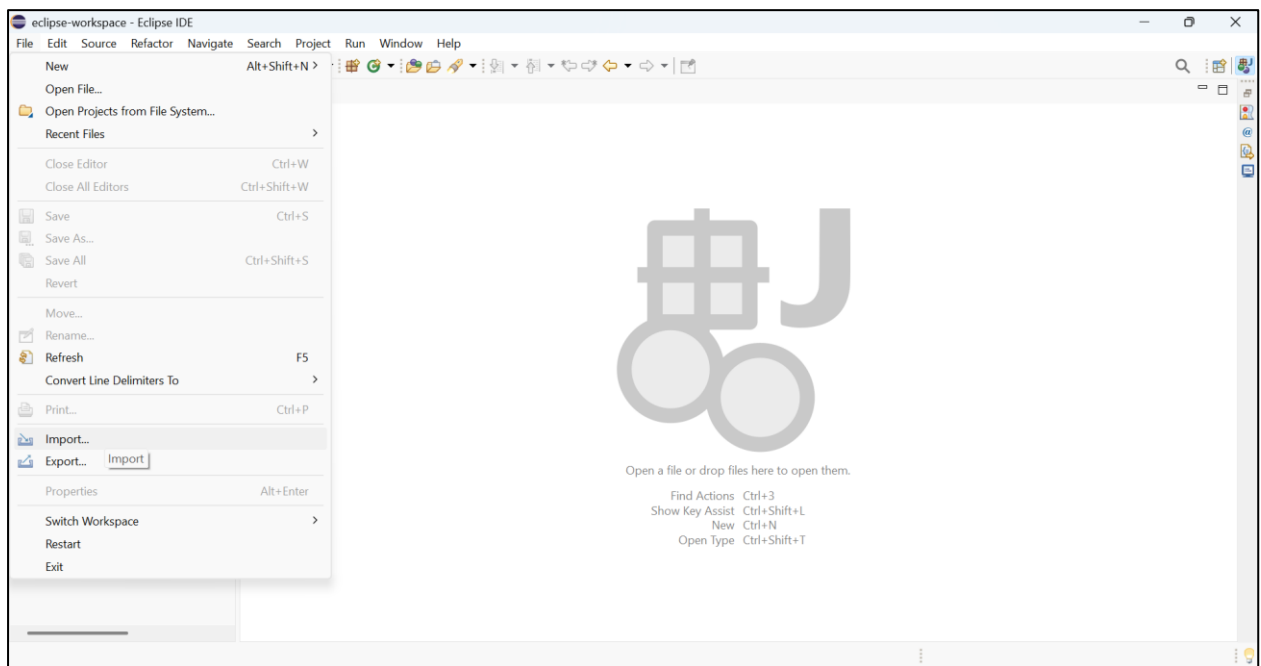
At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a button with three dots '...'.

Click **Generate**.

Import Project in Eclipse IDE

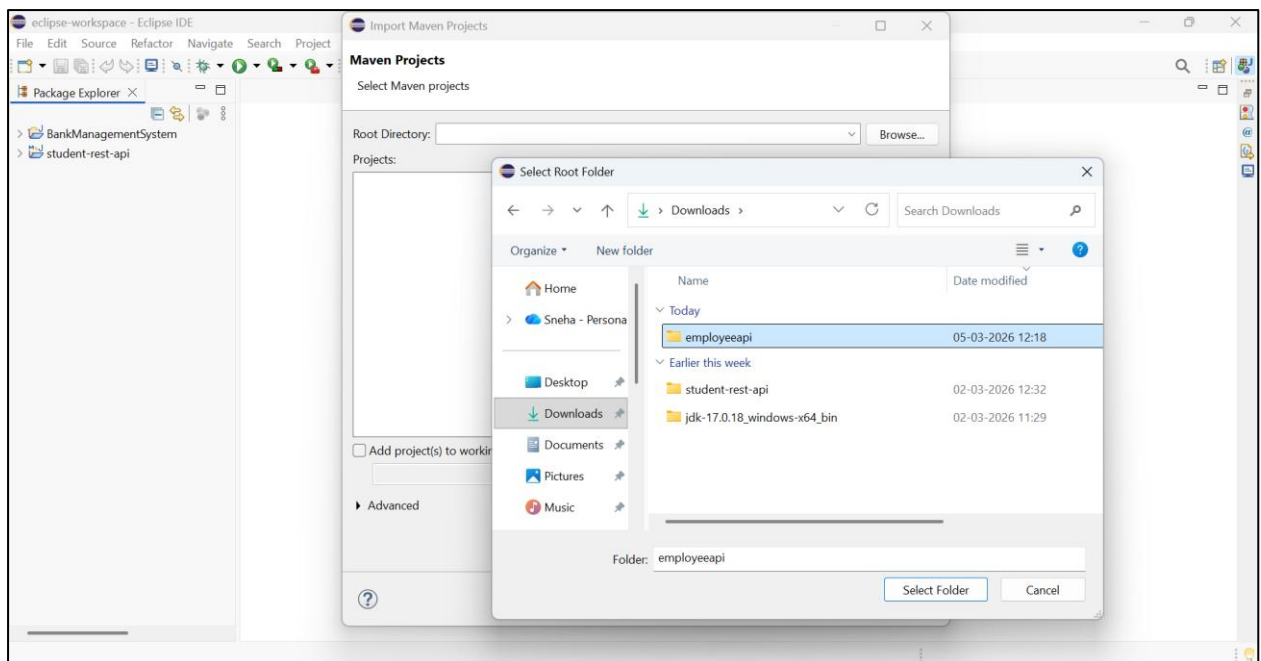
1. Open Eclipse
2. Click

File → Import

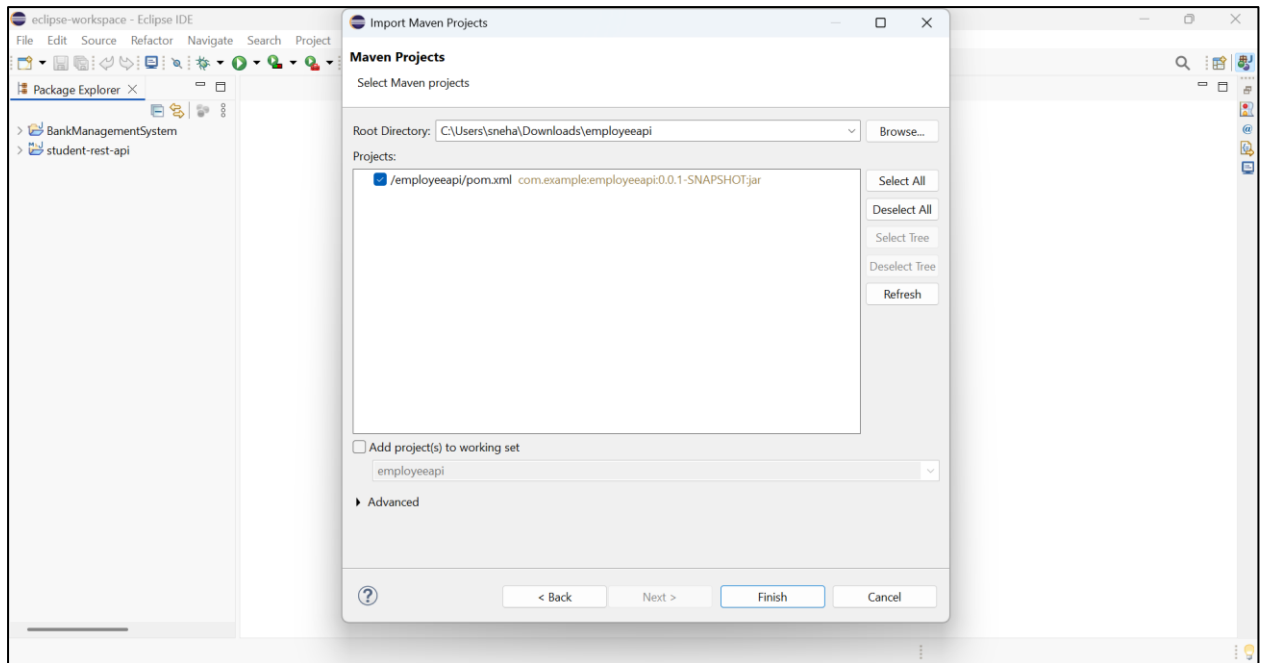


- Select

Existing Maven Project



- Choose downloaded project folder



- Click **Finish**
- Expand your project:

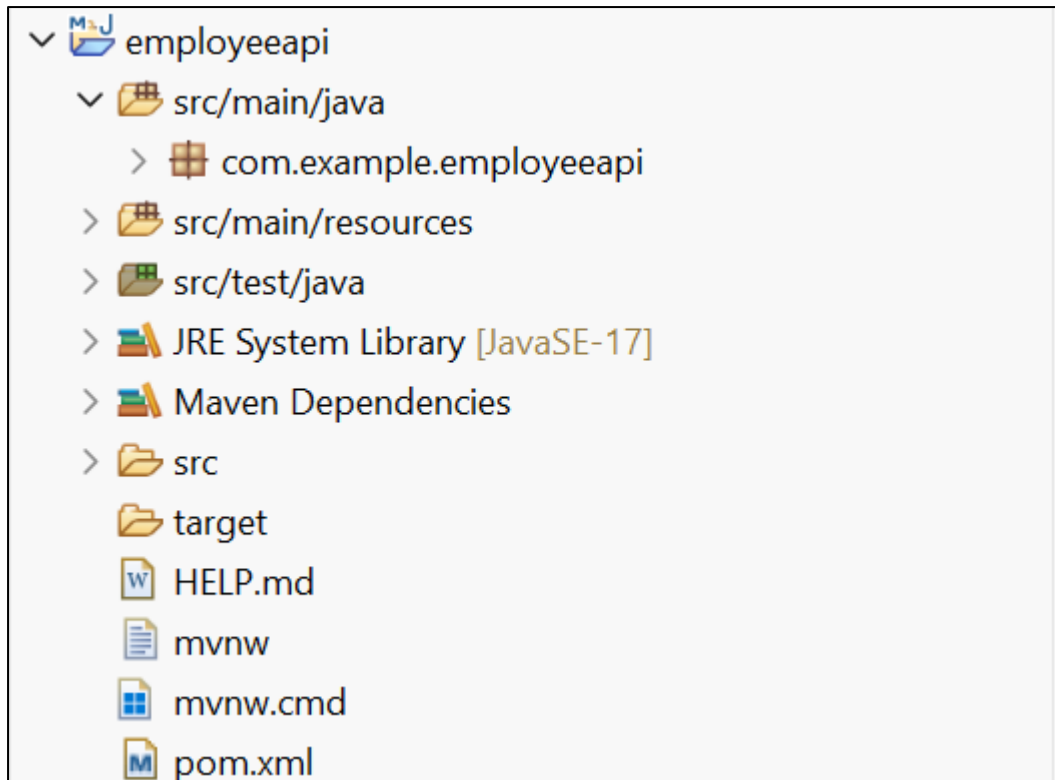
employeeapi

Open the folder:

src/main/java

You will see the base package:

com.example.employeeapi

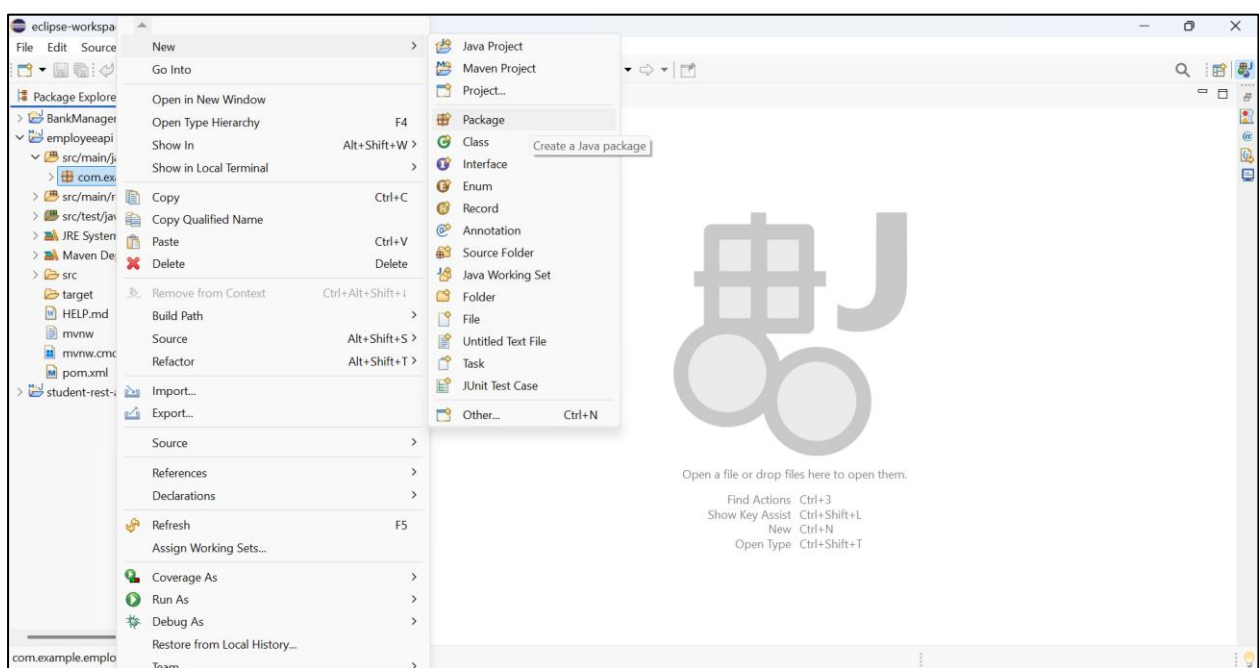


Create Model Package

- Right click on
com.example.employeeapi

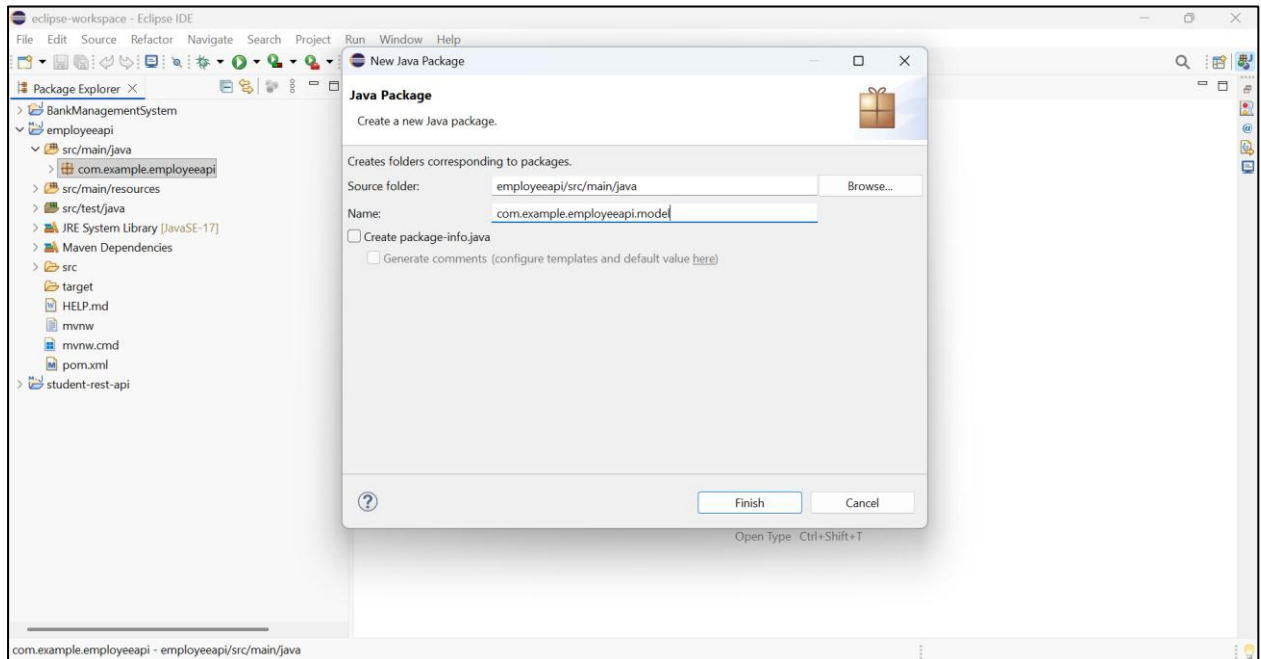
- Click

New → Package

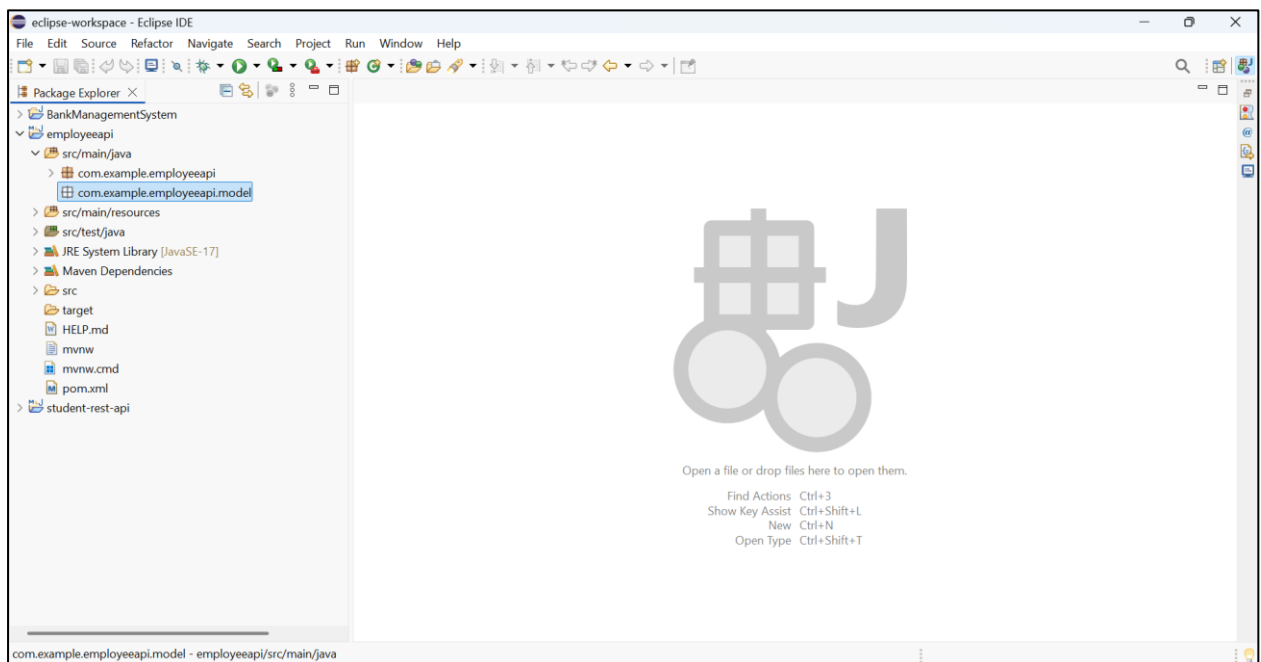


- Enter package name:

`com.example.employeeapi.model`



- Click **Finish**

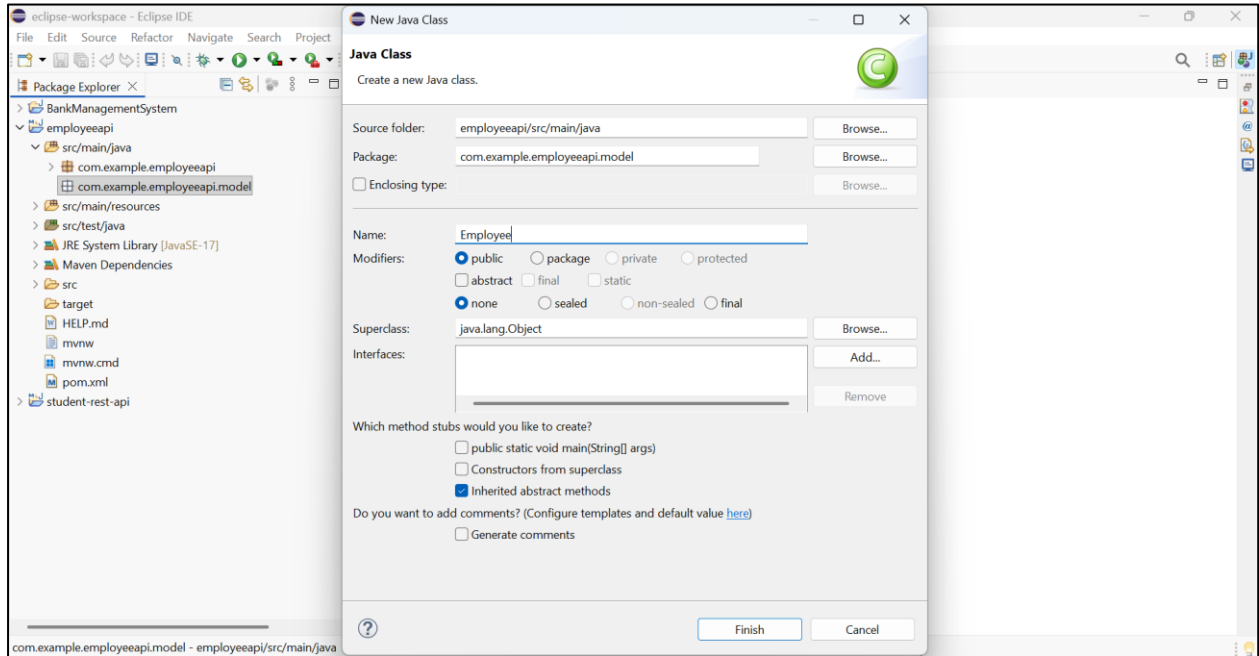


Create Employee Class

- Right click on
model package

- Click
- New → Class
- Enter

Class Name: Employee



1. Click **Finish**

Now write **Employee Entity Code** inside the class.

Employee Entity

```
package com.example.employeeapi.model;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    private int id;
    private String name;
    private String department;
    private double salary;

    public Employee() {}

    public Employee(int id, String name, String department, double salary) {
        this.id = id;
    }
}
```

```
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

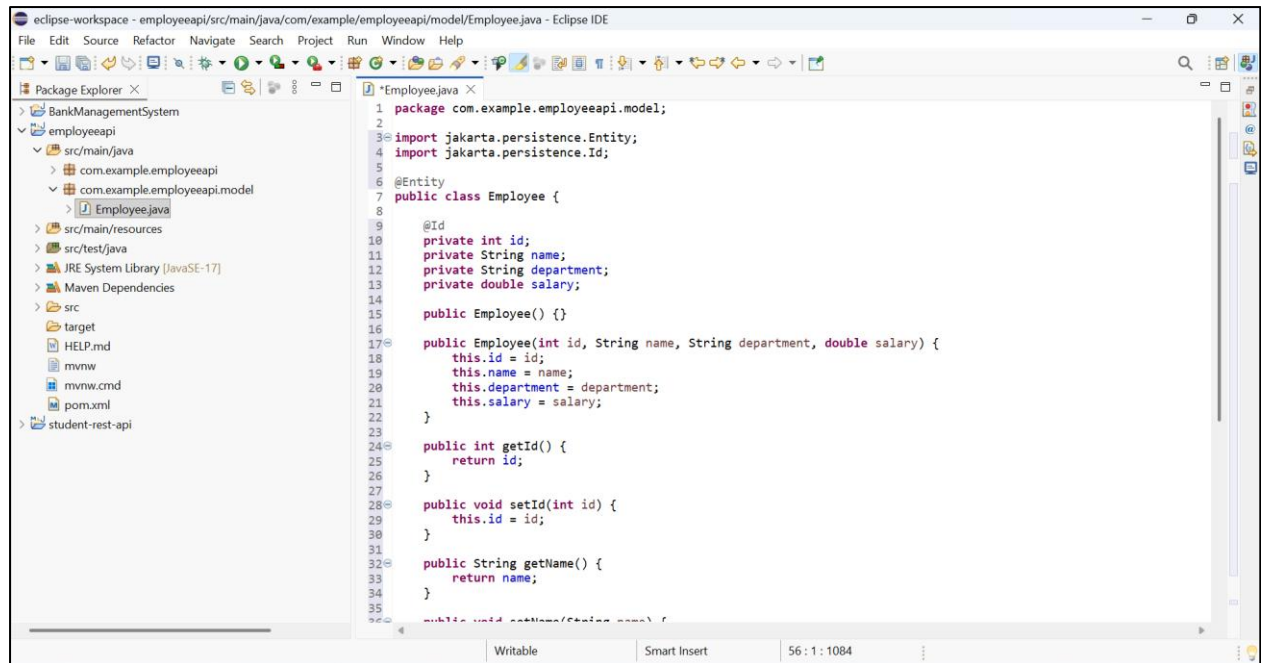
    public void setName(String name) {
        this.name = name;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

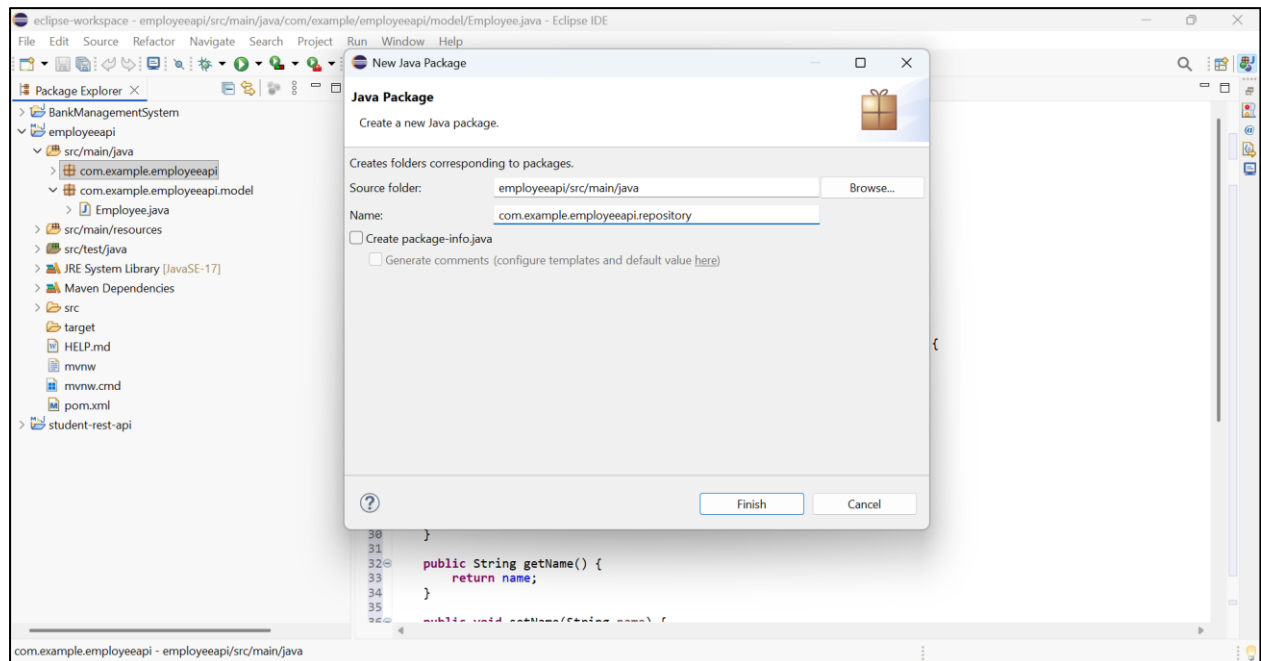
    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```



Create Repository Package

- Right click on
- com.example.employeeapi
- Click
- New → Package
- Enter:
- com.example.employeeapi.repository

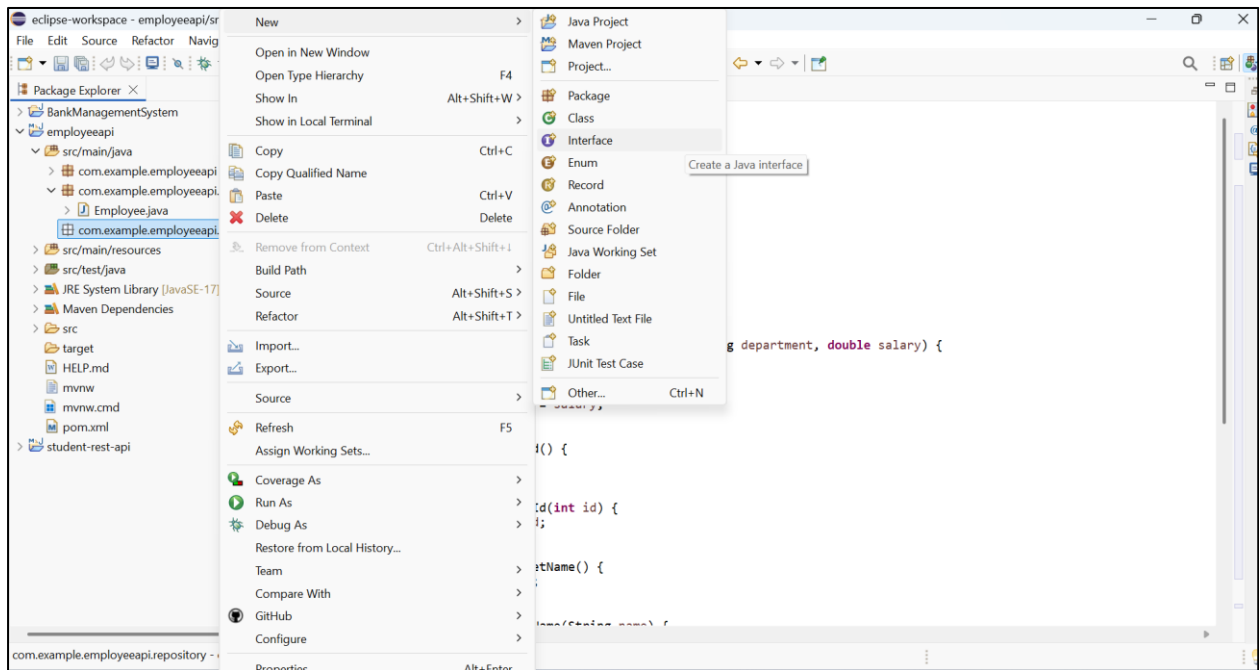


- Click **Finish**

Create EmployeeRepository Interface

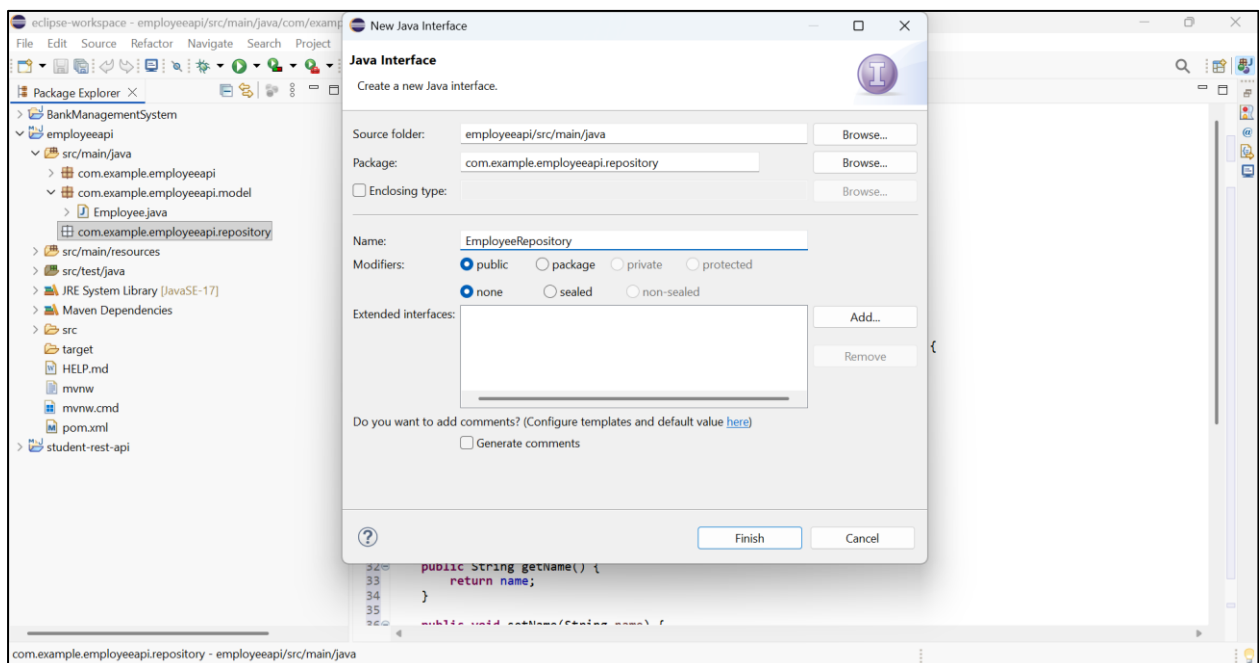
- Right click
- repository package
- Click

New → Interface



- Enter

EmployeeRepository



- Click **Finish**

Write repository code.

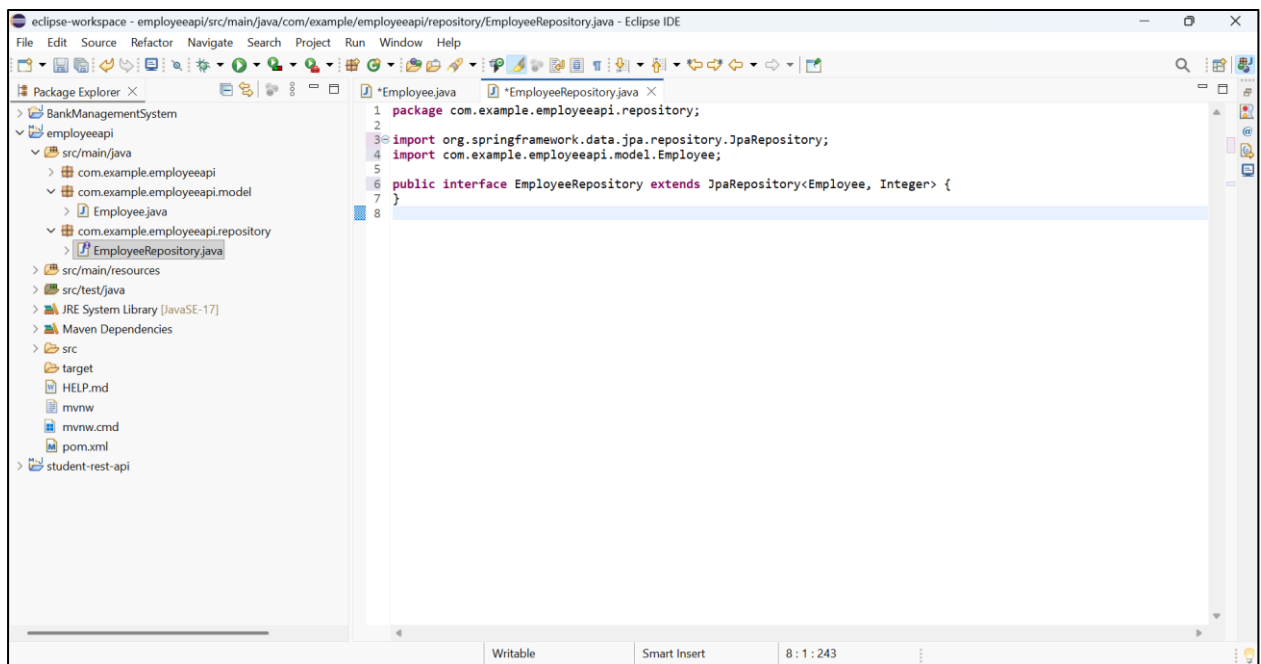
Repository Layer

```
package com.example.employeeapi.repository;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import com.example.employeeapi.model.Employee;
```

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
}
```



Create Service Package

- Right click

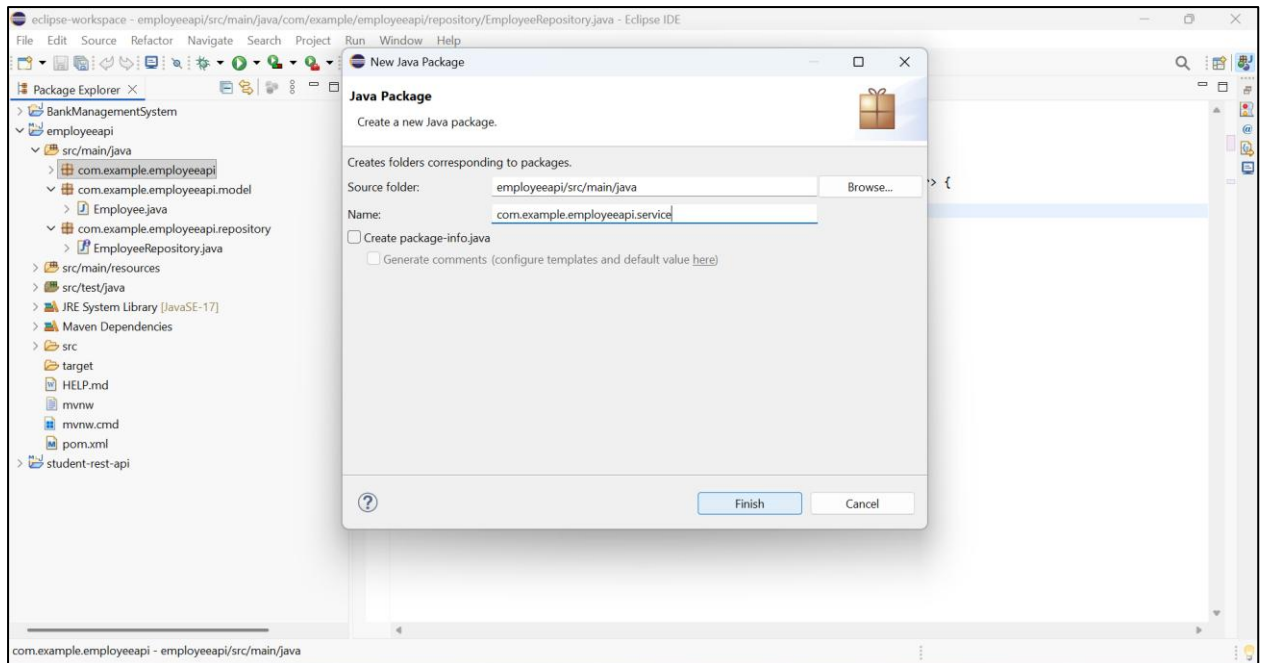
`com.example.employeeapi`

- Click

New → Package

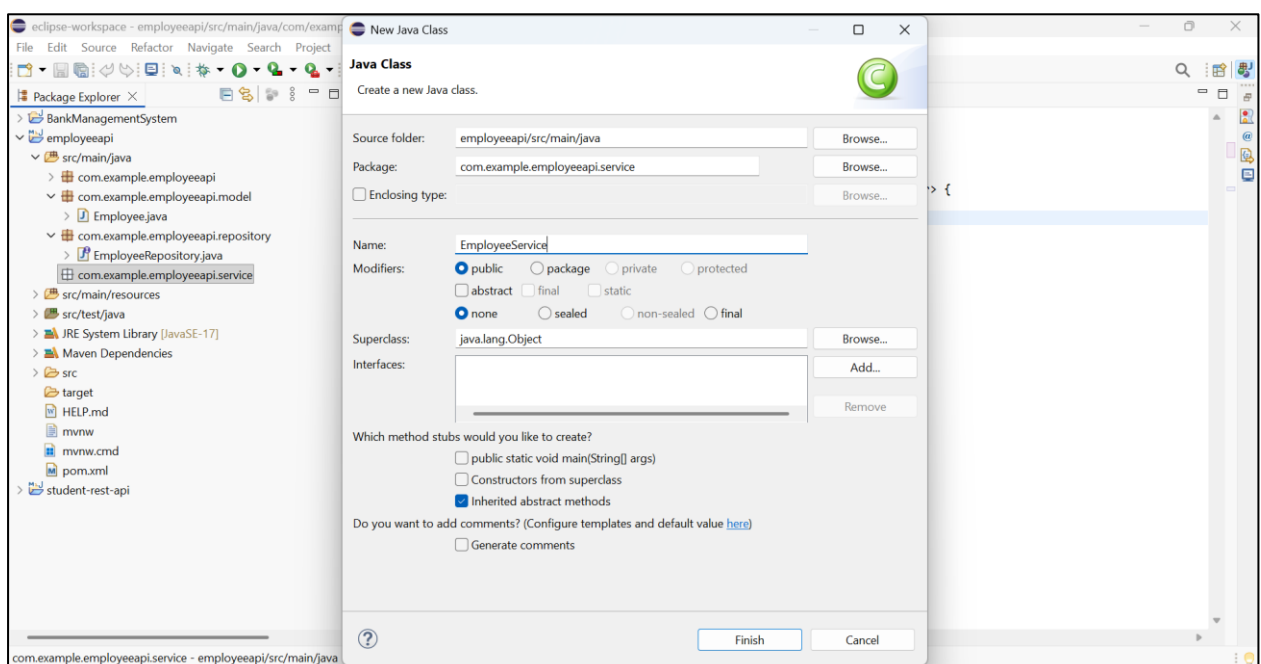
Name it

`com.example.employeeapi.service`



- Click **Finish**
- **Create EmployeeService Class**
- Right click
- service package
- Click
- New → Class
- Enter

EmployeeService



- Click **Finish**

Write **Service Layer** code.

Service Layer

```
package com.example.employeeapi.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.employeeapi.model.Employee;
import com.example.employeeapi.repository.EmployeeRepository;

@Service
public class EmployeeService {

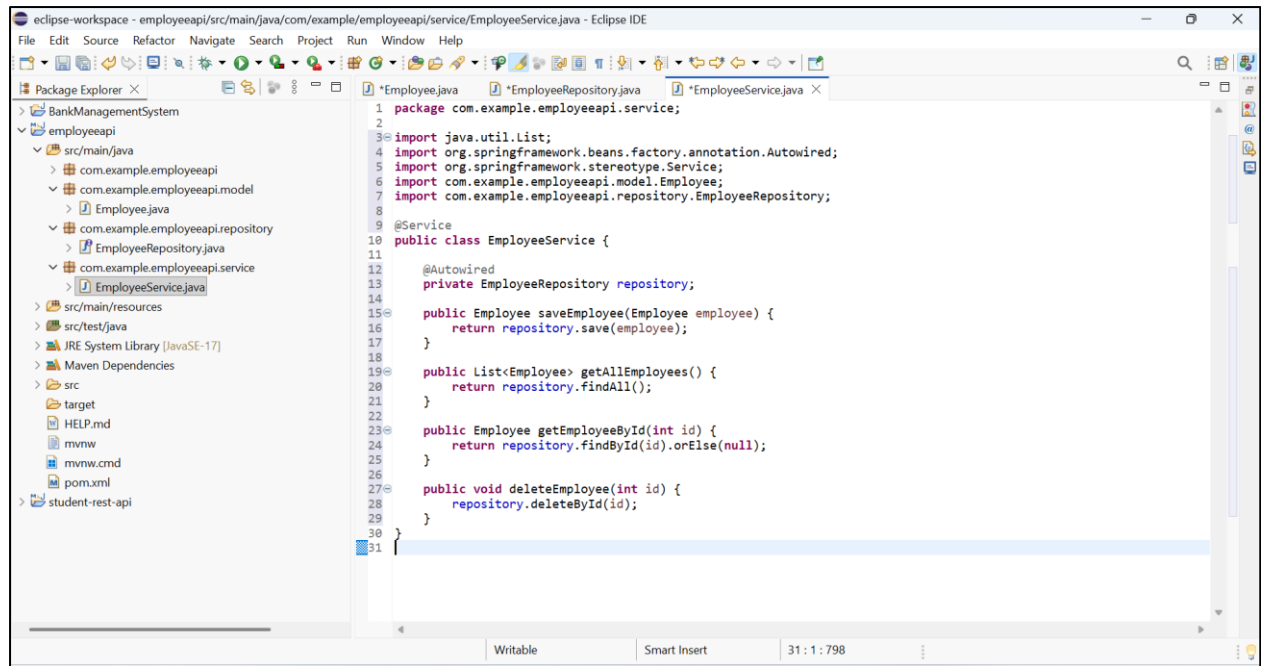
    @Autowired
    private EmployeeRepository repository;

    public Employee saveEmployee(Employee employee) {
        return repository.save(employee);
    }

    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    public Employee getEmployeeById(int id) {
        return repository.findById(id).orElse(null);
    }

    public void deleteEmployee(int id) {
        repository.deleteById(id);
    }
}
```

Create Controller Package

- Right click

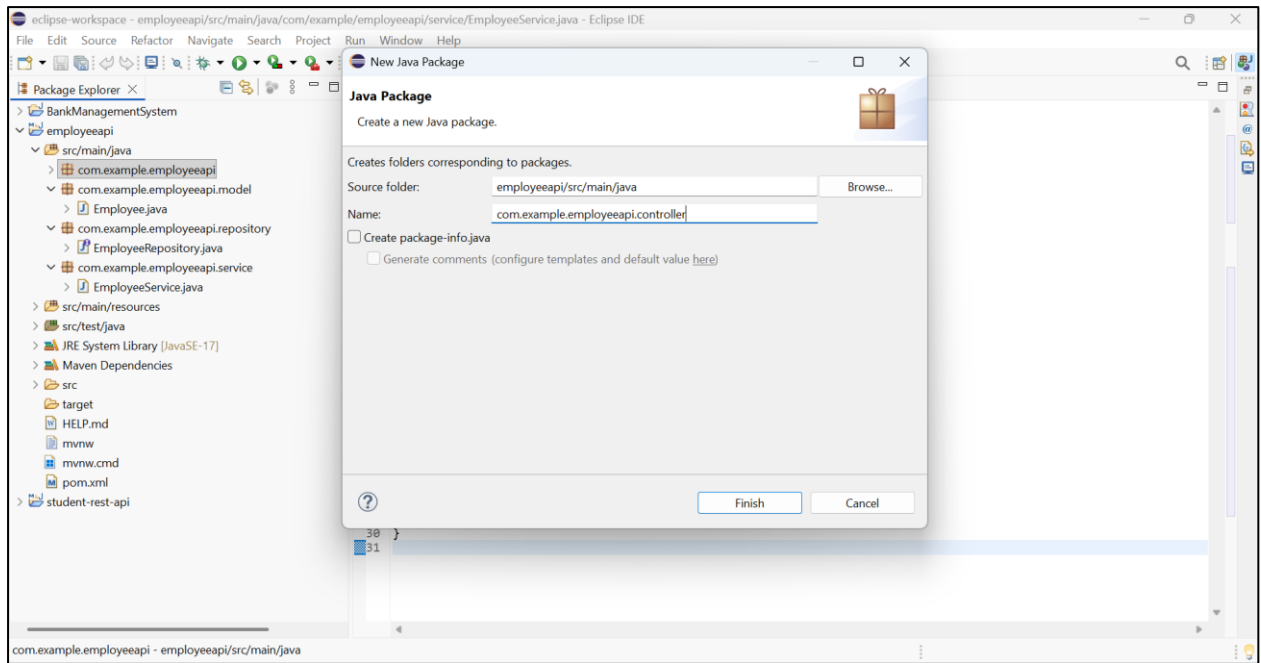
com.example.employeeapi

- Click

New → Package

- Name:

com.example.employeeapi.controller



- Click **Finish**

Create EmployeeController Class

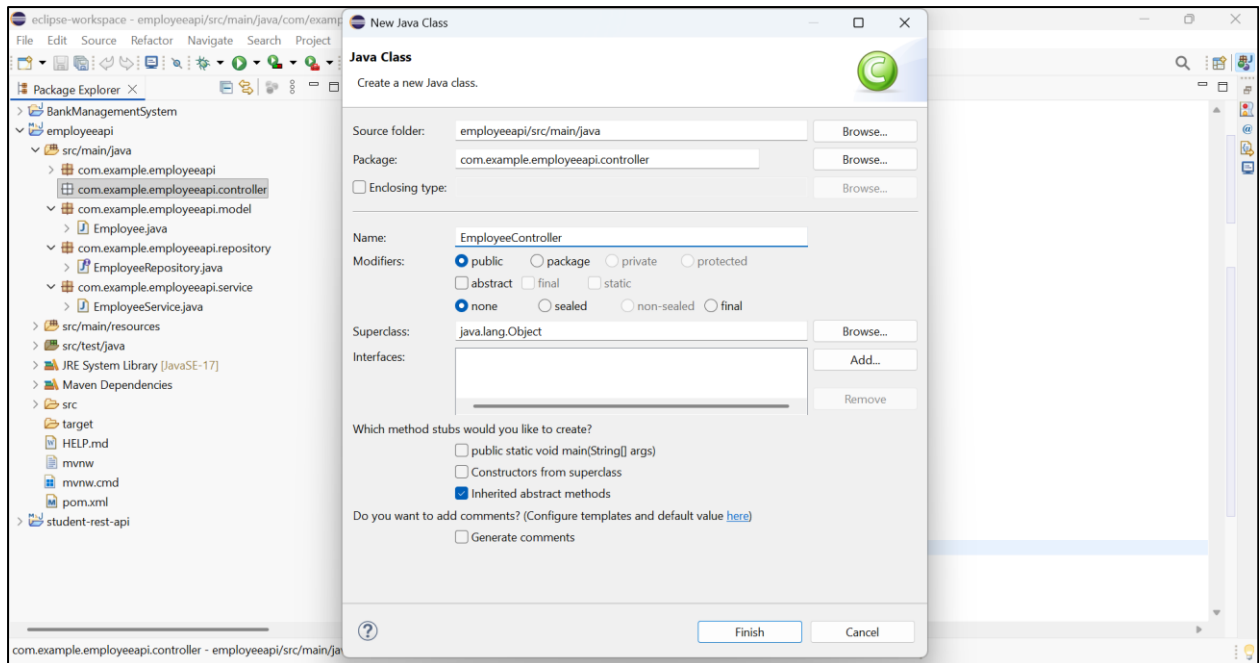
- Right click
controller package

- Click

New → Class

- Enter

EmployeeController



1. Click **Finish**

Write the **Controller** code.

Controller Layer

```
package com.example.employeeapi.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.employeeapi.model.Employee;
import com.example.employeeapi.service.EmployeeService;

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {
        return service.saveEmployee(employee);
    }

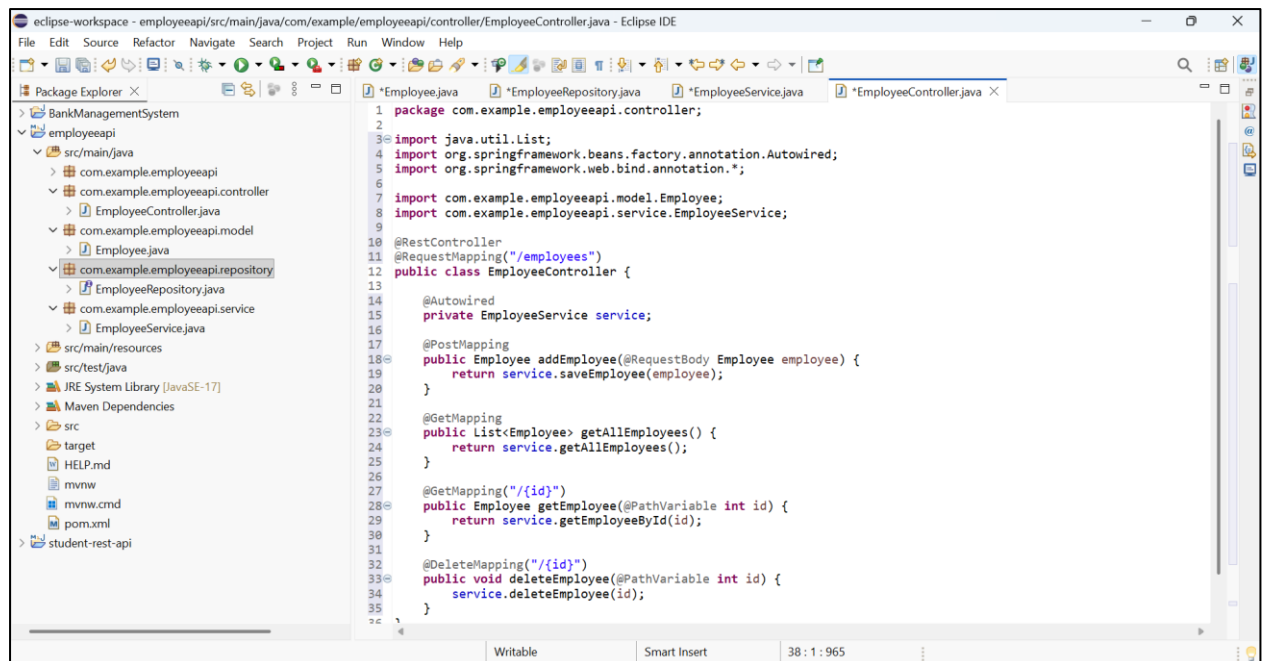
    @GetMapping
    public List<Employee> getAllEmployees() {
        return service.getAllEmployees();
    }
}
```

```

@GetMapping("/{id}")
public Employee getEmployee(@PathVariable int id) {
    return service.getEmployeeById(id);
}

@DeleteMapping("/{id}")
public void deleteEmployee(@PathVariable int id) {
    service.deleteEmployee(id);
}
}

```



Check Main Application File

Open:

EmployeeApiApplication.java

Location:

com.example.employeeapi

Main Application

```
package com.example.employeeapi;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

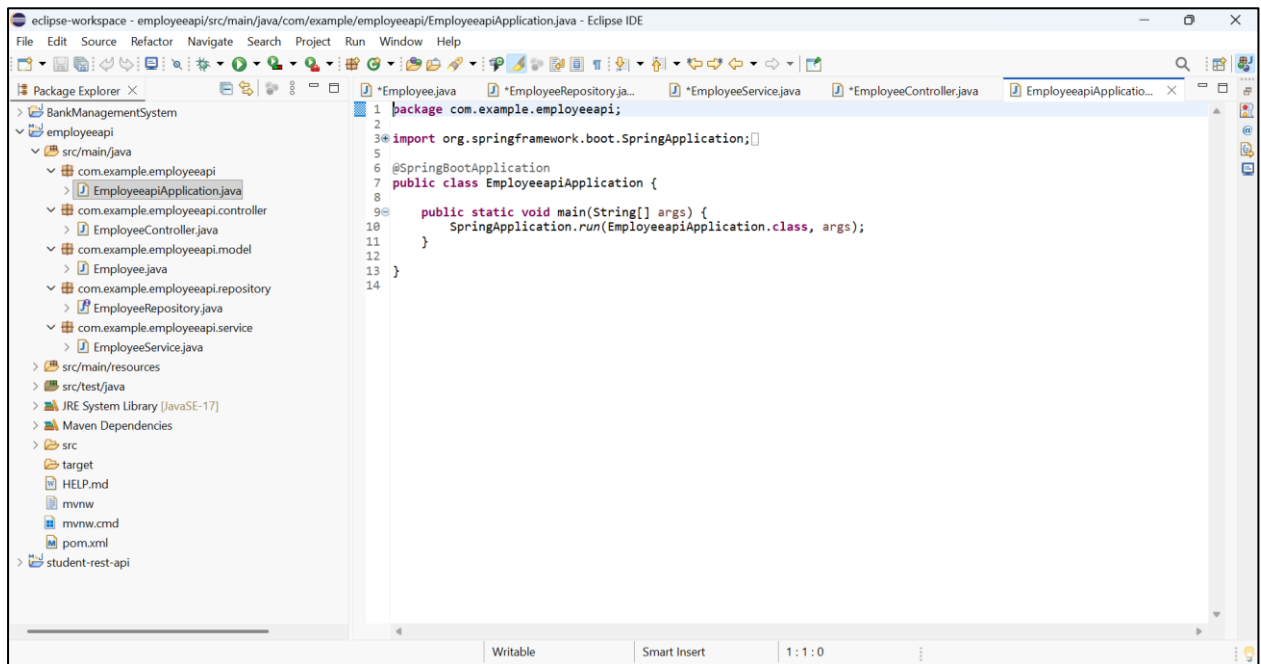
```

@SpringBootApplication
public class EmployeeApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeApiApplication.class, args);
    }

}

```



Ensure the code contains:

```

@SpringBootApplication
public class EmployeeApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(EmployeeApiApplication.class, args);
    }
}

```

This is the **Spring Boot** starting point.

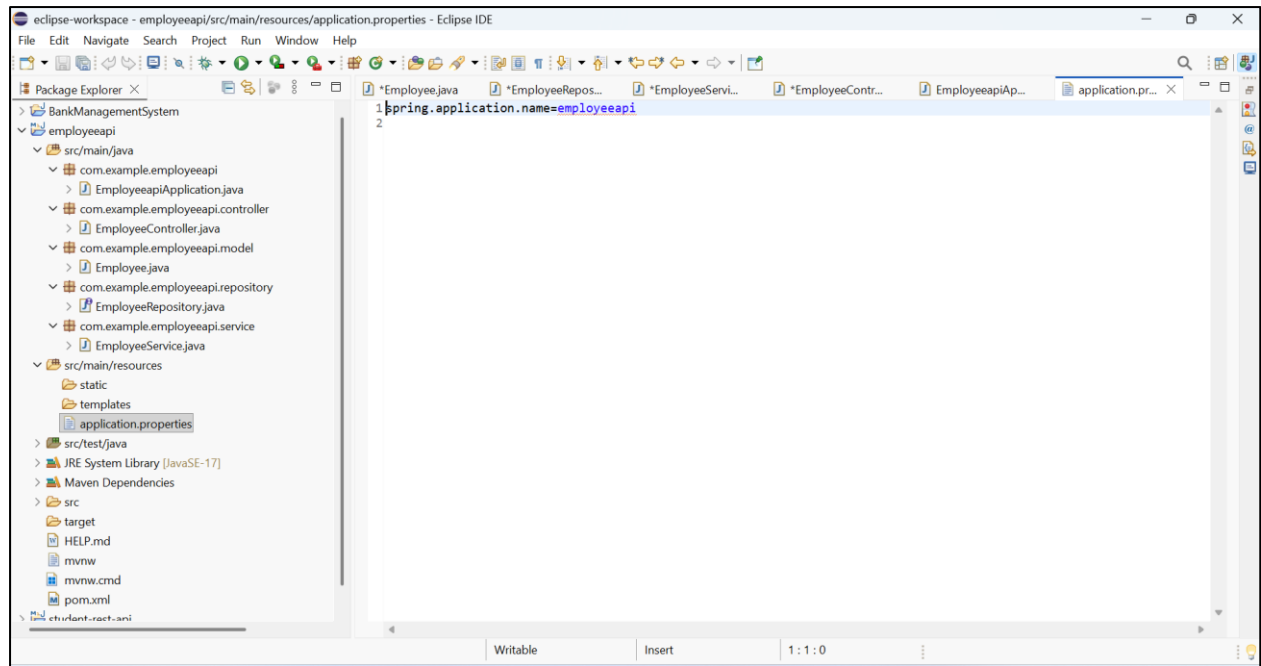
Update application.properties (Important)

Go to:

src/main/resources

Open:

application.properties



Add:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

This enables **in-memory database**.

Update Maven Dependencies

Open:

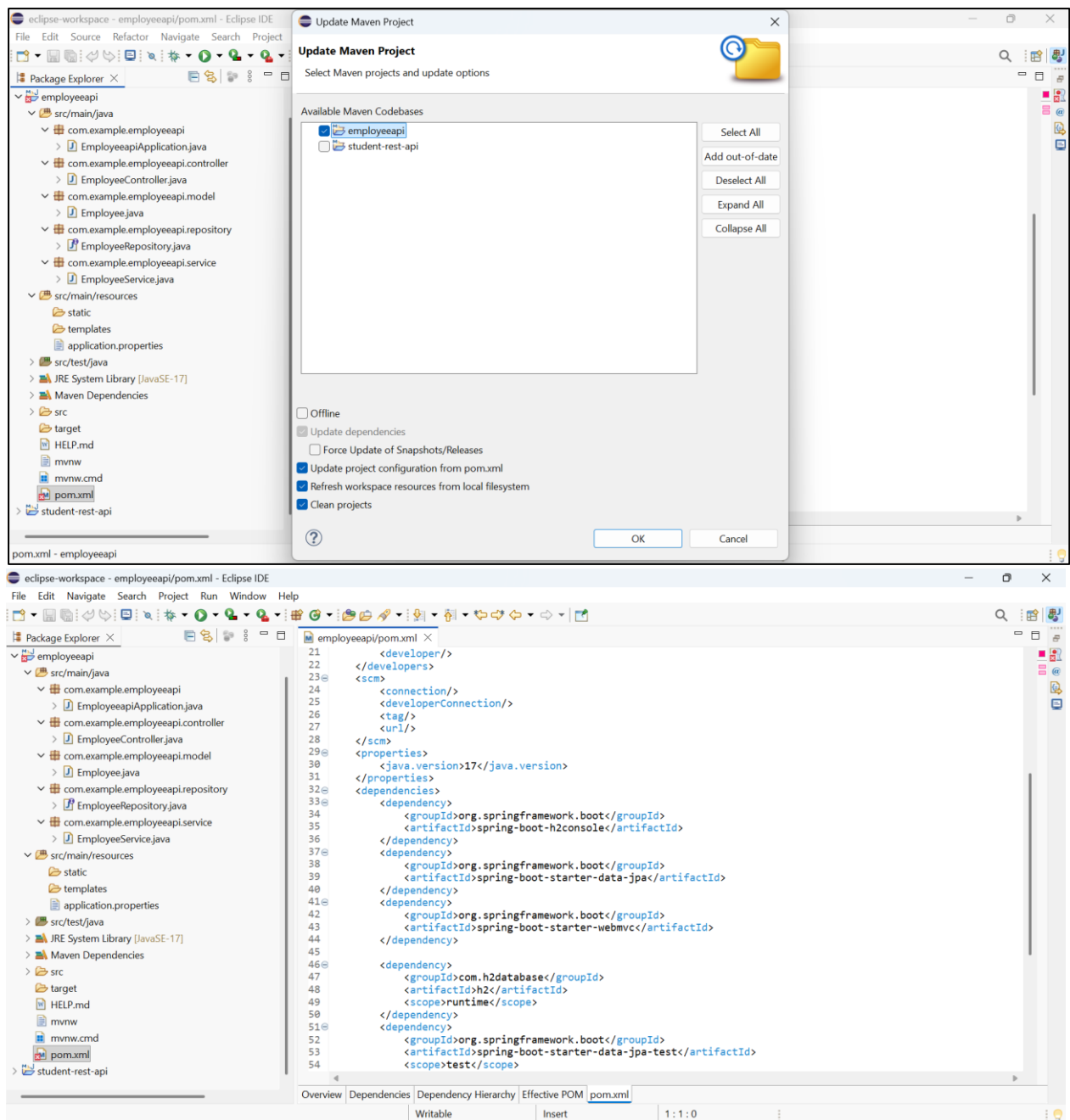
pom.xml

Check dependencies include:

Spring Web

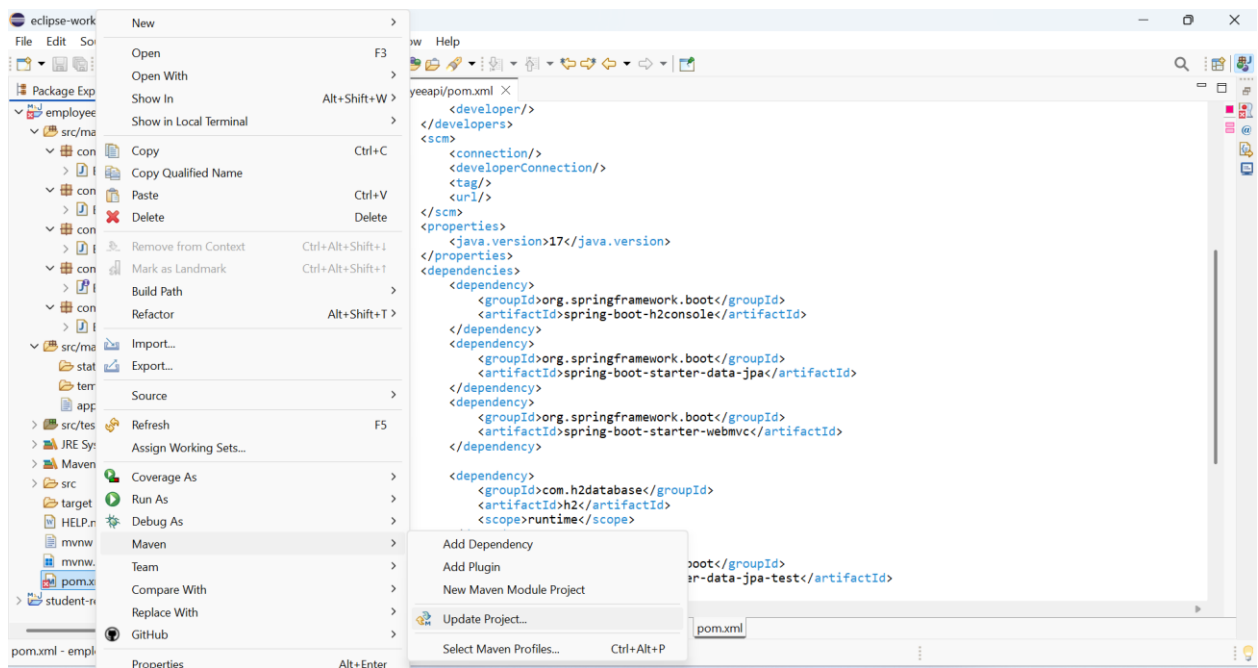
Spring Data JPA

H2 Database



Right click project →

Maven → Update Project



Run the Spring Boot Application

1. Go to **Project Explorer**
2. Open package:

com.example.employeeapi

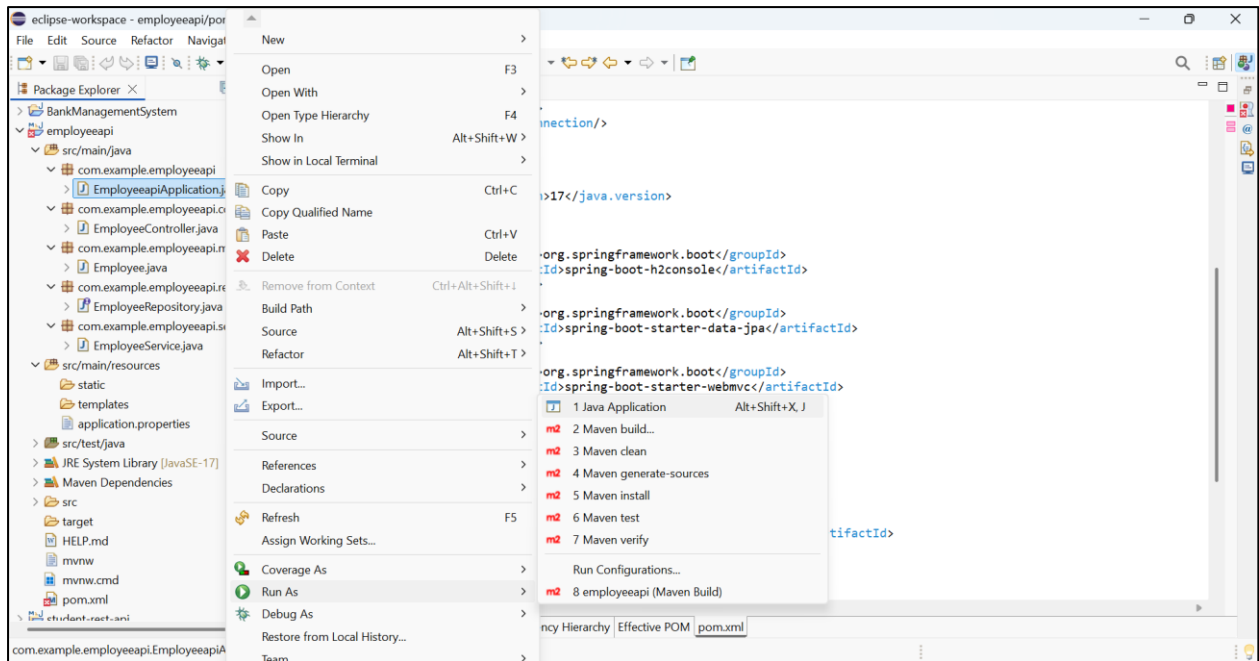
- Right-click:

EmployeeApiApplication.java

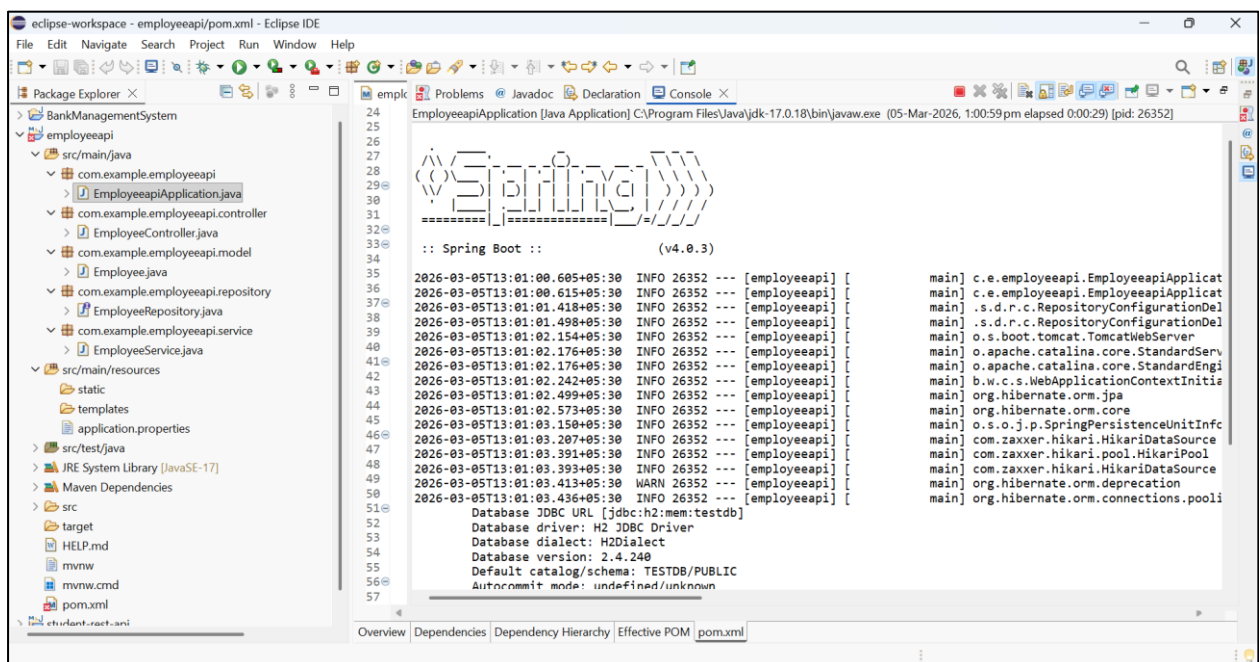
- Click:

Run As → Java Application

This will start the **Spring Boot** server.

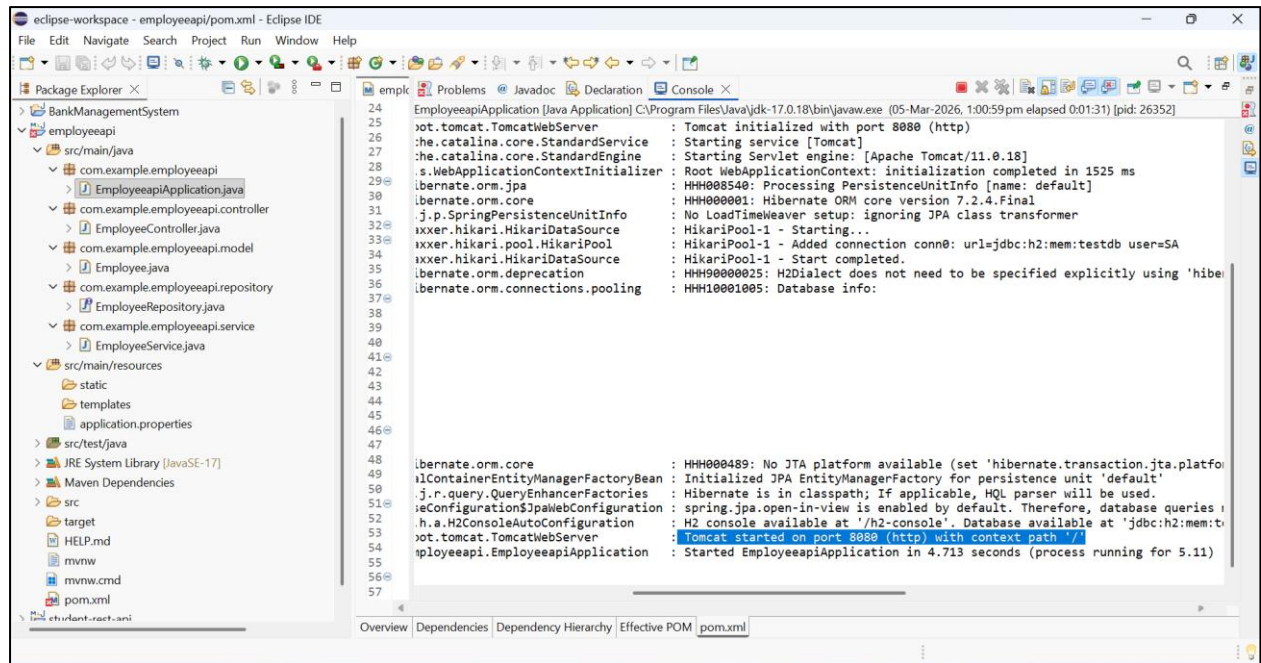


The server will start.



Console will show:

Tomcat started on port(s): 8080
Started EmployeeApiApplication



Check Using Browser (Simplest)

1. Open **Google Chrome / Edge**
2. In the address bar type:

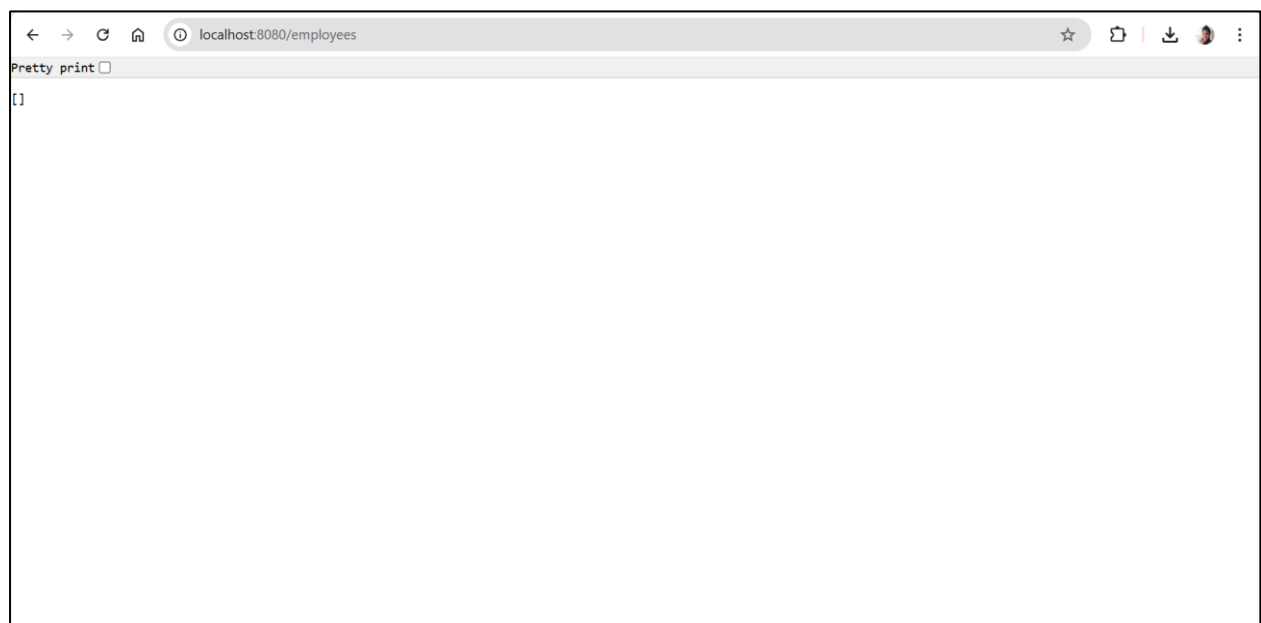
`http://localhost:8080/employees`

- Press **Enter**

Possible Output

If no employee is added yet:

[]



This means **API is working but database is empty.**

Testing API using Postman

Open postman

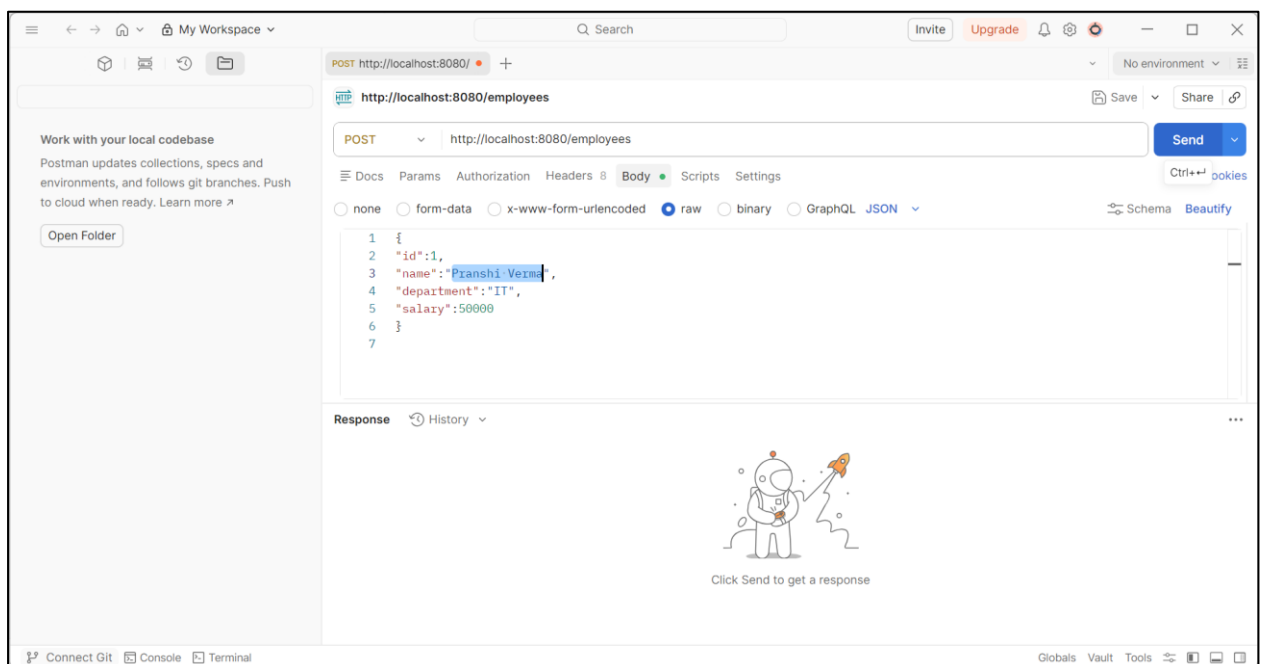
Create Employee (POST)

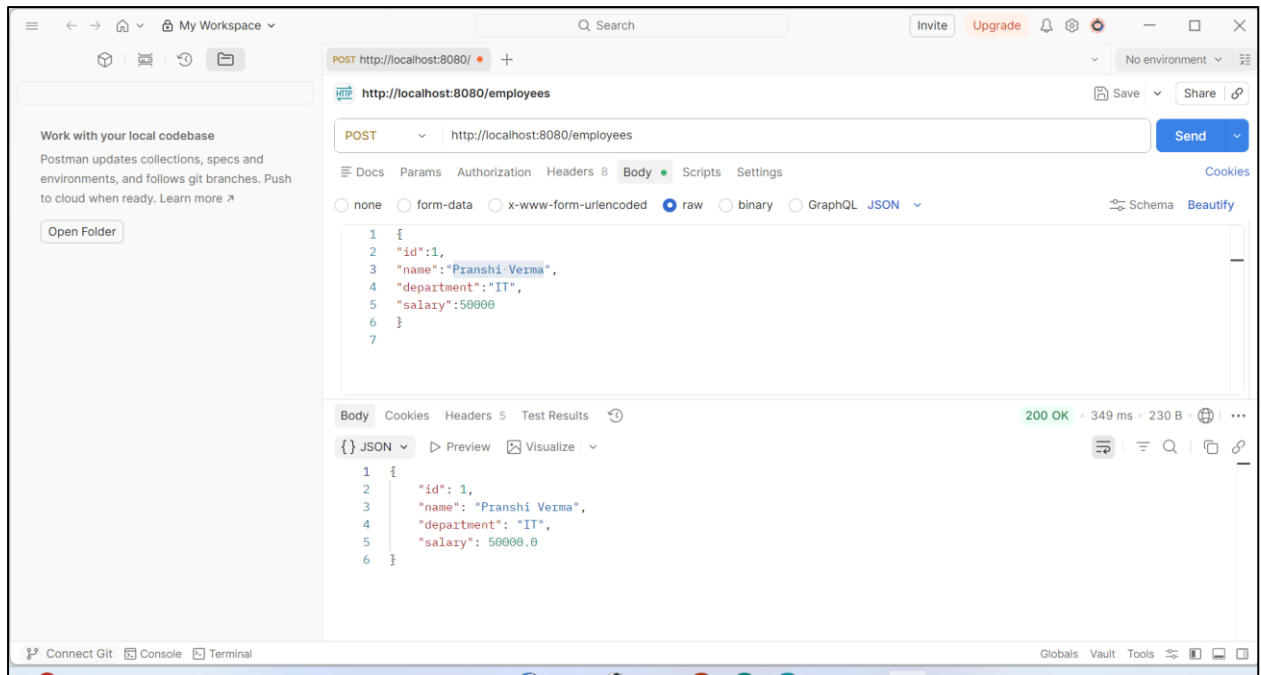
URL

`http://localhost:8080/employees`

Body

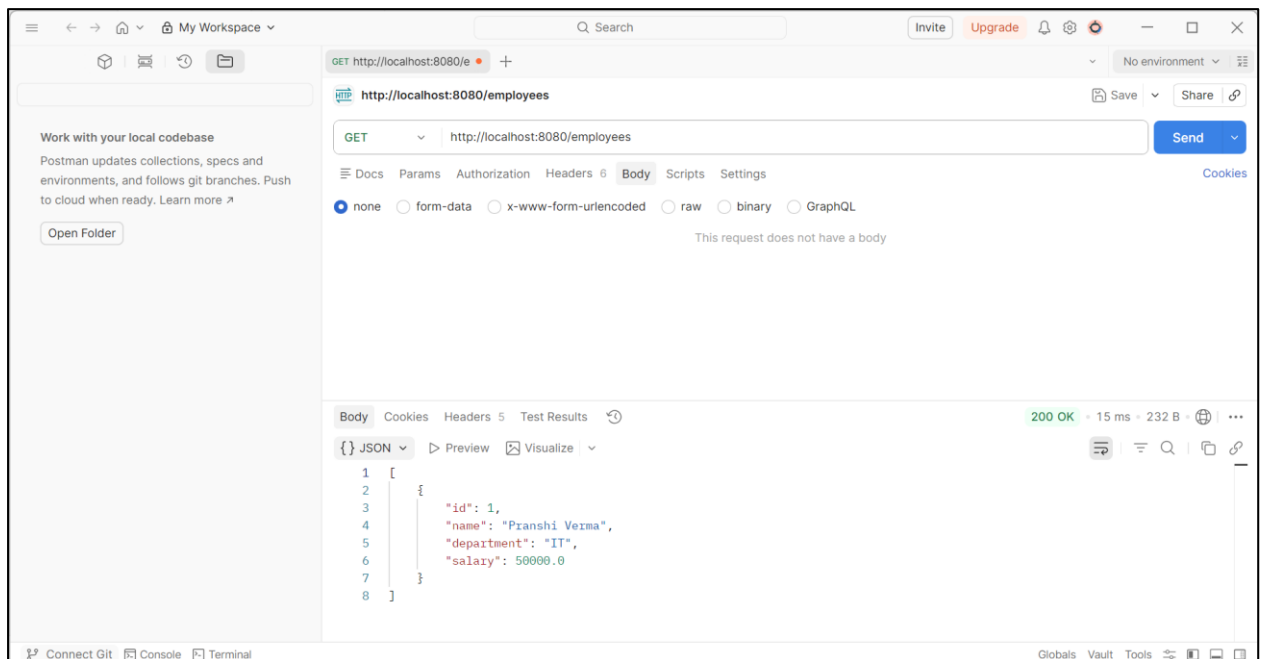
```
{  
  "id":1,  
  "name":"Pranshi Verma",  
  "department":"IT",  
  "salary":50000  
}
```





Get All Employees (GET)

<http://localhost:8080/employees>



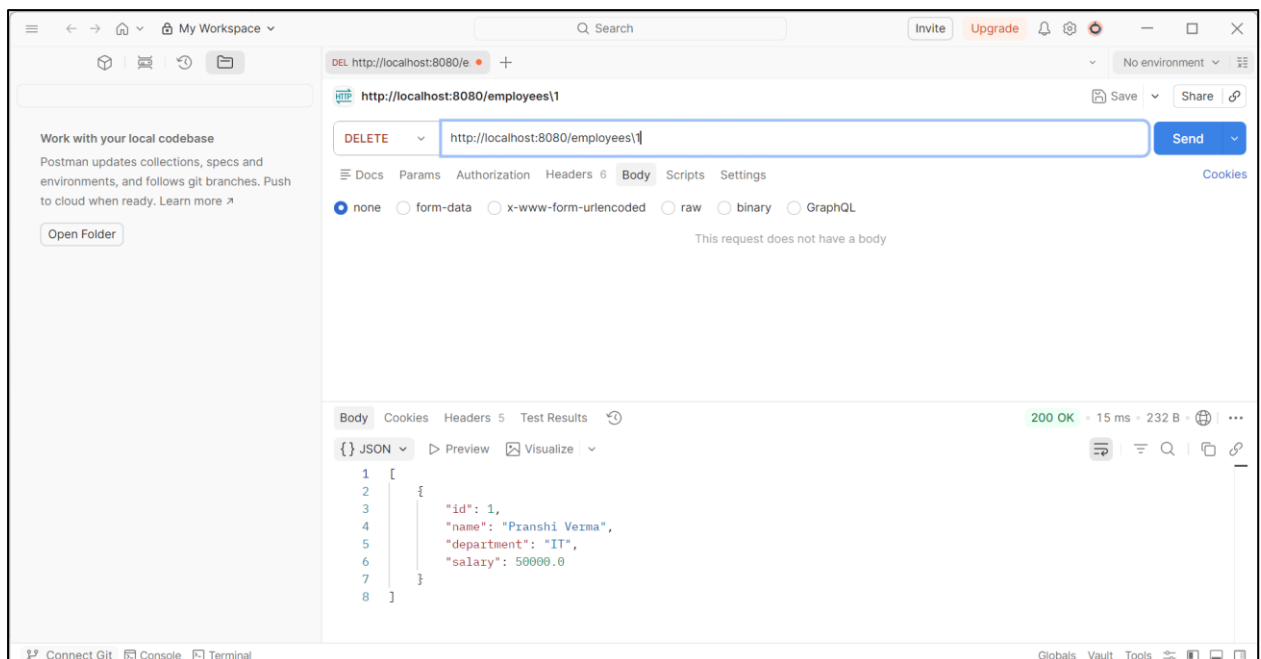
Get Employee by ID

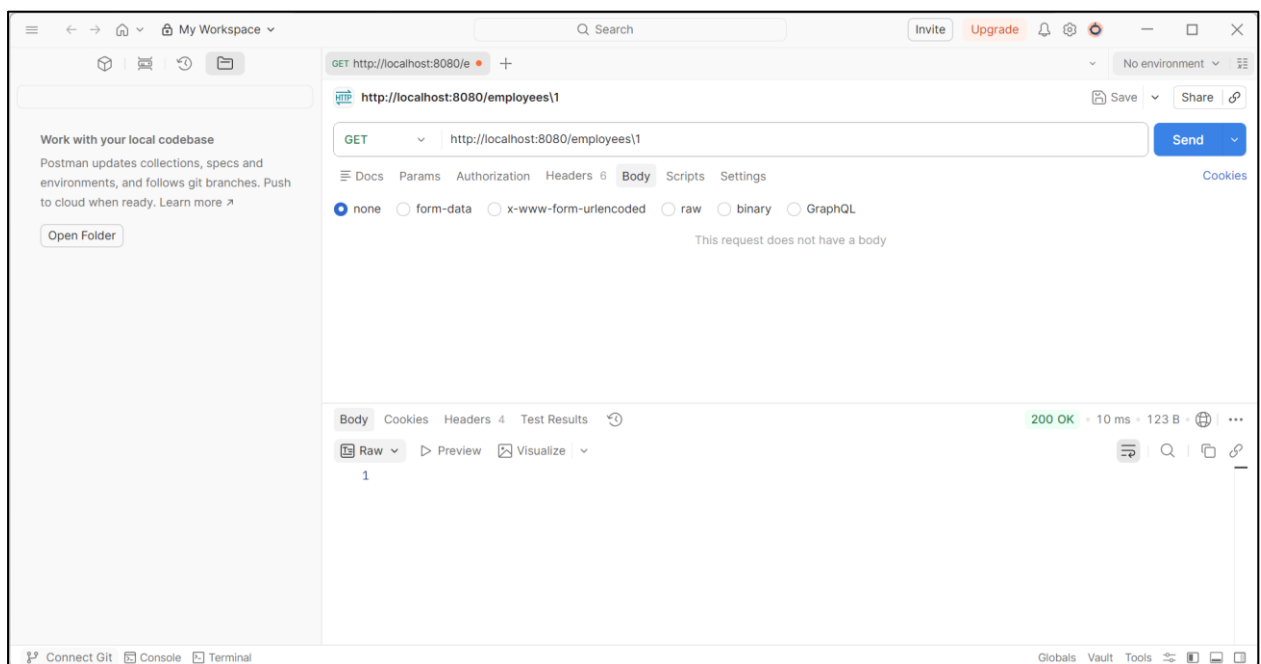
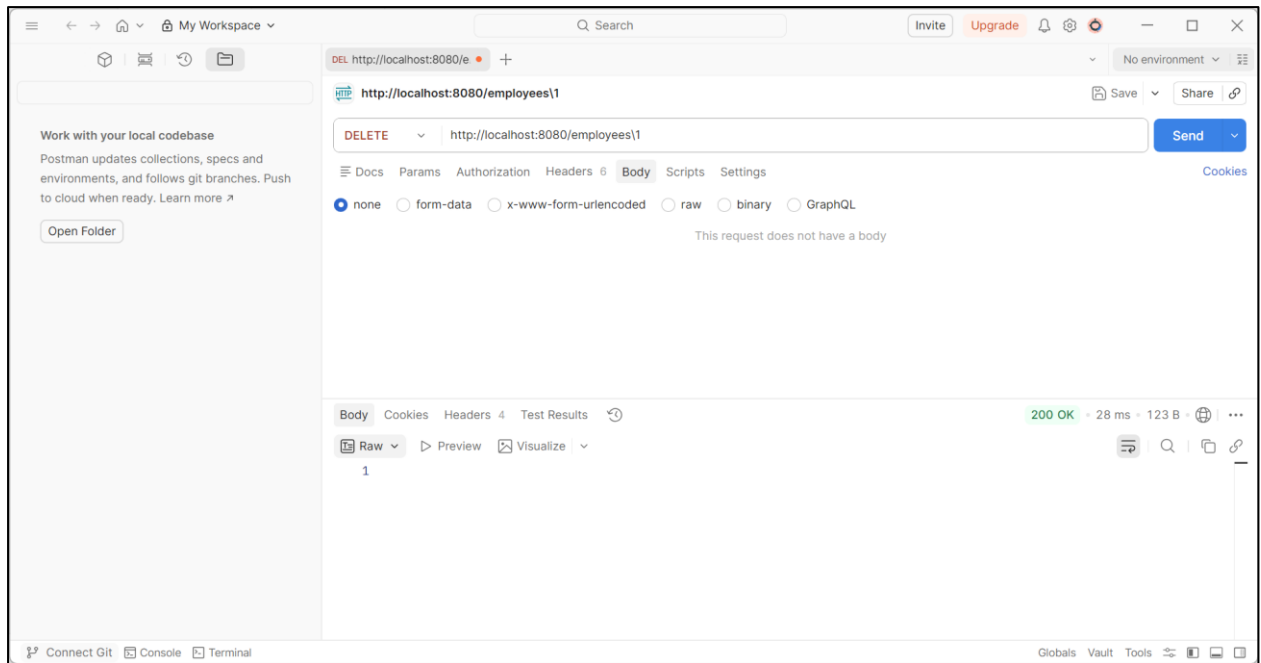
<http://localhost:8080/employees/1>



Delete Employee

<http://localhost:8080/employees/1>





No Data is available after successfully deletion.

Expected Outcome

Outcome:

Students successfully implemented **CRUD operations** using **Spring Boot REST APIs** and connected the application with a database using **Spring Data JPA**.